Microsoft® Windows

Software Development Kit

Programming Guide

Microsoft Corporation

Information in this document is subject to change without notice and does not represent a commitment on the part of Microsoft Corporation. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy this software on magnetic tape, disk, or any other medium for any purpose other than the purchaser's personal use.

© Copyright Microsoft Corporation, 1984, 1985, 1986

Microsoft®, the Microsoft logo, and MS-DOS® are registered trademarks of Microsoft Corporation.

IBM is a registered trademark of International Business Machines Corporation.

Document Number 050051052-103-I01-1086

Contents

1 Introduction 1

1.1 Overview 3
1.2 Application Development 3
1.3 Windows Applications 4
1.4 The Windows Display Screen 4

2 Creating Simple Applications 7

2.1 Introduction 9
2.2 The HELLO Application 9
2.3 What HELLO Looks Like 10
2.4 The Program Source: HELLO.C 12
2.5 The Module Definition File: HELLO.DEF 25
2.6 The Resource File: HELLO.RC 27
2.7 Putting HELLO Together 28
2.8 The TYPE Application 30
2.9 What TYPE Does 30
2.10 TYPE's Source Files 32
2.11 The Source File: TYPE.C 32
2.12 TYPE's Local Functions 37

3 Using GDI in Applications 43

3.1 Introduction 45
3.2 The SHAPES Application 45
3.3 How SHAPES Works 45
3.4 The SHAPES.C File 47
3.5 Processing Menu Input 53
3.6 Menus in a Resource File 59
3.7 The Default Menu for the Window Class 60
3.8 Defining an Accelerator Table 60
3.9 Loading and Using an Accelerator Table 61
3.10 The TRACK Application 62
3.11 How TRACK Works 62
3.12 The TRACK.C File 66
3.13 Setting the Coordinate System 73
3.14 Reading and Setting the Menu 74

3.15 Making the Drawing Box 75
3.16 Borrowing Data from the Previous Instance 77

4 Using Fonts and Dialog Boxes 79

4.1 Introduction 81
4.2 What FONTTEST Does 81
4.3 The Source File: FONTTEST.C 84
4.4 Reading FONTTEST's Menu 91
4.5 Displaying the Sample String 92
4.6 FONTTEST's Dialog Box 93
4.7 The Resource Script: FONTTEST.RC 97
4.8 The Module Definition File: FONTTEST.DEF 99

5 Creating Windows Applications 101

5.1 Introduction 103
5.2 Windows Applications 103
5.3 The Main Function 104
5.4 Messages and Queues 105
5.5 Window Messages 111
5.6 An Application as a Task 119
5.7 Terminating an Application 123

6 Creating and Using Windows 125

6.1 Introduction 127
6.2 The Standard Window 127
6.3 Window Classes 128
6.4 Tiled Windows 130
6.5 Creating a Tiled Window 131
6.6 Child Windows 131
6.7 Popup Windows 134
6.8 Window Styles 137
6.9 Painting Windows 138
6.10 Window Subclassing 144

7 Controls and Dialog Boxes 147

7.1 Introduction 149
7.2 Control Windows 149
7.3 Button Controls 150

7.4 Edit Controls 153
7.5 Static Controls 161
7.6 List Boxes 162
7.7 List Box Control Styles 163
7.8 Scroll Bar Controls 167
7.9 WS_TABSTOP and WS_GROUP Styles 168
7.10 Dialog Boxes 169
7.11 Default Responses in Message Boxes 174
8.10 Miscellaneous Rules 201

8 Using System Resources 175

8.1 Introduction 177
8.2 Memory Management 177
8.3 Resources 179
8.4 Dynamic Linking 180
8.5 Changing Menus 181
8.6 Using the System Menu 185
8.7 Using the Mouse Cursor When No Mouse Is Available 188
8.8 Rules About Disk Files 190
8.9 Using a Printer 194

9 The Clipboard 203

9.1 Introduction 205
9.2 Copying Data to the Clipboard 205
9.3 Reading Data From the Clipboard 206
9.4 Rendering Data on Request 207
9.5 Rendering Formats Before Terminating 207
9.6 Standard Data Formats 208
9.7 The Clipboard Viewer Application 209
9.8 Controlling the Display of Data in the Clipboard Viewer 209
9.9 Clipboard Viewer Chain 211
9.10 Metafile Pictures 212
9.11 Standards for Playing Metafile Pictures 216

Appendixes 219

A Creating Windows Libraries 221

A.1 Windows Libraries 223
A.2 Initializing the Local Heap For Libraries 223
A.3 Calling Routines in a Library 224

B Creating A Font Resource 225

B.1 Font Resources 227
B.2 Creating Font Files 227
B.3 Adding Fonts to Executable Files 227
B.4 Creating Font Resource-Only Files 228
B.5 Compiling and Linking the Font Resource 231

C GDI Definitions 233

C.1 Coordinate Systems 235
C.2 Transformation Equations 235
C.3 GDI Window and Viewport Functions 237
C.4 Mapping Modes 237
C.5 Completely Constrained 238
C.6 Partially Constrained: MM_ISOTROPIC 239
C.7 Unconstrained: MM_ANISOTROPIC 239
C.8 When to Use the Mapping Modes 239
C.9 GDI Transformation Defaults Values 240
C.10 Logical Screen Size 240
C.11 Default System Font 241
C.12 ANSI Fixed Font 242
C.13 Selecting Fonts 242
C.14 How the Font Mapper Works 243

Index 247

Figures

Figure 2.1 HELLO's Window 10

Figure 2.2 About... Command in the System Menu 11

Figure 2.3 About Dialog Box 11

Figure 2.4 TYPE's Window 30

Figure 2.5 Multiple Instances of TYPE 31

Figure 3.1 SHAPES's Star 45

Figure 3.2 SHAPE's Menu 46

Figure 3.3 A New Shape 47

Figure 3.4 TRACK's Menu 62

Figure 3.5 Track's Drawing Box 63

Figure 3.6 The Star 63

Figure 3.7 A Change of Selection 64

Figure 3.8 A New Shape 65

Figure 4.1 FONTTEST Window 81

Figure 4.2 Selecting the Options... Command 82

Figure 4.3 Dialog Box 82

Figure 4.4 Changing A Control 83

Figure 4.5 Character Height Increased 84

Chapter 1
Introduction

1.1 Overview 3
1.2 Application Development 3
1.3 Windows Applications 4
1.4 The Windows Display Screen 4

1.1 Overview

Microsoft® Windows is an extension to your computer's operating system that lets applications share the resources of your computer and interact with the user through a visual interface called a "window." Windows offers:

- Multiple, interactive windows for all applications
- Shared use of system resources such as memory, display screen, keyboard, and mouse
- Access to the Graphics Device Interface (GDI) for computer graphics and text
- Data exchange between applications
- Portability across all computers running Windows

Windows takes complete control of a computer's memory, display screen, keyboard, mouse, and timer. It then carefully manages these resources, sharing them between all running applications. It accesses system resources through a standard OEM interface, making it completely device and machine-independent. Thus, a Windows application that runs on one system will run on any other Windows system.

This manual explains how to create Windows applications. In particular, it describes the steps you need to take to produce a loadable and executable Windows module.

The following sections list the steps needed to develop a Windows application and provide an overview of Windows and Windows applications.

1.2 Application Development

Creating a Windows application from scratch involves the following steps:

1. Create any cursors, icons, or brushes needed using **Iconedit**, the icon editor.
2. Create descriptions for any menus or dialog boxes to be used by the application.

3. Create a module definition file.
4. Write the application source files, using the format and functions described in this guide.
5. Compile and link all sources using **Cc**, the C compiler, and **Link4**, the Windows linker.
6. Compile and add resources to the executable file using **Rc**, the Windows resource compiler.

Since most Windows applications are written in the C programming language, all examples in this manual are given in C. An explanation of the Assembly language macros used to create applications is given in the *Microsoft Windows Reference Manual.*

1.3 Windows Applications

A Windows application is any program that has been specifically designed to run under Windows. Windows applications use only the standard Windows functions to access system memory, the display screen, the keyboard, and the mouse. A Windows application interacts with the user through one or more windows that it creates and maintains. It interacts with floppy disk drives and other storage devices through standard MS-DOS system calls.

1.4 The Windows Display Screen

The Windows display screen has two parts: the work area and the icon area.

The work area in the central portion of the display screen provides space for all open windows. Whenever an application or user opens a window, Windows places the full-size window in this area. If other open windows are present, Windows rearranges and resizes them so that the new window and all previously open windows can be seen together. This special display technique is called "tiling." The user can direct Windows to change the current arrangement of windows in the work area, or even expand one window to full-screen, temporarily covering all other windows. Again, Windows rearranges and resizes windows as requested. Note that rearrangement and resizing is performed either by Windows, or by the user;

applications never need to worry about how to fit their windows in the work area.

The icon area, at the bottom of the screen, provides space for the icons of all windows that are not currently open. Windows and the user have complete control over the placement of icons. This means that, although an application can control the shape of an icon, it cannot control the placement of the icon in the icon area.

Chapter 2
Creating Simple Applications

2.1 Introduction 9
2.2 The HELLO Application 9
2.3 What HELLO Looks Like 10
2.4 The Program Source: HELLO.C 12
2.4.1 HELLO's Functions 16
2.4.2 Loading Strings 17
2.4.3 Registering the Window Class 18
2.4.4 Creating a Window 20
2.4.5 Changing the System Menu 21
2.4.6 The Main Loop 22
2.4.7 The Window Function 22
2.4.8 About Dialog Box 24
2.5 The Module Definition File: HELLO.DEF 25
2.6 The Resource File: HELLO.RC 27
2.7 Putting HELLO Together 28
2.8 The TYPE Application 30
2.9 What TYPE Does 30
2.10 TYPE's Source Files 32
2.11 The Source File: TYPE.C 32
2.12 TYPE's Local Functions 37
2.12.1 Carrying Out Early Tasks 38
2.12.2 Getting the Input Focus 38
2.12.3 Painting With a WM_PAINT Message 39

2.12.4 Painting Without a WM_PAINT Message 40
2.12.5 Using a System Font 41
2.12.6 Creating Multiple Instances 42

2.1 Introduction

This chapter illustrates how to create simple Window applications. It presents two sample applications named HELLO and TYPE. The applications presented here are similar to applications provided on the Windows Software Development Kit disks.

2.2 The HELLO Application

HELLO is a very simple Windows application that creates a window and displays the message "Hello Windows!" HELLO illustrates the files you need to create an application. It contains:

File	Contents
Source program file	Program statements. It can be written in the C programming language, in Pascal, or in assembly language.
Module definition file	Definition of the application in terms of segments, memory requirements, and exported and imported functions.
Resource script file	Filenames of the icon, cursor, and other resources used by the application.

The following sections explain the content and meaning of each file and show how to combine the files into a single program.

2.3 What HELLO Looks Like

HELLO, like all other Windows applications, creates and displays a tiled window. On the Windows display screen, HELLO's window looks like this:

Figure 2.1 HELLO's Window

The window has a caption bar at the top of the window. It contains the caption "Hello Windows!". Also in the caption bar is a system menu icon and a size box. The size box is used to adjust the size of the window. The user can click the system menu icon to reveal the system menu. This menu contains standard system commands plus an About... command added to the menu by HELLO.

Figure 2.2 About... Command in the System Menu

When the user selects the About... command, HELLO displays a dialog box containing information about the application.

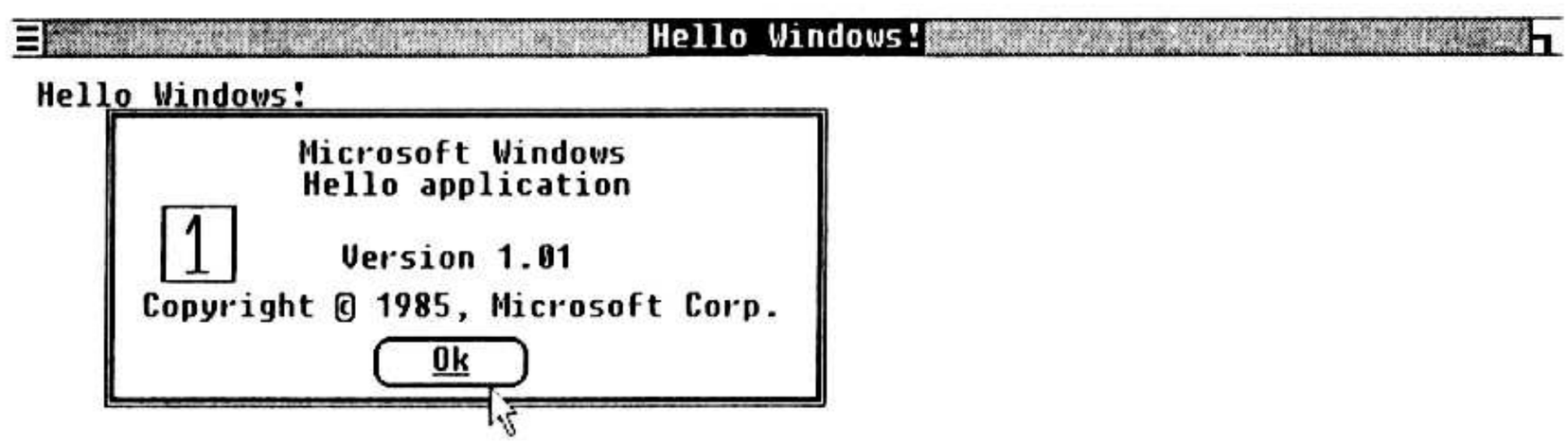


Figure 2.3 About Dialog Box

Just below the caption bar is the client area. The client area is the exclusive property of the application. An application can write any text or

draw any figure in the client area. For example, HELLO writes the message "Hello Windows!".

The window fills the entire screen except for the Windows icon area. When the window is just iconic (not terminated), its icon appears in the icon area along with the MS-DOS Executive icon.

2.4 The Program Source: HELLO.C

The HELLO.C file is a C language source file that contains the program source statements for HELLO. Every Windows application source must define:

File	Definition
A main function: WinMain	The main function is the program starting point. It prepares a window for the application and reads messages from the Windows queues. The main function must be named **WinMain**.
A window class	A window class is a template that defines the attributes of a window, such as the shape of the window's cursor and name of the window's menu. You need a window class to create windows for your application. You create the class by registering it with the **RegisterClass** function.
A window function: WndProc	The window function is the application's workhorse. You use it to process all input to your window. Input consists of messages that Windows and your main function have sent.
A window	A window is the chief input and output tool of every application. Users select commands from the window with the mouse, and enter data into the window with the keyboard. You create a window by using the **CreateWindow** function.

The following pages contain a copy of the actual contents of HELLO.C, and an explanation of the file.

```
/*  hello.c
    Hello Application
    Windows Premiere Edition
    Copyright (c) Microsoft 1985        */

#include "windows.h"
#include "hello.h"

char *szAppName;
char *szAbout;
char *szWindowTitle;
int TitleLength;

static HANDLE hInst;
FARPROC lpprocAbout;

long FAR PASCAL HelloWndProc(HWND, unsigned, WORD, LONG);

BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
 if (message == WM_COMMAND) {
     EndDialog( hDlg, TRUE );
     return TRUE;
     }
 else if (message == WM_INITDIALOG)
     return TRUE;
 else return FALSE;
}


HelloPaint( hDC )
HDC hDC;
{
 TextOut( hDC, 10, 10, (LPSTR)szWindowTitle, TitleLength );
}
```

```
int PASCAL WinMain( hInstance, hPrevInstance, lpszCmdLine, cmdShow )
HANDLE hInstance, hPrevInstance;
LPSTR lpszCmdLine;
int cmdShow;
{
 MSG   msg;
 HWND  hWnd;
 HMENU hMenu;

 /* Loading from string table */
 szAppName = (char *)LocalAlloc( LPTR, 10 );
 szAbout   = (char *)LocalAlloc( LPTR, 10 );
 szWindowTitle = (char *)LocalAlloc( LPTR, 15 );
 LoadString( hInstance, IDSNAME, (LPSTR)szAppName, 10 );
 LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );
 TitleLength = LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 15 );

 if (!hPrevInstance)
     if (!HelloInit( hInstance ))
         return FALSE;

 hWnd = CreateWindow((LPSTR)szAppName,
                     (LPSTR)szWindowTitle,
                     WS_TILEDWINDOW,
                     0,     /*  x - ignored for tiled windows */
                     0,     /*  y - ignored for tiled windows */
                     0,     /* cx - ignored for tiled windows */
                     0,     /* cy - ignored for tiled windows */
                     (HWND)NULL,       /* no parent */
                     (HMENU)NULL,   /* use class menu */
                     (HANDLE)hInstance, /* handle to window instance */
                     (LPSTR)NULL        /* no params to pass on */
                     );

 hInst = hInstance;
 lpprocAbout = MakeProcInstance( (FARPROC)About, hInstance);
 hMenu = GetSystemMenu(hWnd, FALSE);
 ChangeMenu(hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR);
 ChangeMenu(hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING);

 ShowWindow( hWnd, cmdShow );
 UpdateWindow( hWnd );

 while (GetMessage((LPMSG)&msg, NULL, 0, 0))
     {
     TranslateMessage((LPMSG)&msg);
     DispatchMessage((LPMSG)&msg);
     }
 exit( msg.wParam );
}
```

```
/* Procedure called when the application is loaded */
int HelloInit( hInstance )
HANDLE hInstance;
{
 PWNDCLASS   pHelloClass;

 pHelloClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS) );

 pHelloClass->hCursor        = LoadCursor( NULL, IDC_ARROW );
 pHelloClass->hIcon          = LoadIcon( hInstance, (LPSTR)szAppName );
 pHelloClass->lpszMenuName   = (LPSTR)NULL;
 pHelloClass->lpszClassName  = (LPSTR)szAppName;
 pHelloClass->hbrBackground  = GetStockObject( WHITE_BRUSH );
 pHelloClass->hInstance      = hInstance;
 pHelloClass->style          = CS_HREDRAW | CS_VREDRAW;
 pHelloClass->lpfnWndProc    = HelloWndProc;

 if (!RegisterClass( (LPWNDCLASS)pHelloClass ) )
     return FALSE;   /* Initialization failed */

 LocalFree( (HANDLE)pHelloClass );
 return TRUE;    /* Initialization succeeded */
}


/* Procedures which make up the window class. */
long FAR PASCAL HelloWndProc(hWnd, message, wParam, lParam)
HWND        hWnd;
unsigned    message;
WORD        wParam;
LONG        lParam;
{
 PAINTSTRUCT  ps;

 switch (message)
 {
 case WM_SYSCOMMAND:
     switch (wParam)
     {
     case IDSABOUT:
         DialogBox(hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout);
         break;
     default:
         return(DefWindowProc(hWnd, message, wParam, lParam));
         break;
     }
     break;

 case WM_DESTROY:
     PostQuitMessage(0);
     break;

 case WM_PAINT:
     BeginPaint(hWnd, (LPPAINTSTRUCT)&ps);
     HelloPaint(ps.hdc);
     EndPaint(hWnd, (LPPAINTSTRUCT)&ps);
```

```
        break;

    default:
        return(DefWindowProc(hWnd, message, wParam, lParam));
        break;
    }
    return(0L);
}
```

2.4.1 HELLO's Functions

HELLO has five functions: **WinMain**, **HelloInit**, **HelloWndProc**, **HelloPaint**, and **About**. The **WinMain** and **HelloWndProc** functions are HELLO's main and window functions, respectively. **HelloInit** and **HelloPaint** are local functions, used within HELLO only. **About** is the dialog box function that displays HELLO's About dialog box.

Windows calls HELLO's main function, **WinMain**, as soon as the application is loaded into memory. **WinMain** is the application's executive. It controls all subsequent execution of the application.

WinMain uses the **LocalAlloc** and **LoadString** functions to load the ANSI strings it will use during execution. These strings have been defined in the HELLO's resource script file, so HELLO can load when needed and discard them when not.

WinMain uses **HelloInit** to register a window class the first time HELLO is loaded. If the user creates a new instance of HELLO, **WinMain** assumes that the registered window class is still available, and does not attempt to register it again. The **hPrevInstance** parameter is not NULL only if the application has been previously loaded.

WinMain calls the **CreateWindow**, **ShowWindow**, and **UpdateWindow** functions to create and display a window for the application. It then enters a program loop to read messages from the Windows queues. Before displaying the window, HELLO uses the **GetSystemMenu** and **ChangeMenu** functions to add the About... command to the system menu.

Windows calls HELLO's window function, **HelloWndProc**, whenever a message has been sent to it by **WinMain** or by Windows itself. **HelloWndProc** examines each message using a **switch** statement and either processes the message or chooses a default action. **HelloWndProc** calls **HelloPaint** whenever it receives a WM_PAINT message. This is a request by Windows or by **WinMain** to paint the client area of the window. It calls the **DefWindowProc** function when it wants the default actions.

The **WinMain**, **HelloWndProc**, and **About** functions use the PASCAL calling convention. Since Windows calls these functions directly and always uses this convention, PASCAL is required. **HelloWndProc** and **About** also use the FAR keyword in their definitions since Windows uses a 32-bit address whenever it calls a function. (The main function, **WinMain**, does not need the FAR keyword.) Local functions, such as **HelloInit** and **HelloPaint**, can use any calling convention.

Note

The #**INCLUDE** statement at the beginning of HELLO.C defines the Windows include file, WINDOWS.H. This file is an ordinary C language source file that contains definitions for all of Windows special constants, data types, data structures, and functions. You should include this file in all your C language source files.

2.4.2 Loading Strings

Applications that use many strings can define them as resources and load them when needed using the **LoadString** function. HELLO loads the strings it uses for its window's class name, caption title, and About... command name as the first task in **WinMain**:

```
char *szAppName;
char *szAbout;
char *szWindowTitle;
int TitleLength;

/* Loading from string table */
szAppName = (char *)LocalAlloc( LPTR, 10 );
szAbout   = (char *)LocalAlloc( LPTR, 10 );
szWindowTitle = (char *)LocalAlloc( LPTR, 15 );
LoadString( hInstance, IDSNAME, (LPSTR)szAppName, 10 );
LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );
TitleLength = LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 15 );
```

The function first allocates space for the strings using the **LocalAlloc** function. This function allocates space in HELLO's local heap and returns a pointer to this space. Each pointer is assigned to a variable, **szAppName**, **szAbout**, and **szWindowTitle**, and used in a **LoadString** function to indicate where the strings are to be loaded. Each string is identified by a

constant: IDSNAME, IDSABOUT, and IDSTITLE. These constants, defined in the HELLO.H include file, are associated with the strings in HELLO's resource script file, HELLO.RC:

```
STRINGTABLE
BEGIN
    IDSNAME,     "Hello"
    IDSABOUT,    "About..."
    IDSTITLE,    "Hello Windows!"
END
```

2.4.3 Registering the Window Class

Any application that intends to use a window must register a window class. You register a class by passing a data structure containing information about the class to the **RegisterClass** function.

HELLO registers its window class in a local function named **HelloInit**:

```
/* Procedure called when the application is loaded */
int HelloInit( hInstance )
HANDLE hInstance;
{
 PWNDCLASS   pHelloClass;

 pHelloClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS) );

 pHelloClass->hCursor         = LoadCursor( NULL, IDC_ARROW );
 pHelloClass->hIcon           = LoadIcon( hInstance, (LPSTR)szAppName );
 pHelloClass->lpszMenuName    = (LPSTR)NULL;
 pHelloClass->lpszClassName   = (LPSTR)szAppName;
 pHelloClass->hbrBackground   = GetStockObject( WHITE_BRUSH );
 pHelloClass->hInstance       = hInstance;
 pHelloClass->style           = CS_HREDRAW | CS_VREDRAW;
 pHelloClass->lpfnWndProc     = HelloWndProc;

 if (!RegisterClass( (LPWNDCLASS)pHelloClass ) )
     return FALSE;   /* Initialization failed */

 LocalFree( (HANDLE)pHelloClass );
 return TRUE;    /* Initialization succeeded */
}
```

The function allocates temporary storage for the class data structure using the **LocalAlloc** function, and assigns the structure's address to the pointer variable **pHelloClass.** The **LocalAlloc** function allocates memory from the application's local heap. The local heap is reserved exclusively for the application. You define its size when you create the application's module definition file.

Once the structure is available, **HelloInit** assigns initial values to the structure's members. Each member defines some attribute of the window class.

Member	Purpose
hCursor	Defines the shape of the mouse cursor. You can load a cursor resource from the executable file, or use one of Windows' standard cursors. HELLO uses the standard Windows arrow, IDC_ARROW.
hIcon	Defines the shape of the window icon. You can load an icon resource from the executable file, or use one of Windows' standard icons. HELLO loads its own icon, named "Hello."
lpszMenuName	Defines the name of the window's menu. The menu contains a list of commands that the user can select with the mouse. HELLO does not use a menu, so it assigns NULL to this member.
lpszClassName	Defines the name of the class. Every class needs a unique name to distinguish it from other classes. HELLO names its class "Hello."
hbrBackground	Defines the color of the brush used to paint the window's background. HELLO uses a stock white brush, WHITE_BRUSH. The white brush is just one of several stock brushes that you can use for the window background.
hInstance	Defines the module instance that first defines the window class. A module instance is an invocation of an application. Each time the user invokes HELLO, Windows creates a new instance. The instance handle **hInstance** is passed to **HelloInit** by **WinMain.**
style	Defines the window class style. HELLO uses the CS_VREDRAW and CS_HREDRAW styles. These direct Windows to ask HELLO to redraw its window whenever the window's vertical or horizontal measurements change.
lpfnWndProc	Defines the class's window function. HELLO's window function is named **HelloWndProc.**

After the members have been assigned values, **HelloInit** calls the **RegisterClass** function. This function registers the class if it is

successful. Otherwise, it returns FALSE. If Windows cannot register a class, HELLO returns immediately to **WinMain.**

Since Windows keeps its own copy of a registered window, **HelloInit** can delete the temporary structure **pHelloClass.** You can free storage that you have allocated from the local heap by using the **LocalFree** function.

2.4.4 Creating a Window

An application should create and display a window as soon as it starts. You can create a window using the **CreateWindow** function and display it using the **ShowWindow** function.

```
hWnd = CreateWindow((LPSTR)szAppName,
                    (LPSTR)szWindowTitle,
                    WS_TILEDWINDOW,
                    0,     /*  x - ignored for tiled windows */
                    0,     /*  y - ignored for tiled windows */
                    0,     /* cx - ignored for tiled windows */
                    0,     /* cy - ignored for tiled windows */
                    (HWND)NULL,       /* no parent */
                    (HMENU)NULL,  /* use class menu */
                    (HANDLE)hInstance, /* handle to window instance */
                    (LPSTR)NULL        /* no params to pass on */
                    );

hInst = hInstance;

        .
        .
        .

ShowWindow( hWnd, cmdShow );
UpdateWindow( hWnd );
```

CreateWindow takes several parameters that define what the window is, what it contains. The parameters, in order, are:

Parameters	Purpose
Class Name	The name of the window's window class. HELLO uses its own class, "Hello."
Caption Text	The character string to appear in the windows caption bar. HELLO uses "Hello Windows!."
Window Style	A predefined window style. HELLO needs a tiled window, so it uses the WS_ TILEDWINDOW style.

Parent Window	The handle of the window's parent window. HELLO's tiled window has no parent, so (HWND)NULL is given.
Menu Handle	The handle of the menu to use in place of the class menu. HELLO wants the class menu (which is no menu), so (HMENU)NULL is given.
Module Instance	The handle of the module instance to own the window. HELLO wants the current instance to own the window.
Additional Data	A long pointer to data to be used when the window is created. HELLO has no additional data, so (LPSTR)NULL is given.

The **ShowWindow** function directs Windows to display the new window. **WinMain** should call **ShowWindow** soon after creating the window, and should pass the **cmdShow** parameter to it. The **cmdShow** parameter to **WinMain** is supplied by Windows and defines how the window is displayed: as an open window, or as an icon.

The **UpdateWindow** function is used to direct Windows to call the window function with a WM_PAINT message. This lets HELLO paint the contents of the window's client area immediately after the window is displayed. **HelloWndProc** calls **HelloPaint** when it receives a WM_PAINT message. **HelloPaint** uses the **TextOut** function to write the message "Hello Windows!" in the upper left corner of the client area.

2.4.5 Changing the System Menu

Applications can add commands to the system menu by using the **GetSystemMenu** and **ChangeMenu** functions:

```
hMenu = GetSystemMenu(hWnd, FALSE);
ChangeMenu(hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR);
ChangeMenu(hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING);
```

The **GetSystemMenu** function returns a handle to the system menu. The **ChangeMenu** functions use this handle to identify the menu. The first **ChangeMenu** function appends a separator to the menu. This is the solid horizontal bar that separates the About... command name from the rest of the system command names. The second **ChangeMenu** function appends the About.. command name. The command is assigned a constant value, IDSABOUT, which will be used later to identify the command.

Commands should be added to the system menu before showing the window.

2.4.6 The Main Loop

Once **WinMain** has created and displayed a window, it can begin its primary duty: to read messages from the application queue and dispatch them to the appropriate window.

Windows does not send input directly to an application. Instead, it places all mouse, keyboard, and timer input into an application queue along with messages sent to the application by Windows and other applications. It is the application's responsibility to read the application queue, retrieve the messages that are for it, and dispatch them so the appropriate window function can process them. **WinMain** does this by using the **GetMessage**, **TranslateMessage**, and **DispatchMessage** functions in the following program loop:

```
while ( GetMessage((LPMSG)&msg, NULL, 0, 0) )
    {
    TranslateMessage((LPMSG)&msg);
    DispatchMessage((LPMSG)&msg);
    }
```

GetMessage reads the next message from a queue, **TranslateMessage** translates the message and generates additional messages if it is from the keyboard, and **DispatchMessage** sends the message to the window function. This loop continues until **GetMessage** returns FALSE. **GetMessage** makes **WinMain** wait if no message is available. While it is waiting, Windows lets other applications execute.

2.4.7 The Window Function

The window function carries out all work that directly affects HELLO's window. It receives messages from Windows and **WinMain** through the parameters **message**, *wParam*, and *lParam*. The **message** parameter defines the message type, and *wParam* and *lParam* contain additional information about the message.

HelloWndProc processes each message with a **switch** statement. It responds to only three Windows messages: WM_PAINT, WM_DESTROY, and WM_SYSCOMMAND. All other messages are passed to the default function, **DefWindowProc**, which carries out default actions for unwanted messages.

```
/* Procedures which make up the window class. */
long FAR PASCAL HelloWndProc(hWnd, message, wParam, lParam)
HWND        hWnd;
unsigned    message;
WORD        wParam;
LONG        lParam;
{
 PAINTSTRUCT  ps;

 switch (message)
 {
 case WM_SYSCOMMAND:
     switch (wParam)
     {
     case IDSABOUT:
         DialogBox(hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout);
         break;
     default:
         return(DefWindowProc(hWnd, message, wParam, lParam));
         break;
     }
     break;

 case WM_DESTROY:
     PostQuitMessage(0);
     break;

 case WM_PAINT:
     BeginPaint(hWnd, (LPPAINTSTRUCT)&ps);
     HelloPaint(ps.hdc);
     EndPaint(hWnd, (LPPAINTSTRUCT)&ps);
     break;

 default:
     return(DefWindowProc(hWnd, message, wParam, lParam));
     break;
 }
 return(0L);
}
```

HelloWndProc receives the WM_DESTROY message when the user requests Windows to terminate HELLO. The message gives HELLO the opportunity to finish its processing before being terminated. The **PostQuitMessage** returns an exit code to Windows.

HelloWndProc receives a WM_PAINT message when **WinMain** calls **UpdateWindow** and whenever the user moves or resizes HELLO's window. The **BeginPaint** function retrieves information about the window's client area. In particular, it returns a display context (DC) that can be used in any GDI graphics function to paint in the client area. This display context is passed to **HelloPaint**. The **EndPaint** function signals Windows that **HelloWndProc** has finished painting.

HelloWndProc receives a WM_SYSCOMMAND message when the user selects a system command from the system menu. **HelloWndProc** examines the **wParam** parameter to determine which command was selected. If the parameter is equal to IDSABOUT (the About... command's constant identifier), **HelloWndProc** calls the **DialogBox** function to display HELLO's About dialog box. Otherwise, the **DefWindowProc** is called to carry out the system command.

HelloWndProc must let messages that it does not use be processed by **DefWindowProc**. This ensures that any special actions that affect the window, the application, or Windows itself can be carried out.

HelloWndProc should return zero to indicate success. Windows expects a long (32-bit) return value, so a constant zero should be cast as a long: e.g., "0L" or "(long)0." If **DefWindowProc** processes a message, its return value should be returned.

2.4.8 About Dialog Box

An application that uses a dialog box must create a dialog box template that defines the appearance and content of the dialog box and a dialog box function to process user input to the dialog box. HELLO's dialog box function is **About**. Its dialog box template is defined in the resource script file HELLO.RC.

The **About** function, like other dialog box functions, has the same parameters as a window function, but only receives messages that are specific to the dialog box. Unlike window functions, **About** usually processes only user input messages, like WM_COMMAND, and does not have to send unprocessed messages to the **DefWindowProc** function:

```
BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
 if (message == WM_COMMAND) {
      EndDialog( hDlg, TRUE );
      return TRUE;
      }
 else if (message == WM_INITDIALOG)
      return TRUE;
 else return FALSE;
}
```

About processes two messages, WM_COMMAND and WM_INITDIALOG. The WM_INITDIALOG message is sent to a dialog box function by

Windows to let the function carry out special actions before displaying the dialog box. The WM_COMMAND message is a result of user input. **About** responds to all user input in the same way, it calls the **EndDialog** function, which directs Windows to remove the dialog box and continue execution of the application. The **EndDialog** function is used to terminate modal dialog boxes, that is, boxes created using the **DialogBox** function. **About**, like all other dialog box functions, returns TRUE if it processes a message, and returns FALSE if it does not.

A dialog box template defines the placement and size of the dialog box, the placement and size of controls in the dialog box, and the style, function, and identifying numbers or names of the dialog box and controls. HELLO has the following dialog box template:

```
ABOUTBOX DIALOG 22, 17, 144, 75
STYLE WS_POPUP | WS_DLGFRAME
BEGIN
    CTEXT "Microsoft Windows"                    -1, 37,  5, 68, 8
    ICON  "hello"                                -1,  9, 23,  0, 0
    CTEXT "Hello application"                    -1,  0, 14, 144, 8
    CTEXT "Version 1.01"                         -1, 38, 34, 64, 8
    CTEXT "Copyright © 1985, Microsoft Corp."    -1,  5, 47,132, 9
    DEFPUSHBUTTON "Ok"                          IDOK, 53, 59, 32, 14, WS_GROUP
END
```

The **DIALOG** statement starts the dialog box definition. The constant ABOUTBOX identifies the template when the **DialogBox** function is used to create the dialog box. A string can also be used to identify a template. The boxes upper left corner is placed at the point (22,17) in HELLO's client area. The box is 144 units wide by 75 units high.

The **STYLE** statement defines the dialog box style. It is a popup window with a framed border. This is the typical style used with modal dialog boxes. The **BEGIN** and **END** statements mark the beginning and end of the control definitions. The dialog contains text, an icon, and a default pushbutton. The pushbutton lets the user send input to the **About** function to terminate the dialog box.

2.5 The Module Definition File: HELLO.DEF

The HELLO.DEF file is a module definition source file. It defines the name, segments, memory requirements, and exported functions of HELLO. Every Windows application needs a module definition file.

HELLO.DEF contains the following statements:

```
NAME     Hello

DESCRIPTION 'Simple Microsoft Windows Application'

STUB     'WINSTUB.EXE'

CODE     MOVEABLE
DATA     MOVEABLE MULTIPLE

HEAPSIZE  1024
STACKSIZE 4096

EXPORTS
    HelloWndProc @1
    About        @2
```

The **NAME** statement defines the name of the application. This name is used by Windows to identify the application when it is loaded with other applications. The **NAME** statement is required.

The **DESCRIPTION** statement is an optional statement that places the message "Simple Microsoft Windows Application" into the application's executable file. This statement is typically used to add version control or copyright information to the file.

The **STUB** statement is another optional file that defines the MS-DOS executable stub to be placed at the beginning of the file. This executable stub displays a warning message and terminates the application if the user attempts to run it without Windows.

The **CODE** statement defines the memory requirements of that application's code segment. The code segment contains the executable code generated when the HELLO.C file is compiled. In HELLO, the code segment is MOVEABLE. If the application is not running and Windows needs additional space in memory, Windows can move the code segment to a better location. If you do not want a moveable code segment, you can use the FIXED keyword instead.

The **DATA** statement defines the memory requirements of the application's data segment. The data segment contains storage space for all the static variables declared in HELLO.C. It also contains space for the program stack and local heap. The data segment, like the code segment, is MOVEABLE. The MULTIPLE keyword directs Windows to create a new data segment for the application each time the user starts a new instance of the application. Although all instances share the same code segment, each has its own data segment. Applications must have the MULTIPLE keyword.

The **HEAPSIZE** statement defines the size in bytes of the application's local heap. HELLO uses its heap to allocate the temporary structure used to register the window class, so it specifies 1024 bytes of storage. Applications that use the local heap frequently should specify even greater amounts.

The **STACKSIZE** statement defines the size in bytes of the application's stack. The stack is used for temporary storage of function arguments. Any application, like HELLO, that calls its own local function must have a stack. HELLO specifies 4096 bytes of stack storage.

The **EXPORTS** statement defines the names and ordinal values of the functions to be exported by the application. HELLO exports its window function, **HelloWndProc**, which has ordinal value 1 (i.e., it is the first function to be exported), and the dialog box function, **About**, which has ordinal value 2. Windows requires that all functions, except **WinMain**, that are to be called by Windows must be exported in this way.

2.6 The Resource File: HELLO.RC

The HELLO.RC file is a resource script file. It names the file containing the resources to be used by the application. It can also contain definitions for menus and dialog boxes.

HELLO uses three resources: an icon, a string table, and a dialog box template:

```
#include "windows.h"
#include "hello.h"

hello  ICON    hello.ico

ABOUTBOX DIALOG 22, 17, 144, 75
STYLE WS_POPUP | WS_DLGFRAME
BEGIN
    CTEXT "Microsoft Windows"                -1, 37,  5, 68, 8
    ICON  "hello"                            -1,  9, 23,  0, 0
    CTEXT "Hello application"                -1,  0, 14, 144, 8
    CTEXT "Version 1.01"                     -1, 38, 34, 64, 8
    CTEXT "Copyright © 1985, Microsoft Corp."   -1,  5, 47,132, 9
    DEFPUSHBUTTON "Ok"                       IDOK, 53, 59, 32, 14, WS_GROUP
END
```

```
STRINGTABLE
BEGIN
    IDSNAME,     "Hello"
    IDSABOUT,    "About..."
    IDSTITLE,    "Hello Windows!"
END
```

The icon resource, "hello", has type ICON and is in the icon file named HELLO.ICO. HELLO loads this icon when it registers the window class. You can make your own icon, cursor, and bitmap files by using the Windows **Iconedit** program. See the *Windows Programmer's Utility Guide* for details.

The **STRINGTABLE** statement marks the beginning of the string table. This table contains the three ANSI strings used in HELLO. The **DIALOG** statement marks the beginning of the dialog box template definition. The definition specifies the appearance and content of the About dialog box.

The #**include** directives at the beginning of the file cause the constant definitions in the files WINDOWS.H and HELLO.H to be read and processed when the resource script file is processed. These files contain definitions for constants such as IDSNAME, WS_POPUP, and ABOUTBOX.

2.7 Putting HELLO Together

Once you have the source files, you can make HELLO's executable file, HELLO.EXE. To make HELLO.EXE, you must:

1. Use the Microsoft C compiler, **Cc**, to compile the HELLO.C file.
2. Use the Windows linker, **Link4**, to link the HELLO.OBJ object file with the Windows library and the module definition file, HELLO.DEF.
3. Use the Windows resource compiler, **Rc**, to add the resources specified by HELLO.RC to the executable file.

The Microsoft program maintainer, **Make**, can be used to invoke each program automatically. This program takes a description file containing the command lines to use and a list of files to be processed. For example, the following is the description file for HELLO:

```
hello.res: hello.rc hello.ico hello.h
    rc -r hello.rc

hello.obj: hello.c hello.h
    cc -d -c -Asnw -Gsw -Os -Zdpe hello.c

hello.exe: hello.obj hello.res hello.def
    link4 hello,hello/align:16,hello/map, slibw swlibc/NOD, hello.def
    mapsym hello
    rc hello.res
```

The first two lines direct **Make** to create a compiled resource file, HELLO.RES, if the resource script file, HELLO.RC, has been updated. The **-r** option of the **Rc** command creates a compiled resource file without attempting to add it to an executable file.

The next two lines direct **Make** to create the HELLO.OBJ file if HELLO.C or HELLO.H has a more recent access date than the current HELLO.OBJ file. The **Cc** command takes several command line options that prepare the application for execution under Windows.

The last four lines direct **Make** to create the HELLO.EXE file if any one of HELLO.OBJ, HELLO.RES, or HELLO.DEF has a more recent access date than the current HELLO.EXE file. Small Windows applications, like HELLO, must be linked with the Windows **SLIBW.LIB** and **SWLIBC.LIB** libraries. The object file, HELLO.OBJ, and the module definition file, HELLO.DEF, are used as arguments in the **Link4** command. HELLO.MAP is the implied argument of the **Mapsym** command. **Mapsym** creates a symbol file, HELLO.SYM, that can be used to debug the application. **Rc** automatically appends the compiled resources in HELLO.RES to the executable file HELLO.EXE.

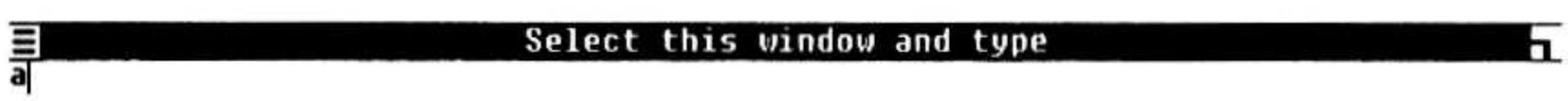
2.8 The TYPE Application

TYPE is another simple Windows application. It illustrates how to read and display input from system keyboard. It also demonstrates how to use the following:

Keyboard Input	TYPE receives and processes keyboard input whenever it is the active window.
The Input Focus	TYPE gets the input focus, or current ownership of the keyboard, when it becomes the active window. Only one application instance at a time can have the input focus.
Multiple Instances	TYPE, like other applications, can be invoked more than once. Multiple instances of an application run as separate programs, but share the same program code.

2.9 What TYPE Does

TYPE creates and displays a window in which the user can type a single line of text. TYPE's window looks like this:



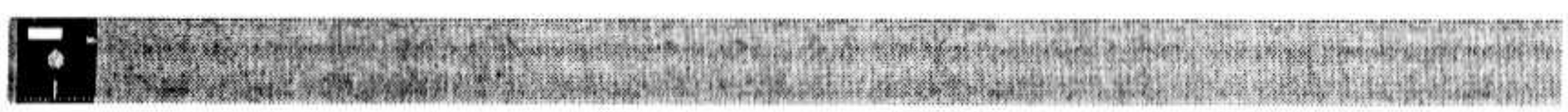


Figure 2.4 TYPE's Window

The window has a caption bar, system menu, and size box like other applications.

To enter text in the window, the user clicks a mouse button when the mouse cursor is inside the window. This makes the window the active window and directs TYPE to get the input focus. The input focus lets TYPE read any text the user types at the keyboard.

The user can type up to the right edge of the window. TYPE ignores any extra characters entered after that point. The user can use the BACKSPACE key to delete the previous character, and use the RETURN key to delete the entire line.

When you start another instance of TYPE, Windows displays a second window:

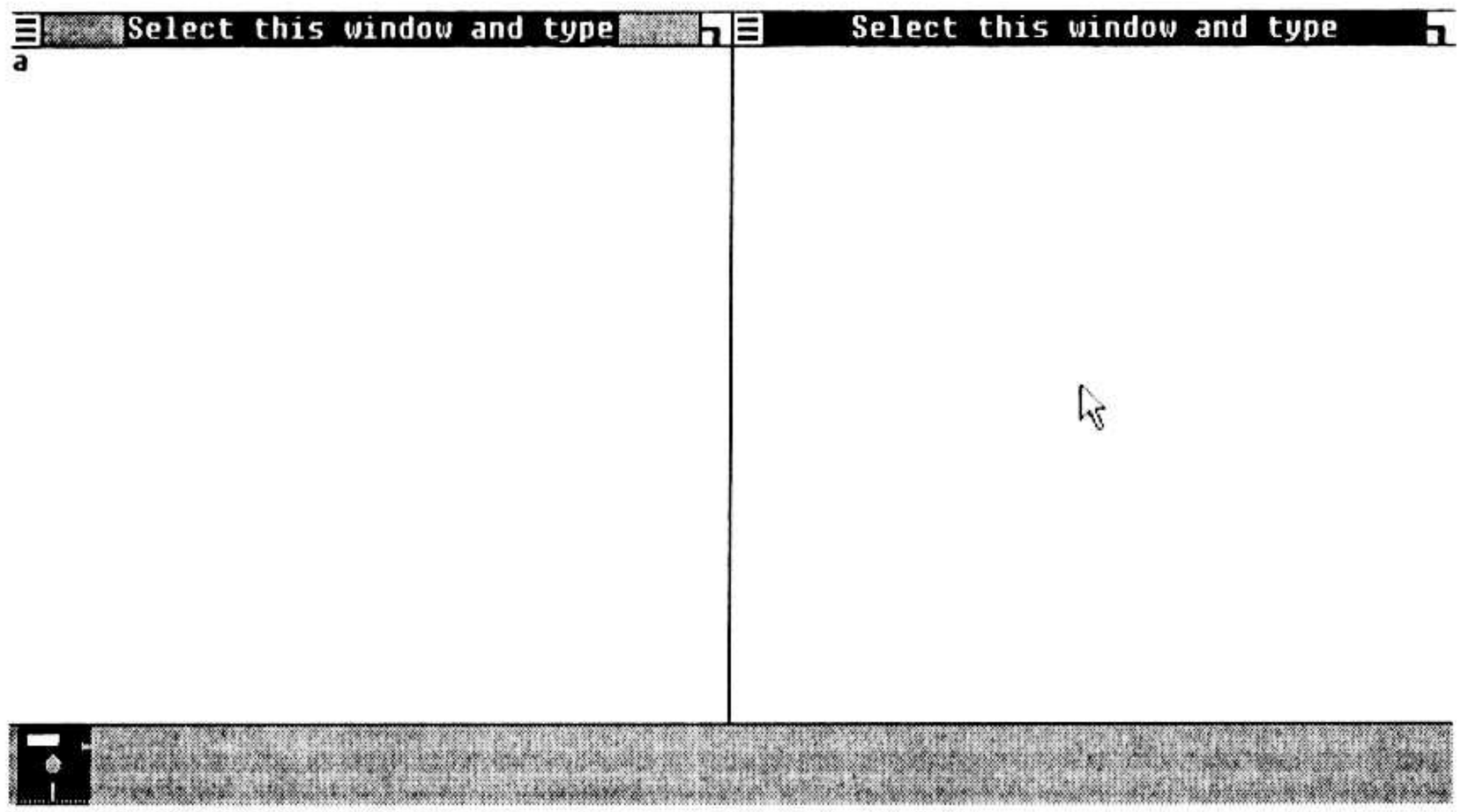


Figure 2.5 Multiple Instances of TYPE

The user can enter text in the new window by clicking the mouse in it. This makes the window the active window and requests the input focus. Since only one window can have the input focus, the original window sits idle while the user enters text in the new window. The user can give the input focus back to the original window by clicking in it.

2.10 TYPE's Source Files

TYPE's source files, TYPE.C, TYPE.DEF, and TYPE.RC, are almost identical to the source files used by HELLO. In general, nearly every application has the same basic design: a main function that defines a window class and creates a window, and a window function that calls local functions to carry out the application's tasks. Applications differ in only the variables and local functions used to carry out the tasks.

2.11 The Source File: TYPE.C

The TYPE.C file contains many of the same statements as the HELLO.C file. Although the functions **WinMain**, **TypeInit**, and **TypeWndProc** are slightly different from their counterparts in HELLO.C, their purpose and operation are identical.

The following is a complete listing of TYPE.C:

```
/*  type.c
    Type Application
    Windows Premiere Release
    Copyright (c) Microsoft 1985        */

#include "windows.h"
#include "type.h"

static char OneLine[80];
static char TinyString[2] = {0, 0};
static int  Curpos = 0;
static int  CharWidth, CharHeight;
static HANDLE hInst;
FARPROC lpprocAbout;

char *szAppName;
char *szWindowTitle;
char *szAbout;

long FAR PASCAL TypeWndProc(HWND, unsigned, WORD, LONG);
```

```
BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND            hDlg;
unsigned        message;
WORD            wParam;
LONG            lParam;
{
 if (message == WM_COMMAND) {
     EndDialog( hDlg, TRUE );
     return TRUE;
     }
 else if (message == WM_INITDIALOG)
     return TRUE;
 else return FALSE;
}

int Strlen( s )
char s[];
{
 int i;

 i = 0;
 while (s[i])
     i++;
 return (i);
}

TypeCreate( hWnd )
HWND hWnd;
{
 HDC hDC;
 TEXTMETRIC TM;
 int count;

 /* Get character width from textmetric of current font */
 hDC = GetDC(hWnd);
 GetTextMetrics (hDC, (TEXTMETRIC  FAR *) &TM);
 CharWidth = TM.tmAveCharWidth;
 CharHeight= TM.tmHeight + TM.tmExternalLeading;
 ReleaseDC(hWnd, hDC);

 /* Initialize text buffer */
 for (count = 0; count <80; count++) OneLine[count] = 0;
}
```

```
TypePaint( hWnd, lpps )
HWND hWnd;
LPPAINTSTRUCT lpps;
{
 HBRUSH  hbr, hbrOld;
 HDC hDC = lpps->hdc;

 if (lpps->fErase) {
     hbr = CreateSolidBrush(GetSysColor(COLOR_WINDOW));
     hbrOld = SelectObject(hDC, hbr);
     FillRect(hDC, (LPRECT)&lpps->rcPaint, hbr);
     SelectObject(hDC, hbrOld);
     DeleteObject(hbr);
     }
 SetBkMode(hDC, TRANSPARENT);
 SetTextColor(hDC, GetSysColor(COLOR_WINDOWTEXT));
 TextOut (hDC, 0, 0, (LPSTR) OneLine, Strlen (OneLine));
 SetBkMode(hDC, OPAQUE);
}

TypeCharInput( hWnd, ch )
HWND hWnd;
int ch;
{
 HDC hDC;
 long lOld, lNew;
 RECT rect;

 hDC = GetDC (hWnd);
 SetBkMode(hDC, TRANSPARENT);
 SetTextColor(hDC, GetSysColor(COLOR_WINDOWTEXT));
 if ((ch > 31) && (ch < 127) && Curpos < 79) {
     TinyString[0] = OneLine[Curpos++] = ch;
     OneLine[Curpos] = 0;
     TextOut (hDC, CharWidth * (Curpos - 1), 0, TinyString, 1);
     }
 else if (ch == BS && Curpos > 0) {
     lOld = GetTextExtent(hDC, (LPSTR)OneLine, Curpos);
     OneLine[--Curpos] = 0;
     lNew = (Curpos > 0 ? GetTextExtent(hDC, (LPSTR)OneLine, Curpos) : 0L);
     rect.top  = 0;              rect.bottom = HIWORD(lOld);
     rect.left = LOWORD(lNew); rect.right  = LOWORD(lOld);
     InvalidateRect(hWnd, (LPRECT)&rect, TRUE);
     }
 else if (ch == CR) {
     Curpos = OneLine[0] = 0;
     InvalidateRect (hWnd, (LPRECT) NULL, TRUE);
     }
 SetBkMode(hDC, OPAQUE);
 ReleaseDC (hWnd, hDC);
}
```

```
int PASCAL WinMain( hInstance, hPrevInstance, lpszCmdLine, cmdShow )
HANDLE hInstance, hPrevInstance;
LPSTR  lpszCmdLine;
int    cmdShow;
{
 MSG msg;
 HWND hWnd;
 HMENU hMenu;

 szAppName     = (char *)LocalAlloc( LPTR, 10 );
 szWindowTitle = (char *)LocalAlloc( LPTR, 30 );
 szAbout       = (char *)LocalAlloc( LPTR, 10 );
 LoadString( hInstance, IDSNAME, (LPSTR)szAppName, 10 );
 LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 30 );
 LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );

 if (!hPrevInstance)
     if (!TypeInit( hInstance ))
         return FALSE;

/* Create a window instance of class "Type" */
hWnd = CreateWindow((LPSTR)szAppName,
                    (LPSTR)szWindowTitle,
                    WS_TILEDWINDOW,
                    0,    /*  x - ignored for tiled windows */
                    0,    /*  y - ignored for tiled windows */
                    0,    /* cx - ignored for tiled windows */
                    0,    /* cy - ignored for tiled windows */
                    (HWND)NULL,       /* no parent */
                    (HMENU)NULL,  /* use class menu */
                    (HANDLE)hInstance, /* handle to window instance */
                    (LPSTR)NULL         /* no params to pass on */
                    );

hInst = hInstance;
lpprocAbout = MakeProcInstance( (FARPROC)About, hInstance);
hMenu = GetSystemMenu(hWnd, FALSE);
ChangeMenu(hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR);
ChangeMenu(hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING);

 ShowWindow(hWnd, cmdShow);
 UpdateWindow(hWnd);

 while (GetMessage((LPMSG)&msg, NULL, 0, 0))
     {
     TranslateMessage((LPMSG)&msg);
     DispatchMessage((LPMSG)&msg);
     }
 exit(msg.wParam);
}
```

```
int TypeInit( hInstance )
HANDLE hInstance;
{
 PWNDCLASS   pTypeClass;

 pTypeClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS) );

 pTypeClass->hCursor        = LoadCursor( NULL, IDC_ARROW );
 pTypeClass->hIcon          = LoadIcon( hInstance, (LPSTR)szAppName );
 pTypeClass->lpszMenuName   = (LPSTR)NULL;
 pTypeClass->lpszClassName  = (LPSTR)szAppName;
 pTypeClass->hbrBackground  = GetStockObject (WHITE_BRUSH);
 pTypeClass->hInstance      = hInstance;
 pTypeClass->style          = CS_VREDRAW | CS_HREDRAW;
 pTypeClass->lpfnWndProc    = TypeWndProc;

 if (!RegisterClass( (LPWNDCLASS)pTypeClass ) )
     return FALSE;

 LocalFree( (HANDLE)pTypeClass );
 return TRUE;
}

/* Procedures which make up the window class. */
long FAR PASCAL TypeWndProc(hWnd, message, wParam, lParam)
HWND        hWnd;
unsigned    message;
WORD        wParam;
LONG        lParam;
{
 PAINTSTRUCT  ps;
 HBRUSH  hbr, hbrOld;
 RECT    rect;

 switch (message)
     {
     case WM_SYSCOMMAND:
         switch (wParam) {
             case IDSABOUT:
             DialogBox(hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout);
             break;
         default:
             return(DefWindowProc(hWnd, message, wParam, lParam));
             break;
         }
     break;

 case WM_CREATE:
     TypeCreate(hWnd);
     break;

 case WM_DESTROY:
     PostQuitMessage(0);
     break;
```

```
case WM_SETFOCUS:
    /* Create flashing caret.  Use default values */
    CreateCaret( hWnd, (HBITMAP)NULL, 1, CharHeight );
    SetCaretPos( CharWidth * Curpos, 0 );
    ShowCaret( hWnd );
    break;

case WM_KILLFOCUS:
    HideCaret( hWnd );
    DestroyCaret();
    break;

case WM_PAINT:
    BeginPaint(hWnd, (LPPAINTSTRUCT)&ps);
    TypePaint(hWnd, (LPPAINTSTRUCT)&ps);
    EndPaint(hWnd, (LPPAINTSTRUCT)&ps);
    break;

    case WM_ERASEBKGND:
        GetClientRect(hWnd, (LPRECT)&rect);
        hbr = CreateSolidBrush(GetSysColor(COLOR_WINDOW));
        hbrOld = SelectObject((HDC)wParam, hbr);
        FillRect((HDC)wParam, (LPRECT)&rect, hbr);
        SelectObject((HDC)wParam, hbrOld);
        DeleteObject(hbr);
        break;

    case WM_CHAR:
        TypeCharInput(hWnd, wParam);
        SetCaretPos( CharWidth * Curpos, 0 );
        break;

    default:
        return(DefWindowProc(hWnd, message, wParam, lParam));
        break;
    }
 return(0L);
}
```

2.12 TYPE's Local Functions

TYPE has several local functions that it uses to format and display the keyboard input. These functions are local functions because they are called by the window function **TypeWndProc** and never by Windows itself.

TYPE also uses several static variables. The **OneLine** variable is a character array that contains the current line. The **TinyString** variable is a two-element character array that contains the current character. The **Curpos** variable is an integer variable that contains the current cursor position in terms of characters. The **CharWidth** variable contains the width, in pixels, of the average character.

2.12.1 Carrying Out Early Tasks

The **TypeCreate** function is called by the window function when it receives a WM_CREATE message. Windows sends a WM_CREATE message to a window function whenever a window that belongs to the function's window class is created; for example, when **WinMain** calls **CreateWindow**.

The **TypeCreate** function uses the **GetTextMetrics** function to retrieve information about the currently selected font. TYPE uses the average width of a character in this font to determine where to place each character as it is typed. Since the currently selected font is a property of the window's display context, **TypeCreate** retrieves a copy of the context by using the **GetDC** function. After using the context, **TypeCreate** releases it with the **ReleaseDC** function. A window's display context should always be released as soon as possible after using it. Windows prevents other applications from accessing the display associated with the context while the context is in use.

2.12.2 Getting the Input Focus

Windows makes keyboard input available to the window that currently has the input focus. A window receives the input focus when the user makes the window active and Windows passes a WM_SETFOCUS message to the application's window function to let it prepare for reading characters from the keyboard. A window loses the focus when the user makes another window active. Windows passes a WM_KILLFOCUS message to signal the change.

TYPE uses the WM_SETFOCUS message to display a caret in the window and the WM_KILLFOCUS message to remove the caret.

```
case WM_SETFOCUS:
    /* Create flashing caret.  Use default values */
    CreateCaret( hWnd, (HBITMAP)NULL, 1, CharHeight );
    SetCaretPos( CharWidth * Curpos, 0 );
    ShowCaret( hWnd );
    break;

case WM_KILLFOCUS:
    HideCaret( hWnd );
    DestroyCaret();
    break;
```

After the window has received the input focus, all characters typed by the user are available to the **WinMain** function in the form of virtual keyboard messages: WM_KEYUP and WM_KEYDOWN. **WinMain** translates the

messages into WM_CHAR messages and passes them to the window function. The window function processes the WM_CHAR messages, using the ANSI value to display the corresponding character. It ignores the WM_KEYUP and WM_KEYDOWN messages since these are virtual key messages and do not contain ANSI information.

2.12.3 Painting With a WM_PAINT Message

A WM_PAINT message is a signal from Windows or from the application itself that significant changes have occurred in the window and that the contents need to be updated. In TYPE, the **TypePaint** function is used to update the window. It is called by **TypeWndProc** whenever a WM_PAINT message is received. It updates the window by writing the complete contents of the current line.

```
TypePaint( hWnd, lpps )
HWND hWnd;
LPPAINTSTRUCT lpps;
{
 HBRUSH  hbr, hbrOld;
 HDC hDC = lpps->hdc;

 if (lpps->fErase) {
      hbr = CreateSolidBrush(GetSysColor(COLOR_WINDOW));
      hbrOld = SelectObject(hDC, hbr);
      FillRect(hDC, (LPRECT)&lpps->rcPaint, hbr);
      SelectObject(hDC, hbrOld);
      DeleteObject(hbr);
      }
 SetBkMode(hDC, TRANSPARENT);
 SetTextColor(hDC, GetSysColor(COLOR_WINDOWTEXT));
 TextOut (hDC, 0, 0, (LPSTR) OneLine, Strlen (OneLine));
 SetBkMode(hDC, OPAQUE);
}
```

TypeWndProc passes a long pointer to a structure having PAINTSTRUCT type. This structure, retrieved by the **BeginPaint** function in the window function, contains information about the paint request, such as the window's display context and a flag indicating whether or not all or part of the window needs to be erased. If the **fErase** flag is TRUE, **TypePaint** uses a **FillRect** function and **rcPaint** from the paint structure to erase the window. The **CreateSolidBrush** function creates the brush to be used in this operation with the **GetSysColor** function defining the brush color.

To draw actual characters in the window, the **SetBkMode** function sets the background mode of the window to TRANSPARENT and the **SetTextColor** function sets the foreground color of the character. In TRANSPARENT mode, the subsequent **TextOut** function draws a character without overwriting the background. This means the original contents of the screen behind the character remain visible. The **GetSysColor** function is used to make sure the foreground color matches the color of text in menus and the window's caption bar.

The **Strlen** function is a simple utility that returns the length of a given null-terminated string. It is used in the **TextOut** call.

2.12.4 Painting Without a WM_PAINT Message

An application can write or draw in the window at any time. It does not have to be in response to a WM_PAINT message. The **TypeCharInput** function, which processes each character typed by the user, writes into the window whenever it finds a printable character or a backspace. The function is called when **TypeWndProc** receives a WM_CHAR message, not a WM_PAINT. WM_CHAR messages contain single characters from the keyboard.

To determine what action to take, **TypeCharInput** checks each character. The first check looks for a printable character:

```
if ((ch > 31) && (ch < 127) && Curpos < 79)
    {
    TinyString[0] = OneLine[Curpos++] = ch;
    OneLine[Curpos] = 0;
    TextOut (hDC, Width * (Curpos - 1), 0, TinyString, 1);
    }
```

TypeCharInput appends a printable character to the end of the **OneLine** variable while updating the cursor position. It then writes the character into the window at the current pixel location given by `Width * (Curpos - 1)`.

The second check looks for backspace characters:

```
else if (ch == BS && Curpos > 0)
    {
    lOld = GetTextExtent(hDC, (LPSTR)OneLine, Curpos);
    OneLine[--Curpos] = 0;
    lNew = (Curpos > 0 ? GetTextExtent(hDC, (LPSTR)OneLine, Curpos) : 0L);
    rect.top  = 0;              rect.bottom = HIWORD(lOld);
    rect.left = LOWORD(lNew); rect.right  = LOWORD(lOld);
    InvalidateRect(hWnd, (LPRECT)&rect, TRUE);
    }
```

Whenever it encounters a backspace character, **TypeCharInput** moves the cursor position back one and deletes the character at that position. To delete the character, **TypeCharInput** computes the coordinates of the rectangle containing the character, then uses **InvalidateRect** to signal a need to erase and repaint the contents of the rectangle.

In general, if an application chooses to write or draw without a WM_PAINT, it must get a display context for the window by calling the **GetDC** function just before the actual operation. It must release the display context immediately after painting by using the **ReleaseDC** function. This method of getting and releasing a display context is required by all applications that use the default display context for a window. Applications that use class or private display contexts can get and keep the display context as long as the window does not move or change size. The type of display context for a window is defined by the window class style argument.

The third check looks for carriage returns:

```
else if (ch == CR)
    {
    Curpos = OneLine[0] = 0;
    InvalidateRect (hWnd, (LPRECT) NULL, TRUE);
    }
```

Whenever it encounters a carriage return, **TypeCharInput** deletes all characters in the line by setting the first element of **OneLine** to 0. **TypeCharInput** does not actually write to the window when it finds a carriage return. Instead, it uses the **InvalidateRect** to signal Windows that a WM_PAINT message is needed. The "(LPRECT)NULL" argument specifies that the entire window needs updating. The "TRUE" argument is a hint for Windows to paint the client area with the class background brush before issuing the WM_PAINT message.

2.12.5 Using a System Font

In Windows, writing text in a window is a graphics operation. You draw characters in the window by using the **TextOut** and the currently selected font of the display context. A font is a collection of characters that have a similar design and size. If you do not explicitly define a selected font, Windows provides a default system font. This is what TYPE uses.

The default system font is a fixed-pitch font whose characters belong to the ANSI character set. Fixed-pitch fonts have equal spacing between characters, so all characters are equal to the font's average width. You can retrieve information about the font, such as how many pixels high and wide the characters are, by using the **GetTextMetrics** function. TYPE uses

this to find the average character width. It uses the average character width to determine where to place the next character typed.

2.12.6 Creating Multiple Instances

When a user makes a new instance of an application, Windows creates a new data segment for the instance and calls the main function again, making a separate invocation of the application that runs at the same time as the original invocation.

Windows passes **WinMain** a new module instance handle and the module instance handle of the previous instance, **hPrevInstance**. **WinMain** can use this argument to access the previous module, if necessary. It can also use it to determine whether or not a window class should be registered. Since registered window classes are available to all instances, an instance does not need to register classes that have been registered by previous instances.

The new instance can create a window and enter a program loop just like the original instance. It can also allocate storage space and assign values to that space. Since the new instance receives a new data segment, any data created by the original instance is not available.

Chapter 3
Using GDI in Applications

3.1 Introduction 45
3.2 The SHAPES Application 45
3.3 How SHAPES Works 45
3.4 The SHAPES.C File 47
3.5 Processing Menu Input 53
3.5.1 Painting Shapes 54
3.5.2 Setting Display Context Attributes 55
3.5.3 Drawing Rectangles 57
3.5.4 Drawing an Ellipse 58
3.5.5 Drawing a Triangle 58
3.5.6 Drawing a Star 58
3.6 Menus in a Resource File 59
3.7 The Default Menu for the Window Class 60
3.8 Defining an Accelerator Table 60
3.9 Loading and Using an Accelerator Table 61
3.10 The TRACK Application 62
3.11 How TRACK Works 62
3.12 The TRACK.C File 66
3.13 Setting the Coordinate System 73
3.14 Reading and Setting the Menu 74
3.15 Making the Drawing Box 75
3.16 Borrowing Data from the Previous Instance 78

3.1 Introduction

This chapter explains how to use the Graphics Display Interface (GDI) functions to draw shapes in a window and to modify the attributes of the display context associated with the window. It presents two sample applications named SHAPES and TRACK. The applications presented here are similar to applications provided on floppy disk.

3.2 The SHAPES Application

The SHAPES application is a graphics application that illustrates how to use the GDI output functions to draw lines and regular geometric shapes in a window.

3.3 How SHAPES Works

SHAPES creates and displays a window that can contain a star, rectangle, triangle, or ellipse. When SHAPES is first invoked, it displays a star:

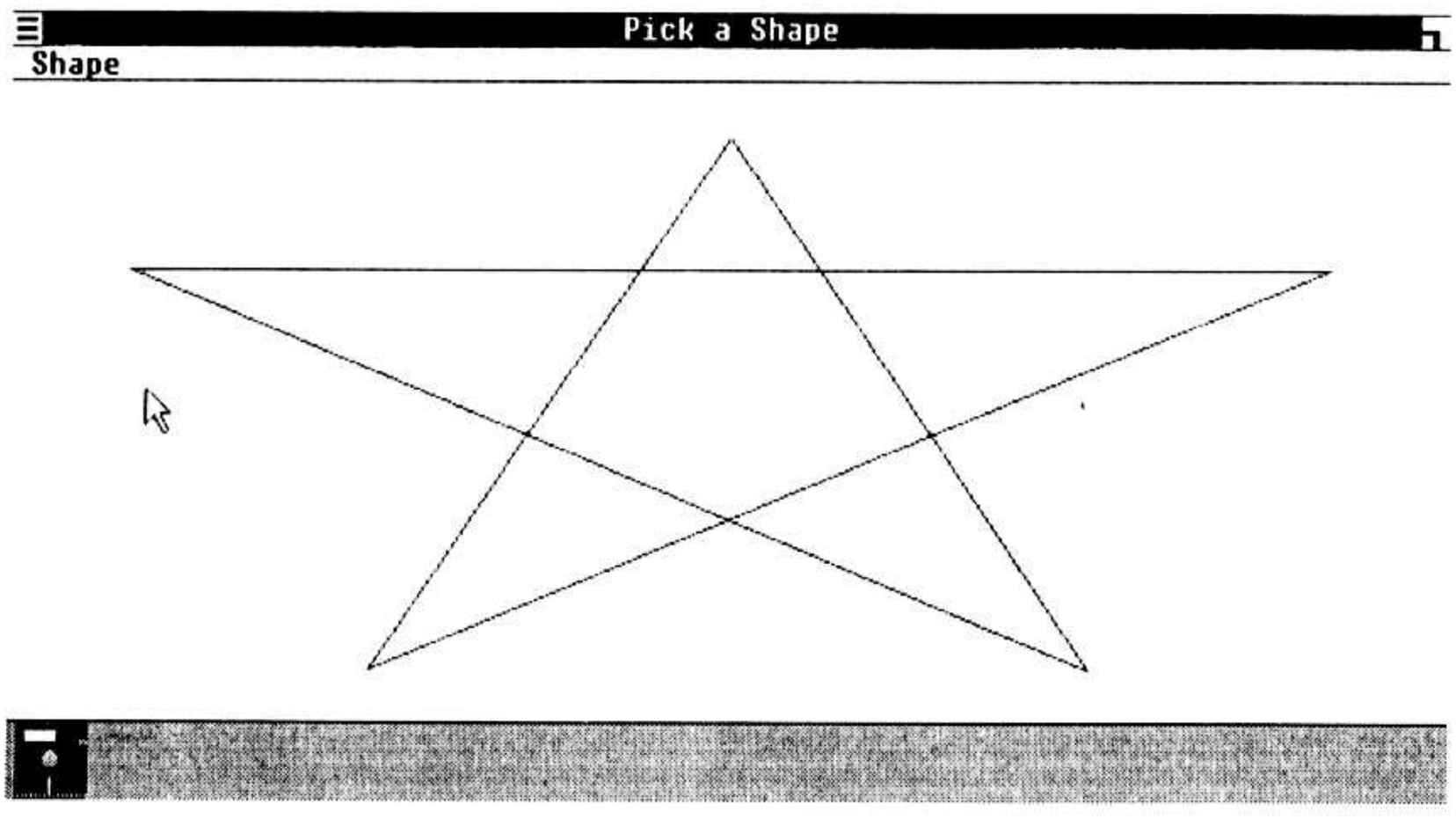


Figure 3.1 SHAPES's Star

The user can choose the shape to be displayed by selecting a command from the Shape menu. The menu contains five commands: Clear, Ellipse, Rectangle, Star, and Triangle. The menu appears immediately below the caption bar.

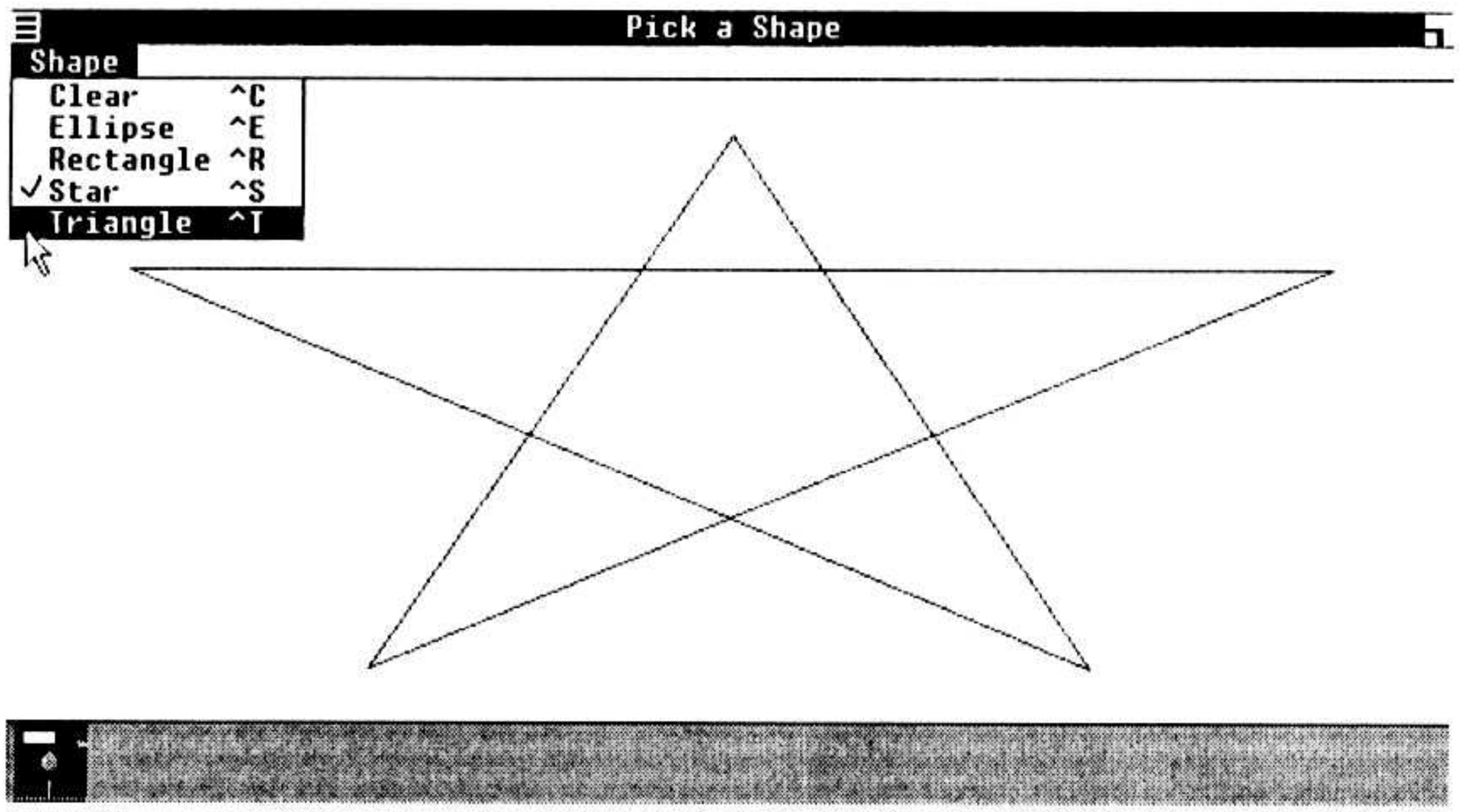


Figure 3.2 SHAPE's Menu

Selecting the Triangle command destroys the star and replaces it with the large triangle.

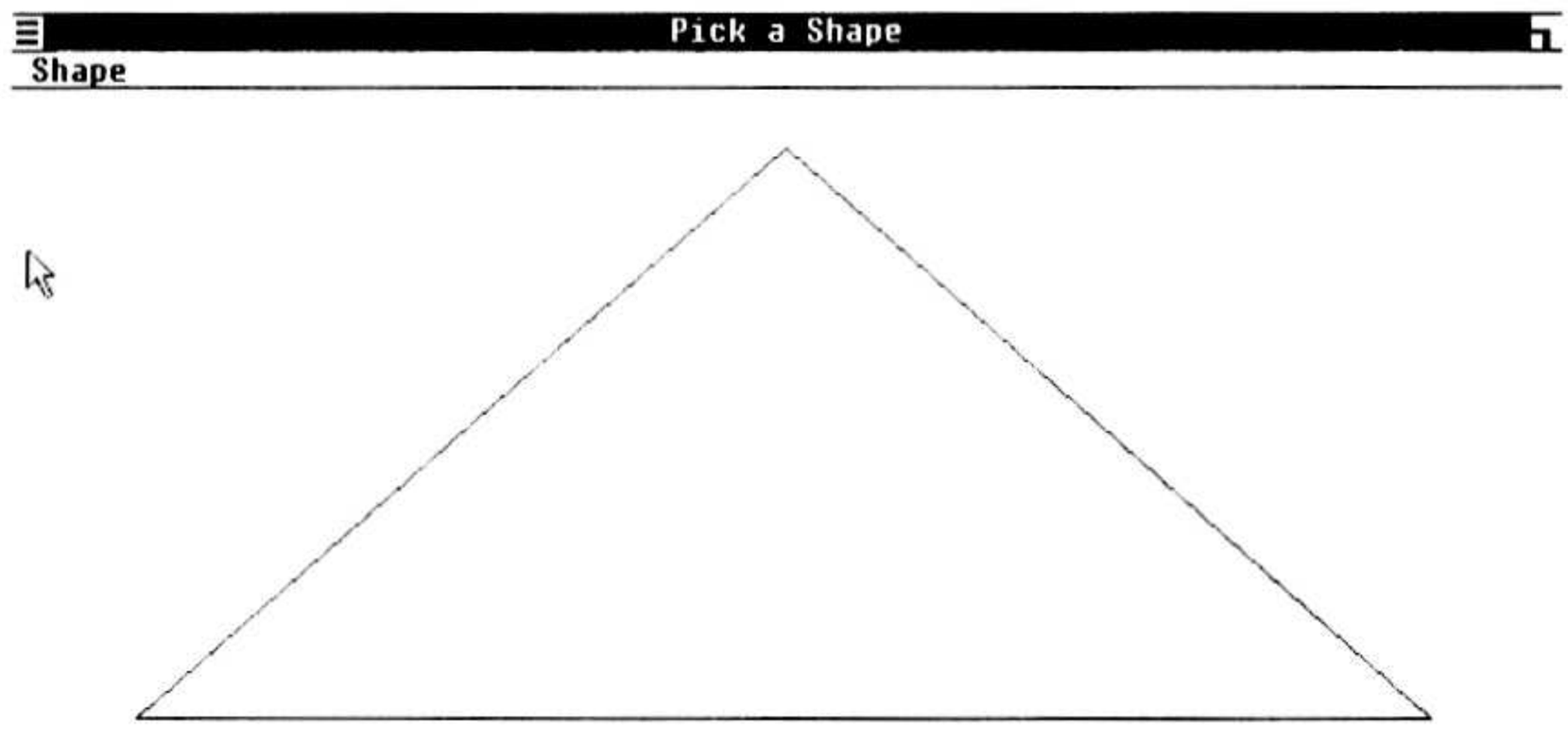


Figure 3.3 A New Shape

3.4 The SHAPES.C File

The SHAPES.C file defines a main function, **WinMain**, and a window function, **ShapesWndProc**. The main function calls **ShapesInit** to register a window class, and then calls the **CreateWindow** function to create the window.

The window function examines all input messages and responds to WM_DESTROY, WM_PAINT, and WM_COMMAND messages. It calls a variety of local functions to carry out its tasks.

The following is a complete listing of SHAPES.C:

```
/*  shapes.c
    Microsoft Windows
    Shapes Application
    Premiere Edition
    Copyright (c) Microsoft 1985        */

#include "windows.h"
#include "shapes.h"

static HANDLE hInst;
static HANDLE hAccelTable;
static int cCurrentShape = IDDSTAR;
```

```
FARPROC lpprocAbout;

char *szAppName;
char *szAbout;
char *szWindowTitle;

static POINT StarPoints[]=
    {
    100, 75,
    0, 75,
    80, 0,
    50, 100,
    20, 0,
    100, 75
    };

long FAR PASCAL ShapesWndProc(HWND, unsigned, WORD, LONG);

BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
    if (message == WM_COMMAND) {
        EndDialog( hDlg, TRUE );
        return TRUE;
        }
    else if (message == WM_INITDIALOG)
        return TRUE;
    else return FALSE;
}


ShapesSetupDC( hWnd, hDC )
HWND hWnd;
HDC hDC;
{
    RECT ClientRect;
    SetBkMode( hDC, OPAQUE );
    SetROP2( hDC, R2_COPYPEN );
    GetClientRect( hWnd, (LPRECT)&ClientRect );
    SetMapMode( hDC, MM_ANISOTROPIC );
    SetWindowOrg( hDC, -10, 110 );
    SetWindowExt( hDC, 120, -120 );
    SetViewportOrg( hDC, 0, 0 );
    SetViewportExt( hDC,
                    ClientRect.right-ClientRect.left,
                    ClientRect.bottom-ClientRect.top );
}
```

```
ShapesPaint( hWnd, lpps )
HWND hWnd;
LPPAINTSTRUCT lpps;
{
    HDC hShapesDC;
    HBRUSH hbr, hbrOld;
    HPEN hpen, hpenOld;

    ShapesSetupDC( hWnd, hShapesDC = lpps->hdc );
    if (lpps->fErase) {
        hbr = CreateSolidBrush( GetSysColor(COLOR_WINDOW) );
        hbrOld = SelectObject( hShapesDC, hbr );
        FillRect( hShapesDC, (LPRECT)&lpps->rcPaint, hbr );
        SelectObject( hShapesDC, hbrOld );
        DeleteObject( hbr );
        }
    hpen = CreatePen( PS_SOLID, 0, GetSysColor(COLOR_WINDOWTEXT) );
    hpenOld = SelectObject( hShapesDC, hpen );
    SelectObject( hShapesDC, GetStockObject(NULL_BRUSH) );
    switch (cCurrentShape)
    {
    case IDDCLEAR:
        break;
    case IDDRECT:
        DrawRect( hShapesDC );
        break;
    case IDDELLIPSE:
        DrawEllipse( hShapesDC );
        break;
    case IDDTRIANGLE:
        DrawTriangle( hShapesDC );
        break;
    case IDDSTAR:
        DrawStar( hShapesDC );
        break;
    }
    SelectObject( hShapesDC, hpenOld );
    DeleteObject( hpen );
}

DrawRect( hDC )
HDC hDC;
{
    Rectangle( hDC, 0, 0, 100, 100 );
}


DrawEllipse( hDC )
HDC hDC;
{
    Ellipse( hDC, 0, 100, 100, 0 );
}
```

```
DrawTriangle( hDC )
HDC hDC;
{
    MoveTo( hDC, 0, 0 );
    LineTo( hDC, 100, 0 );
    LineTo( hDC, 50, 100 );
    LineTo( hDC, 0, 0 );
}


DrawStar( hDC )
HDC hDC;
{
    Polyline( hDC, StarPoints, 6 );
}
```

```
ShapesCommand( hWnd, id )
HWND hWnd;
int id;
{
    HMENU hMenu = GetMenu( hWnd );

    CheckMenuItem( hMenu, cCurrentShape, MF_UNCHECKED );
    cCurrentShape = id;
    CheckMenuItem( hMenu, cCurrentShape, MF_CHECKED );
    InvalidateRect( hWnd, (LPRECT)NULL, TRUE );
}


int PASCAL WinMain( hInstance, hPrevInstance, lpszCmdLine, cmdShow )
HANDLE hInstance, hPrevInstance;
LPSTR  lpszCmdLine;
int    cmdShow;
{
    MSG   msg;
    HWND  hWnd;
    HMENU hMenu;

    /* Loading from string table */
    szAppName = (char *)LocalAlloc( LPTR, 10 );
    szAbout   = (char *)LocalAlloc( LPTR, 10 );
    szWindowTitle = (char *)LocalAlloc( LPTR, 15 );
    LoadString( hInstance, IDSNAME, (LPSTR)szAppName,.10 );
    LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );
    LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 15 );

    if (!hPrevInstance) {
        if (!ShapesInit( hInstance ))
            return FALSE;
        }
    else
        /* Copy global instance variables from previous instance */
        GetInstanceData( hPrevInstance, (PSTR)&hAccelTable, sizeof(hAccelTable) );
```

```
    hWnd = CreateWindow( (LPSTR)szAppName,
                         (LPSTR)szWindowTitle,
                         WS_TILEDWINDOW,
                         0,     /*  x - ignored for tiled windows */
                         0,     /*  y - ignored for tiled windows */
                         0,     /* cx - ignored for tiled windows */
                         0,     /* cy - ignored for tiled windows */
                         (HWND)NULL,       /* no parent */
                         (HMENU)NULL,  /* use class menu */
                         (HANDLE)hInstance, /* handle to window instance */
                         (LPSTR)NULL        /* no params to pass on */
                        );

    hInst = hInstance;
    lpprocAbout = MakeProcInstance( (FARPROC)About, hInstance );
    hMenu = GetSystemMenu( hWnd, FALSE );
    ChangeMenu( hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR );
    ChangeMenu( hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING );

    ShowWindow( hWnd, cmdShow );
    UpdateWindow( hWnd );

    while (GetMessage( (LPMSG)&msg, NULL, 0, 0) ) {
        if (TranslateAccelerator( hWnd, hAccelTable, (LPMSG)&msg ) == 0) {
            TranslateMessage( (LPMSG)&msg );
            DispatchMessage( (LPMSG)&msg );
            }
        }
    exit( msg.wParam );
}


int ShapesInit( hInstance )
HANDLE hInstance;
{
    PWNDCLASS   pShapesClass;

    pShapesClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS) );

    pShapesClass->hCursor        = LoadCursor( NULL, IDC_ARROW );
    pShapesClass->hIcon          = LoadIcon( hInstance, (LPSTR)szAppName );
    pShapesClass->lpszMenuName   = (LPSTR)szAppName;
    pShapesClass->hInstance      = hInstance;
    pShapesClass->lpszClassName  = (LPSTR)szAppName;
    pShapesClass->hbrBackground  = NULL;
    pShapesClass->style          = CS_HREDRAW | CS_VREDRAW;
    pShapesClass->lpfnWndProc    = ShapesWndProc;

    if (!RegisterClass( (LPWNDCLASS)pShapesClass) )
        return FALSE;   /* Initialization failed */

    LocalFree( (HANDLE)pShapesClass );
    hAccelTable = LoadAccelerators( hInstance, (LPSTR)szAppName );

    return TRUE;    /* Initialization succeeded */
}
```

```
LONG FAR PASCAL ShapesWndProc( hWnd, message, wParam, lParam )
HWND hWnd;
unsigned message;
WORD wParam;
LONG lParam;
{
    PAINTSTRUCT ps;
    RECT rect;
    HBRUSH hbr, hbrOld;

    switch (message)
    {
    case WM_SYSCOMMAND:
        switch (wParam)
        {
        case IDSABOUT:
            DialogBox( hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout );
            break;
        default:
            return( DefWindowProc(hWnd, message, wParam, lParam) );
            break;
        }
        break;

    case WM_CREATE:
        CheckMenuItem( GetMenu(hWnd), cCurrentShape, MF_CHECKED );
        break;

    case WM_PAINT:
        BeginPaint( hWnd, (LPPAINTSTRUCT)&ps );
        ShapesPaint( hWnd, (LPPAINTSTRUCT)&ps );
        EndPaint( hWnd, (LPPAINTSTRUCT)&ps );
        break;

    case WM_ERASEBKGND:
        GetClientRect( hWnd, (LPRECT)&rect );
        hbr = CreateSolidBrush( GetSysColor(COLOR_WINDOW) );
        hbrOld = SelectObject( (HDC)wParam, hbr );
        FillRect((HDC)wParam, (LPRECT)&rect, hbr);
        SelectObject( (HDC)wParam, hbrOld );
        DeleteObject( hbr );
        break;

    case WM_COMMAND:
        ShapesCommand( hWnd, wParam );
        break;

    case WM_DESTROY:
        PostQuitMessage( 0 );
        break;

    default:
        return( DefWindowProc(hWnd, message, wParam, lParam) );
        break;
    }
    return(0L);
}
```

3.5 Processing Menu Input

SHAPES uses a menu for user input. The menu generates a WM_COMMAND message when the user clicks a mouse button on a menu command. Each message has an accompanying **wParam** argument that defines which menu item is selected. The **wParam** value is equal to the value assigned to the menu item in the resource script file.

When **ShapesWndProc** receives a WM_COMMAND message it calls **ShapesCommand**, a local function:

```
case WM_COMMAND:
    ShapesCommand( hWnd, wParam );
    break;
```

ShapesCommand sets the static variable **cCurrentShape** to the **wParam** value. This value is passed to the **id** parameter of **ShapesCommand:**

```
ShapesCommand( hWnd, id )
HWND hWnd;
int id;
{
    HMENU hMenu = GetMenu( hWnd );

    CheckMenuItem( hMenu, cCurrentShape, MF_UNCHECKED );
    cCurrentShape = id;
    CheckMenuItem( hMenu, cCurrentShape, MF_CHECKED );
    InvalidateRect( hWnd, (LPRECT)NULL, TRUE );
}
```

The function uses the **CheckMenuItem** function to place a checkmark next to the name, in the Shapes menu, of the currently selected shape. The **GetMenu** function retrieves the handle to the menu. The first **CheckMenuItem** function removes the check from the previously selected item, and the second places a check next to the new item. The function then calls **InvalidateRect** to direct Windows to create a WM_PAINT message for the window. Since drawing a new shape means changing the entire window, SHAPES uses the WM_PAINT message to make its changes instead of attempting to make them elsewhere. The "NULL" argument in **InvalidateRect** means the entire window should be repainted. The "TRUE" argument directs Windows to paint the window over with the class background color before sending the WM_PAINT message to the window function.

3.5.1 Painting Shapes

The **ShapesPaint** function is called whenever **ShapesWndProc** receives a WM_PAINT message. The function uses the current value of the static variable **cCurrentShape** to decide which shape to draw. Once selected, **ShapesPaint** calls the appropriate local function to carry out the drawing:

```
ShapesPaint( hWnd, lpps )
HWND hWnd;
LPPAINTSTRUCT lpps;
{
    HDC hShapesDC;
    HBRUSH hbr, hbrOld;
    HPEN hpen, hpenOld;

    ShapesSetupDC( hWnd, hShapesDC = lpps->hdc );
    if (lpps->fErase) {
        hbr = CreateSolidBrush( GetSysColor(COLOR_WINDOW) );
        hbrOld = SelectObject( hShapesDC, hbr );
        FillRect( hShapesDC, (LPRECT)&lpps->rcPaint, hbr );
        SelectObject( hShapesDC, hbrOld );
        DeleteObject( hbr );
        }
    hpen = CreatePen( PS_SOLID, 0, GetSysColor(COLOR_WINDOWTEXT) );
    hpenOld = SelectObject( hShapesDC, hpen );
    SelectObject( hShapesDC, GetStockObject(NULL_BRUSH) );
    switch (cCurrentShape)
    {
    case IDDCLEAR:
        break;
    case IDDRECT:
        DrawRect( hShapesDC );
        break;
    case IDDELLIPSE:
        DrawEllipse( hShapesDC );
        break;
    case IDDTRIANGLE:
        DrawTriangle( hShapesDC );
        break;
    case IDDSTAR:
        DrawStar( hShapesDC );
        break;
    }
    SelectObject( hShapesDC, hpenOld );
    DeleteObject( hpen );
}
```

Before deciding the shape, the **ShapesSetupDC** function is used to modify the attributes of a display context for the window. After drawing, **ShapesPaint** validates the window (indicating that it has been painted) using the **ValidateRect** function.

The modified display context is destroyed by the **EndPaint** function. This means the next time the application retrieves the display context, it will have its original attributes. For this reason, the display context must be modified each time it is retrieved.

Note that Windows destroys the user-defined attributes only when the display context is released. Since class and private display contexts can be held until the corresponding window moves or changes size, the application can preserve the changes as long as the display context still accurately defines the window.

3.5.2 Setting Display Context Attributes

Every display context has a set of default attributes that define how the images are drawn in the window. Display context attributes define the colors used for text and background, the drawing mode used to combine freshly drawn lines with colors already on the screen, and the coordinates system used to map images into the window.

When a display context is first retrieved, it has MM_ TEXT mapping mode. This means it represents a text window in which the origin is at the upper left corner, the y axis points down (coordinates increase as you go lower), and all coordinate values refer to screen pixels. Since this is not always the best mode in which to draw graphics, you need to change the mapping mode by changing some of the attributes of the display context.

The **ShapesSetupDC** function changes the mapping mode and alters the coordinate system of the context so that the origin is at the lower left corner and the y axis points up.

```
ShapesSetupDC( hWnd, hDC )
HWND hWnd;
HDC hDC;
{
    RECT ClientRect;

    SetBkMode( hDC, OPAQUE );
    SetROP2( hDC, R2_COPYPEN );
    GetClientRect( hWnd, (LPRECT)&ClientRect );
    SetMapMode( hDC, MM_ANISOTROPIC );
    SetWindowOrg( hDC, -10, 110 );
    SetWindowExt( hDC, 120, -120 );
    SetViewportOrg( hDC, 0, 0 );
    SetViewportExt( hDC,
                    ClientRect.right-ClientRect.left,
                    ClientRect.bottom-ClientRect.top );
}
```

The **SetMapMode** function sets the new mapping mode to MM_ANISOTROPIC. In this mode, the application must define its own units for drawing images in the client area of the window. In other modes, such as MM_LOMETRIC, Windows defines the units used.

The **SetWindowOrg** and **SetWindowExt** do not refer to the application's window. Instead, they define the origin and extents of a hypothetical window in GDI's logical coordinate system. The logical coordinate system is an internal system in which GDI draws all images before mapping them to the client area. Since the logical coordinate system is usually much larger than the client area, the window is used to determine what part of the logical coordinate system will be visible through the client area when mapped to the screen. The **SetViewportOrg** and **SetViewportExt** functions refer to the origin and extents of the display surface viewport. The viewport plays the same role for the client area coordinate system (called the physical coordinate system) as the window does for the logical coordinate system. When GDI actually maps an image, it aligns the logical and physical coordinate systems by aligning the window and viewport origins. It then uses the window and viewport extents to determine how many units in the logical coordinate system should be mapped to pixels in the client area. For example, if the window extent in the x direction is 120 and the viewport extent in the same direction is 240, a horizontal line drawn exactly 1 unit long in the logical coordinate system will be exactly 2 pixels long in the client area.

ShapeGetDC adjusts the window and viewport origins to (–10, 110) in the logical coordinate system and (0, 0) in the client area. This means that drawing a point at (–10, 110) in the logical coordinate system will appear as a point at (0, 0) in the client area. The point (0, 0) in the physical coordinate system is always the upper left hand corner of the client area.

ShapesGetDC adjusts the window extents to (120, -120) and the viewport extents to the current width and height of the client area. This means that a horizontal line 120 units long in the logical coordinate system will have the full width of the client area when displayed. Since the size of the client area can change as you move and resize other windows on the display screen, this mapping ensures that any image you draw near the window origin will always be visible if it is no longer or higher than 120 logical units.

You may have noticed that the window extent in the y direction is a negative value. This means that GDI will map the negative y axis of the logical coordinate system to the positive y axis of the client area. This is required if you wish to keep the image pointing in the direction you expect to see it. By default, the y axis of client area always points down, while the y axis of the logical coordinate system always points up. If you draw a image in MM_ANISOTROPIC mode (and all other modes except MM_TEXT)

without reversing the orientation of the y axes, the image will be drawn upside down. By making sure that the signs of the y extents for the window and viewport are opposite, you can always draw images with the correct orientation.

One way to think of the window is as a rectangle that encloses everything in the logical coordinate system that will be mapped to the viewport. Similarly, the viewport is a rectangle that encloses the part of the client area that will contain the images from the window. In SHAPES, the window is the rectangle in the logical coordinate system whose upper left corner is (–10, 110) and lower right corner is (110, –10). You can compute the lower right corner by adding the window extents to the window origin. The viewport in SHAPES is the entire client area. When GDI maps images, it aligns the upper left corners of the window and viewport; it also aligns their lower right corners. The result is that the image in the window appears in the client area. You can change what you see by changing the viewport. For example, by setting the x extent of the viewport to twice the width of the client area, only the left half of the image in the window will be visible in the client area. Conversely, by setting the x extent to half the width of the client area, the full image will be confined to the left half of the client area and images that were previously not visible because they were outside and to the right of the window will now appear in the right half.

ShapesGetDC also sets the background mode to OPAQUE, and the drawing mode to R2_COPYPEN. This means the pen overwrites everything.

3.5.3 Drawing Rectangles

SHAPES draws rectangles by using the **Rectangle** function. This function takes the coordinates of opposite corners in a rectangle and draws a rectangle in the client area using the currently selected pen for the border and the currently selected brush to fill the interior.

```
DrawRect (hDC)
HDC    hDC;
{
Rectangle( hDC, 0, 0, 100, 100);
}
```

Coordinates in the **Rectangle** function are logical coordinates. This means the rectangle is drawn in the logical coordinate system before being mapped to the client area. In this case, the lower left corner of the rectangle is (0, 0) and the upper right corner is (100, 100). This means the rectangle is completely within the window defined by **ShapesGetDC** so you will see the complete rectangle when it is mapped to the client area.

3.5.4 Drawing an Ellipse

SHAPES draws ellipses by using the **Ellipse** function. This function takes the coordinates of the upper left and lower right points of a bounding rectangle and draws the ellipse that fills the rectangle. It uses the currently selected pen for the border and the currently selected brush to fill the interior.

```
DrawEllipse (hDC)
HDC     hDC;
{
Ellipse (hDC, 0, 100, 100, 0);
}
```

Coordinates in the **Ellipse** function, as in **Rectangle**, are logical coordinates, so the ellipse is drawn in the logical coordinate system before being mapped to the client area. Since the bounding rectangle has the same size and location as the rectangle drawn by **DrawRect**, the complete ellipse will be visible in the client area.

3.5.5 Drawing a Triangle

SHAPES draws triangles by using a **MoveTo** function followed by a series of **LineTo** functions. These functions move the current position to the given coordinates while drawing a line using the currently selected pen.

```
DrawTriangle (hDC)
HDC     hDC;
{
MoveTo (hDC, 0, 0);
LineTo (hDC, 100, 0);
LineTo (hDC, 50, 100);
LineTo (hDC, 0, 0);
}
```

The coordinates are logical coordinates.

3.5.6 Drawing a Star

SHAPES draws five-pointed stars by using the **Polyline** function. This function draws line segments from one point to another using the currently selected pen. The points are defined by the static array **StarPoints**.

```
static POINT StarPoints[]=
     {
     100, 75,
     0, 75,
     80, 0,
     50, 100,
     20, 0,
     100, 75
     };

DrawStar (hDC)
HDC    hDC;
{
Polyline (hDC, StarPoints, 6);
}
```

The coordinates are logical coordinates.

3.6 Menus in a Resource File

The SHAPES.RC file contains the resource definition for the menu used by SHAPES. It also contains the definition for SHAPE's icon.

A resource definition for a menu consists of the names and values of the menu items, the type of item, and the order in which the items appear when displayed. When the resources are added to the executable file, the menu definition is compiled and added too.

The following is the menu definition SHAPES.RC:

```
shapes menu
begin
     POPUP "Shape"
          begin
               MENUITEM "Clear\t^C"      , IDDCLEAR
               MENUITEM "Ellipse\t^E"    , IDDELLIPSE
               MENUITEM "Rectangle\t^R", IDDRECT    .
               MENUITEM "Star\t^S"       , IDDSTAR
               MENUITEM "Triangle\t^T"  , IDDTRIANGLE
          end
end
```

The menu definition begins with the **MENU** statement. The menu name is "shapes." The **POPUP** statement defines "Shape" to be a menu name. The Shape menu contains five menu items: Clear, Ellipse, Rectangle, Star, and Triangle. The values associated with these menu items are given by the constants IDDCLEAR, IDDELLIPSE, IDDRECT, IDDSTAR, and IDDTRIANGLE. These names are assumed to have been defined in the include file SHAPES.H.

With each menu item name is an accelerator key name. An accelerator key is a way to invoke a menu item quickly and from the keyboard. In this case, the accelerators are all control characters as indicated by the caret (^) preceding the key letter. A tab character (\t) is specified to ensure that the item name and key name are clearly separated when the menu is displayed.

3.7 The Default Menu for the Window Class

The **ShapesInit** function assigns the menu resource name "Shapes" to the **lpszMenuName** field of SHAPE's window class structure:

```
pShapesClass->lpszMenuName    = (LPSTR)szAppName;
```

Windows makes this menu the default menu for the class. If a menu handle is not specified in the **CreateWindow** function, Windows assumes that the default menu is desired.

3.8 Defining an Accelerator Table

Accelerators can be used in menus to shorten the time needed for users to select commands. To use accelerators, such as the accelerators given in SHAPES's menu, you must define the keys to be used and their value in an accelerator table in the resource script file.

The following is the accelerator table in SHAPES.RC:

```
shapes ACCELERATORS
BEGIN
     "^C", IDDCLEAR
     "^E", IDDELLIPSE
     "^R", IDDRECT
     "^S", IDDSTAR
     "^T", IDDTRIANGLE
END
```

The accelerator table begins with the **ACCELERATORS** statement. The table name is "shapes." The table contains five accelerators: ^C, ^E, ^R, ^S, and ^T. The values associated with these keys are given by the constants IDDCLEAR, IDDELLIPSE, IDDRECT, IDDSTAR, and IDDTRIANGLE. These names are assumed to have been defined in the include file SHAPES.H.

3.9 Loading and Using an Accelerator Table

An accelerator table, like other resources, must be loaded when the application first begins execution. The **LoadAccelerators** function in the **ShapesInit** function loads the accelerator table and returns a handle to the table:

```
hAccelTable = LoadAccelerators( hInstance, (LPSTR)szAppName );
```

The handle can be used in subsequent calls to the **TranslateAccelerator** function to refer to the table.

The application uses the accelerator table to translate keyboard input messages in the **WinMain** loop:

```
while (GetMessage((LPMSG)&msg, NULL, 0, 0)) {
    if (TranslateAccelerator(hWnd, hAccelTable, (LPMSG)&msg) == 0) {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage((LPMSG)&msg);
        }
    }
```

The **TranslateAccelerator** function converts key messages that match the accelerator keys into WM_COMMAND messages and dispatches the messages to the appropriate window function. The message value is equal to the value defined for the accelerator key in the accelerator table. The application passes all messages through the **TranslateAccelerator** function to look for accelerators. If the message is not an accelerator, the function returns zero and the application should process the message as usual, using **TranslateMessage** and **DispatchMessage**. If **TranslateAccelerator** returns a non-zero value, an accelerator has been found and processed, so no additional processing of the message is required.

3.10 The TRACK Application

The TRACK application is a graphics application that illustrates how to use the GDI output functions to create patterns and combine them with regular geometric shapes. This application requires a mouse device.

3.11 How TRACK Works

TRACK creates and displays a window that can contain a star, rectangle, triangle, or ellipse. When TRACK is first invoked, it is empty. You must pick the shape to draw by selecting one from the Shape menu:

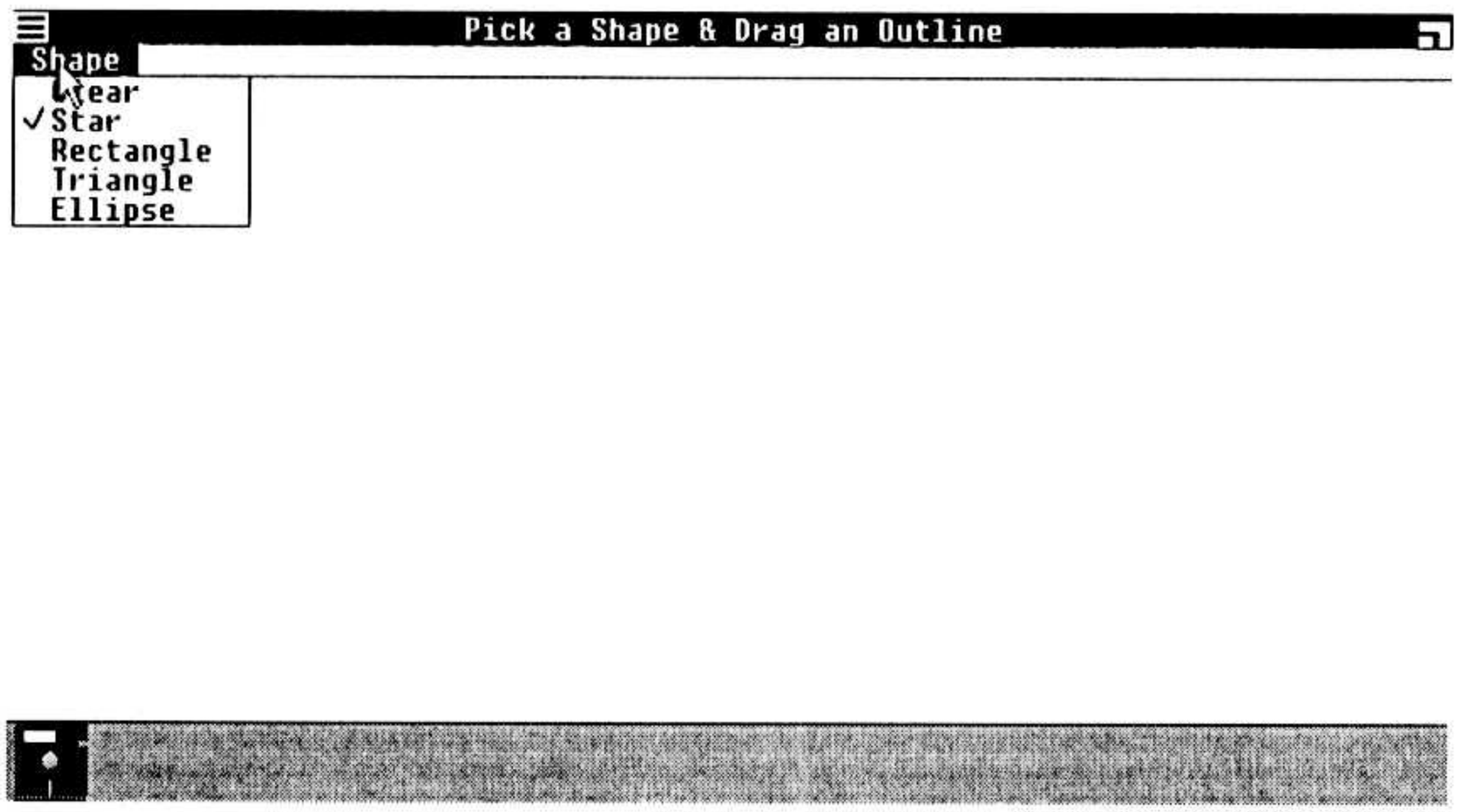


Figure 3.4 TRACK's Menu

The Shape menu is nearly the same as the menu in the SHAPES application. The order of the menu items is slightly different and a checkmark appears next to the Star command. This is the default shape if none is chosen.

To draw a shape, the user must create a drawing box in the client area. The user moves the mouse cursor into the client area, then holds down the left button and moves the mouse in any direction. Moving the mouse causes the drawing box to be highlighted in black:

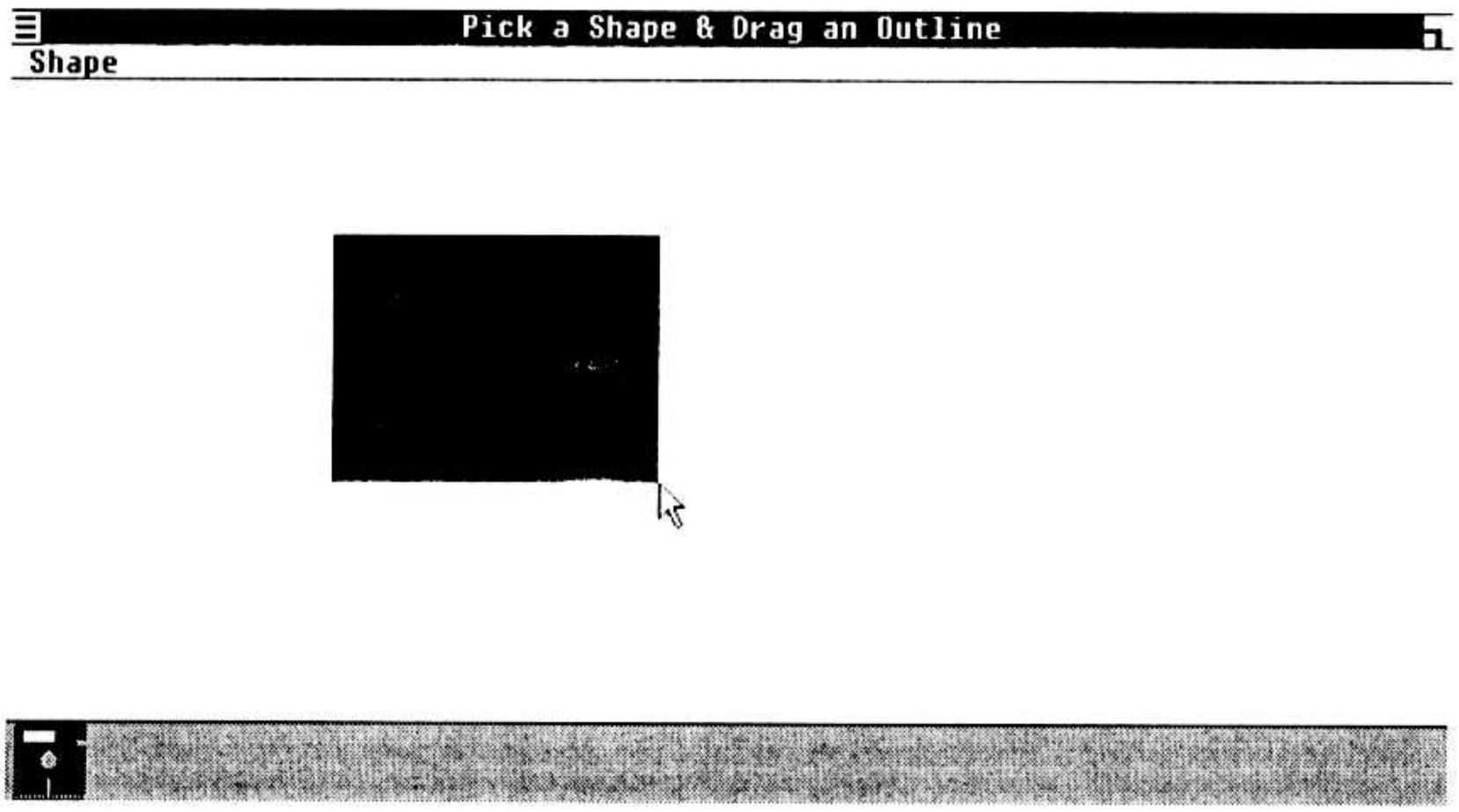


Figure 3.5 Track's Drawing Box

To draw the selected shape, the user releases the mouse button. The shape is adjusted to match the size of the drawing box. The box's highlight is removed:

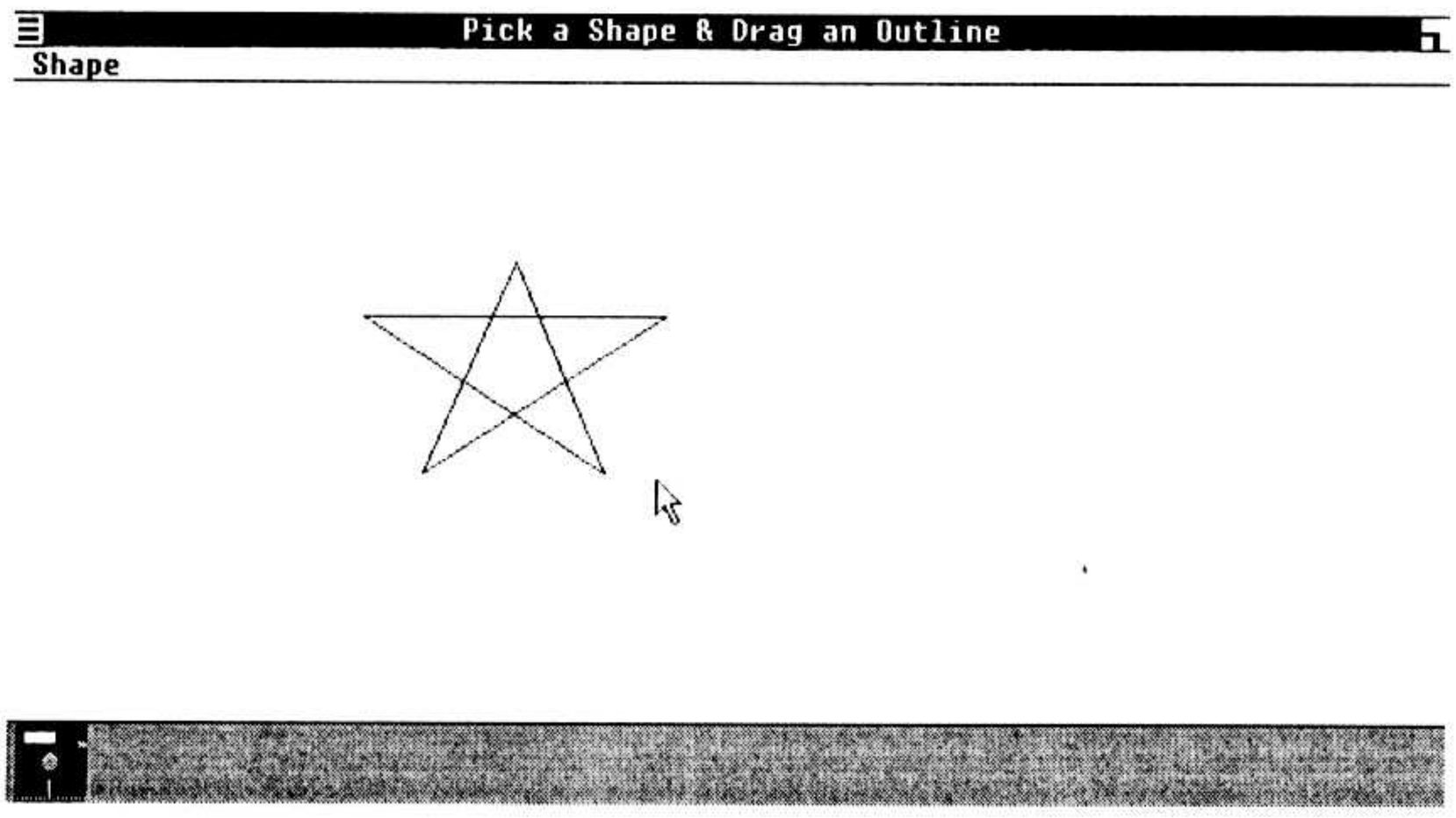


Figure 3.6 The Star

A change to the selected shape changes the window:

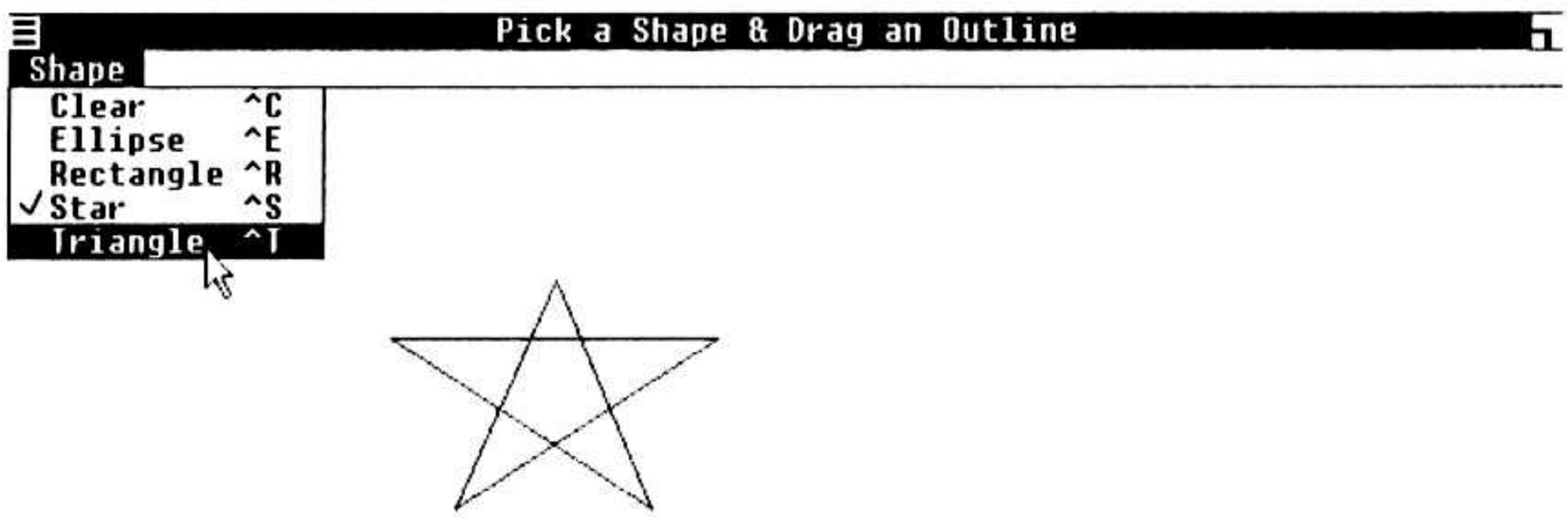


Figure 3.7 A Change of Selection

Drawing a new box removes the old shape and draws a new one:

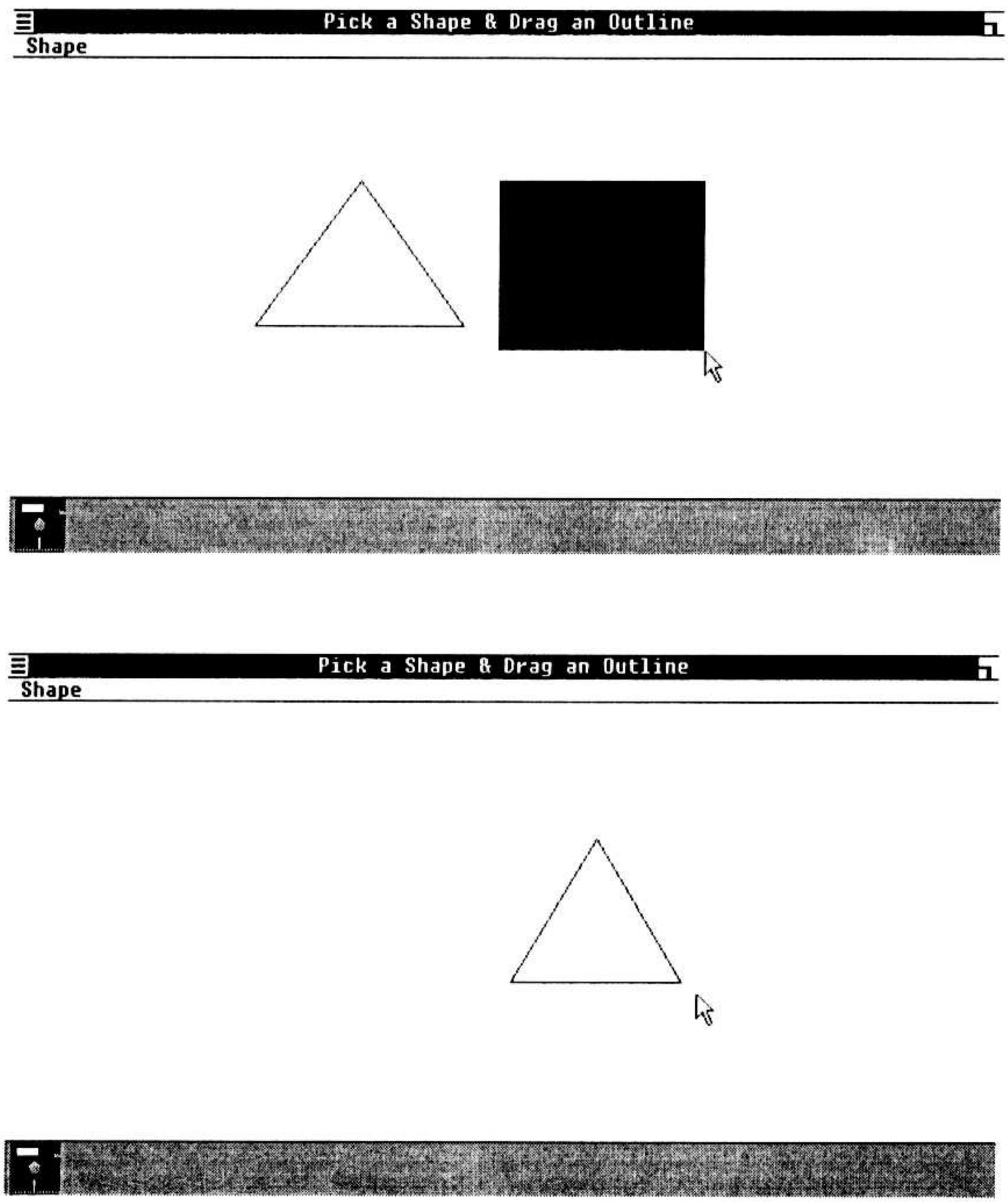


Figure 3.8 A New Shape

3.12 The TRACK.C File

The TRACK.C file defines a main function, **WinMain**, and a window function, **TrackWndProc**. The main function calls **TrackInit** to register a window class, and then calls the **CreateWindow** function to create the window.

The window function examines all input messages and responds to the WM_DESTROY, WM_PAINT, and WM_COMMAND messages and all mouse input messages. It calls a variety of local functions to carry out its tasks.

The following is a complete listing of TRACK.C:

```
/*  Track.c
    Track Application
    Windows Premiere Edition
    Copyright (c) Microsoft 1985        */

#include "windows.h"
#include "track.h"
#define  abs(x) max(x, -(x))

static    HBRUSH  hbrBlack;
static    RECT    rcPaintOutline;
static    HANDLE  hInst;
static    BOOL    bMouseDown = FALSE;
static    int     cCurrentShape = IDDSTAR;
static    HANDLE  hAccelTable;
static    POINT   pOrigin, pOld;
static    POINT   StarPoints[]=
    {
    100, 75,
    0, 75,
    80, 0,
    50, 100,
    20, 0,
    100, 75
    };
FARPROC lpprocAbout;

char *szAppName;
char *szAbout;
char *szWindowTitle;

long FAR PASCAL TrackWndProc(HWND, unsigned, WORD, LONG);
```

```
BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
    if (message == WM_COMMAND) {
        EndDialog( hDlg, TRUE );
        return TRUE;
        }
    else if (message == WM_INITDIALOG)
        return TRUE;
    else return FALSE;
}

TrackSetupPaintDC( hDC )
HDC hDC;
{
    SetMapMode (hDC, MM_ANISOTROPIC);
    SetWindowOrg (hDC, -10, 110);
    SetWindowExt (hDC, 120, -120);
    SetViewportOrg (hDC,
                    min (rcPaintOutline.right, rcPaintOutline.left),
                    min (rcPaintOutline.bottom, rcPaintOutline.top));
    SetViewportExt (hDC,
                    abs (rcPaintOutline.right-rcPaintOutline.left),
                    abs (rcPaintOutline.bottom-rcPaintOutline.top));
}


TrackPaint(hWnd, lpps)
HWND hWnd;
LPPAINTSTRUCT lpps;
{
    HDC hTrackDC;
    HBRUSH  hbr, hbrOld;
    HPEN    hpen, hpenOld;

    TrackSetupPaintDC(hTrackDC = lpps->hdc);

    if (lpps->fErase) {
        hbr = CreateSolidBrush(GetSysColor(COLOR_WINDOW));
        hbrOld = SelectObject(hTrackDC, hbr);
        FillRect(hTrackDC, (LPRECT)&lpps->rcPaint, hbr);
        SelectObject(hTrackDC, hbrOld);
        DeleteObject(hbr);
        }
    hpen = CreatePen(PS_SOLID, 0, GetSysColor(COLOR_WINDOWTEXT));
    hpenOld = SelectObject(hTrackDC, hpen);
    SelectObject(hTrackDC, GetStockObject(NULL_BRUSH));
```

```
    switch (cCurrentShape)
    {
    case IDDCLEAR:
        break;
    case IDDRECT:
        DrawRect(hTrackDC);
        break;
    case IDDELLIPSE:
        DrawEllipse(hTrackDC);
        break;
    case IDDTRIANGLE:
        DrawTriangle(hTrackDC);
        break;
    case IDDSTAR:
        DrawStar(hTrackDC);
        break;
    }
        SelectObject(hTrackDC, hpenOld);
        DeleteObject(hpen);
    }


    DrawRect (hDC)
    HDC    hDC;
    {
        Rectangle(hDC, 0, 0, 100, 100);
    }


    DrawEllipse (hDC)
    HDC    hDC;
    {
        Ellipse (hDC, 0, 100, 100, 0);
    }


    DrawTriangle (hDC)
    HDC    hDC;
    {
        MoveTo (hDC, 0, 0);
        LineTo (hDC, 100, 0);
        LineTo (hDC, 50, 100);
        LineTo (hDC, 0, 0);
    }


    DrawStar (hDC)
    HDC    hDC;
    {
        Polyline (hDC, StarPoints, 6);
    }
```

```
TrackCommand(hWnd, id)
HWND hWnd;
int id;
{
    HMENU    hMenu;

    hMenu = GetMenu (hWnd);
    CheckMenuItem(hMenu, cCurrentShape, MF_UNCHECKED);
    cCurrentShape = id;
    CheckMenuItem(hMenu, cCurrentShape, MF_CHECKED);
    InvalidateRect (hWnd, (LPRECT) NULL, (BOOL) TRUE);
}


TrackMouse(hWnd, code, pt)
HWND hWnd;
unsigned code;
POINT pt;
{
    static    HDC    hDC;

    switch (code)
    {
    case WM_MOUSEMOVE:
        if (!bMouseDown)
            return;
            break;
    case WM_LBUTTONDBLCLK:
    case WM_RBUTTONDBLCLK:
    case WM_LBUTTONDOWN:
    case WM_RBUTTONDOWN:
        if (bMouseDown)
            return;
        bMouseDown = TRUE;
        SetCapture(hWnd);
        hDC = GetDC (hWnd);
        SelectObject (hDC, hbrBlack);
        pOld.x = pOrigin.x = pt.x;
        pOld.y = pOrigin.y = pt.y;
        break;
case WM_LBUTTONUP:
case WM_RBUTTONUP:
    if (!bMouseDown)
        return;
    bMouseDown = FALSE;
    ReleaseCapture();
    rcPaintOutline.left = pOrigin.x;
    rcPaintOutline.top = pOrigin.y;
    rcPaintOutline.right = pt.x;
    rcPaintOutline.bottom = pt.y;
```

```
        PatBlt (hDC,
                pOrigin.x,
                pOrigin.y,
                pt.x - pOrigin.x,
                pt.y - pOrigin.y,
                DSTINVERT);
        ReleaseDC(hWnd, hDC);
        InvalidateRect (hWnd, (LPRECT) NULL, (BOOL) TRUE);
        return;
        break;
    default:
        return;
    }
    PatBlt (hDC,
            pOrigin.x,
            pOld.y,
            pt.x - pOrigin.x,
            pt.y - pOld.y,
            DSTINVERT);
        PatBlt (hDC,
            pOld.x,
            pOrigin.y,
            pt.x - pOld.x,
            pOld.y - pOrigin.y,
            DSTINVERT);
        pOld.x = pt.x;
        pOld.y = pt.y;
}


/* Procedure called every time a new instance of the application
** is created */

int PASCAL WinMain(hInstance, hPrevInstance, lpszCmdLine, cmdShow)
HANDLE hInstance, hPrevInstance;
LPSTR  lpszCmdLine;
int    cmdShow;
{
    MSG   msg;
    HWND  hWnd;
    HMENU hMenu;

    /* Loading from string table */
    szAppName = (char *)LocalAlloc( LPTR, 10 );
    szAbout   = (char *)LocalAlloc( LPTR, 10 );
    szWindowTitle = (char *)LocalAlloc( LPTR, 31 );
    LoadString( hInstance, IDSNAME, (LPSTR)szAppName, 10 );
    LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );
    LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 31 );

    if (!hPrevInstance)
        {
        if (!TrackInit(hInstance))
            return FALSE;
        }
        else
```

```
        {
        GetInstanceData(hPrevInstance,  (PSTR)&hbrBlack, sizeof(hbrBlack));
        GetInstanceData(hPrevInstance,  (PSTR)&hAccelTable, sizeof(hAccelTable));
        }

    hWnd = CreateWindow((LPSTR)szAppName,
                        (LPSTR)szWindowTitle,
                        WS_TILEDWINDOW,
                        0,
                        0,
                        0,
                        0,
                        (HWND)NULL,
                        (HMENU)NULL,
                        (HANDLE)hInstance,
                        (LPSTR)NULL);

        hInst = hInstance;
        lpprocAbout = MakeProcInstance( (FARPROC)About, hInstance);
        hMenu = GetSystemMenu(hWnd, FALSE);
        ChangeMenu(hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR);
        ChangeMenu(hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING);

        ShowWindow(hWnd, cmdShow);
        UpdateWindow(hWnd);

        while (GetMessage((LPMSG)&msg, NULL, 0, 0)) {
            if (TranslateAccelerator(hWnd, hAccelTable, (LPMSG)&msg) == 0) {
                TranslateMessage((LPMSG)&msg);
                DispatchMessage((LPMSG)&msg);
                }
            }
        exit(msg.wParam);
    }


    int TrackInit(hInstance)
    HANDLE hInstance;
    {
    PWNDCLASS   pTrackClass;

    /* set up some default brushes */
    hbrBlack = GetStockObject(BLACK_BRUSH);

    pTrackClass = (PWNDCLASS)LocalAlloc(LPTR, sizeof(WNDCLASS));

    pTrackClass->hCursor        = LoadCursor(NULL, IDC_ARROW);
    pTrackClass->hIcon          = LoadIcon(hInstance, (LPSTR)szAppName );
    pTrackClass->lpszMenuName   = (LPSTR)szAppName;
    pTrackClass->lpszClassName  = (LPSTR)szAppName;
    pTrackClass->hbrBackground  = NULL;
    pTrackClass->hInstance      = hInstance;
    pTrackClass->style          = CS_VREDRAW | CS_HREDRAW;
    pTrackClass->lpfnWndProc    = TrackWndProc;

    if (!RegisterClass((LPWNDCLASS)pTrackClass))
```

```
        return FALSE;

    LocalFree((HANDLE)pTrackClass);
    hAccelTable = LoadAccelerators(hInstance, (LPSTR)szAppName);
    return TRUE;
}

/* Procedures which make up the window class. */
long FAR PASCAL TrackWndProc(hWnd, message, wParam, lParam)
HWND        hWnd;
unsigned    message;
WORD        wParam;
LONG        lParam;
{
    PAINTSTRUCT  ps;
    RECT        rect;
    HBRUSH      hbr, hbrOld;

switch (message)
{
case WM_SYSCOMMAND:
    switch (wParam)
    {
    case IDSABOUT:
        DialogBox(hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout);
        break;
    default:
        return(DefWindowProc(hWnd, message, wParam, lParam));
        break;
    }
    break;

case WM_CREATE:
    CheckMenuItem(GetMenu(hWnd), cCurrentShape, MF_CHECKED);
    break;

case WM_MOUSEMOVE:
case WM_LBUTTONDOWN:
case WM_RBUTTONDOWN:
case WM_LBUTTONUP:
case WM_RBUTTONUP:
case WM_LBUTTONDBLCLK:
case WM_RBUTTONDBLCLK:
    TrackMouse(hWnd, message, MAKEPOINT(lParam));
    break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

case WM_PAINT:
    BeginPaint(hWnd, (LPPAINTSTRUCT)&ps);
    TrackPaint(hWnd, (LPPAINTSTRUCT)&ps);
    EndPaint(hWnd, (LPPAINTSTRUCT)&ps);
    break;
```

```
case WM_ERASEBKGND:
    GetClientRect(hWnd, (LPRECT)&rect);
    hbr = CreateSolidBrush(GetSysColor(COLOR_WINDOW));
    hbrOld = SelectObject((HDC)wParam, hbr);
    FillRect((HDC)wParam, (LPRECT)&rect, hbr);
    SelectObject((HDC)wParam, hbrOld);
    DeleteObject(hbr);
    break;

case WM_COMMAND:
    TrackCommand(hWnd, wParam);
        break;

default:
    return(DefWindowProc(hWnd, message, wParam, lParam));
    break;
}
    return(0L);
}
```

3.13 Setting the Coordinate System

The **TrackSetupPaintDC** function sets the window and viewport for the display context. The function maps a shape drawn in the logical coordinate system onto the area defined by the drawing box. **TrackSetupPaintDC** is called from the **TrackPaint** function before it decides what shape to draw.

To set the window and viewport, **TrackSetupPaintDC** must set the mapping mode to MM_ANISOTROPIC. No other mapping mode lets the application define its own window and viewport extents.

```
TrackSetupPaintDC( hDC )
HDC hDC;
{
    SetMapMode (hDC, MM_ANISOTROPIC);
    SetWindowOrg (hDC, -10, 110);
    SetWindowExt (hDC, 120, -120);
    SetViewportOrg (hDC,
                    min (rcPaintOutline.right, rcPaintOutline.left),
                    min (rcPaintOutline.bottom, rcPaintOutline.top));
    SetViewportExt (hDC,
                    abs (rcPaintOutline.right-rcPaintOutline.left),
                    abs (rcPaintOutline.bottom-rcPaintOutline.top));
}
```

It then uses the **SetWindowOrg** and **SetWindowExt** functions to define the origin and extents of the window. It uses the same values used in the SHAPES application.

TrackSetupPaintDC uses the **SetViewportOrg** and **SetViewportExt** functions to set the origin and extent of the viewport. The viewport, in this case, is the drawing box. The desired viewport origin is the upper left corner of the drawing box. The **min** macro is used to compute the coordinates of this corner. The desired extents are positive values. The **abs** macro guarantees positive results. The lower right corner of the resulting viewport has the same location as the lower right corner of the drawing box.

3.14 Reading and Setting the Menu

TrackWndProc calls **TrackCommand** whenever it receives a WM_COMMAND message. Windows generates these messages when the user selects a menu item. The *id* parameter receives the *wParam* value. This is equal to the value associated with the menu item as defined in the resource script file.

TrackCommand first removes any checkmark from the menu item. It uses the **GetMenu** function to get the handle of the menu. It uses the **CheckMenuItem** function to remove the mark.

```
TrackCommand(hWnd, id)
HWND hWnd;
int id;
{
    HMENU    hMenu;

    hMenu = GetMenu (hWnd);
    CheckMenuItem(hMenu, cCurrentShape, MF_UNCHECKED);
    cCurrentShape = id;
    CheckMenuItem(hMenu, cCurrentShape, MF_CHECKED);
    InvalidateRect (hWnd, (LPRECT) NULL, (BOOL) TRUE);
}
```

TrackCommand sets the static variable **cCurrentShape** to the value of **id**, then places a checkmark next to this menu item. The **InvalidateRect** function informs Windows that the window needs updating.

3.15 Making the Drawing Box

TRACK uses the mouse motions and button clicks to determine the starting point and dimensions of the drawing box. It uses the **PatBlt** function to color the box.

TrackWndProc calls the **TrackMouse** function whenever it detects a mouse input message:

```
case WM_MOUSEMOVE:
case WM_LBUTTONDOWN:
case WM_RBUTTONDOWN:
case WM_LBUTTONUP:
case WM_LBUTTONDBLCLK:
case WM_RBUTTONDBLCLK:
    TrackMouse(hWnd, message, MAKEPOINT(lParam));
    break;
```

TrackMouse checks for a mouse motion first. If a motion is detected and the box is being drawn (**bMouseDown** is TRUE), control passes to the end of the function where **TrackMouse** updates the size of the box.

```
case WM_MOUSEMOVE:
    if (!bMouseDown)
        return;
        break;
```

Otherwise, **TrackMouse** returns immediately.

TrackMouse then checks for down clicks and double clicks. If a click is detected, **TrackMouse** assumes that the user has just started drawing the box. It sets the **bMouseDown** flag to TRUE, if it is not already TRUE.

```
case WM_LBUTTONDBLCLK:
case WM_RBUTTONDBLCLK:
case WM_LBUTTONDOWN:
case WM_RBUTTONDOWN:
    if (bMouseDown)
        return;
    bMouseDown = TRUE;
    SetCapture(hWnd);
    hDC = GetDC (hWnd);
    SelectObject (hDC, hbrBlack);
    pOld.x = pOrigin.x = pt.x;
    pOld.y = pOrigin.y = pt.y;
    break;
```

TrackMouse calls the **SetCapture** function to capture all subsequent

mouse input. This prevents other applications from reading the input if the user pulls the cursor out of the window.

The **TrackMouse** function also retrieves a display context for use in painting the drawing box. TRACK, unlike TYPE and SHAPES, assigns the display context to a static variable, **hDC**, and does not release the context before returning to the window function. Since TRACK captures all mouse input while drawing the box, no application can make a request for the display context anyway. The display context remains valid until **TrackMouse** releases the capture.

The **SelectObject** function is used to select the black brush for subsequent drawing. The assignment statements at the end set the original point and previous points to the start of the drawing box.

If a mouse button is released, **TrackMouse** tests **bMouseDown** to see if it is currently making the drawing box. If not, the function returns immediately. If it is making the box, the button release marks its end.

```
case WM_LBUTTONUP:
case WM_RBUTTONUP:
    if (!bMouseDown)
        return;
    bMouseDown = FALSE;
    ReleaseCapture();
    rcPaintOutline.left = pOrigin.x;
    rcPaintOutline.top = pOrigin.y;
    rcPaintOutline.right = pt.x;
    rcPaintOutline.bottom = pt.y;
    PatBlt (hDC,
            pOrigin.x,
            pOrigin.y,
            pt.x - pOrigin.x,
            pt.y - pOrigin.y,
            DSTINVERT);
    ReleaseDC(hWnd, hDC);
    InvalidateRect (hWnd, (LPRECT) NULL, (BOOL) TRUE);
    return;
    break;
default:
    return;
}
```

TrackMouse releases the mouse capture by calling **ReleaseCapture**. It then saves the current x and y coordinates and the original x and y coordinates in the static structure **rcPaintOutline**. The **PatBlt** function is used to restore the highlighted box to its original color. The **ReleaseDC** releases the display context used to draw the box, and **InvalidateRect** informs Windows that the screen needs updating. **TrackMouse** returns immediately, skipping the last part of the function.

If the function has not returned at this point, the move is valid and the drawing box needs to be adjusted. **TrackMouse** adjusts the box by calling the **PatBlt** function twice.

The first call inverts a small rectangle at the bottom (or top) of the box.

```
PatBlt (hDC,
        pOrigin.x,
        pOld.y,
        pt.x - pOrigin.x,
        pt.y - pOld.y,
        DSTINVERT);
```

The height of this rectangle is the difference between the previous y coordinate, **pOld.y**, and the new y coordinate, **pt.y**. The width of the rectangle is the difference between the original x coordinate, **pOrigin.x**, and the current coordinate, **pt.x**.

The second call inverts a small rectangle to the right (or left) of the box.

```
PatBlt (hDC,
        pOld.x,
        pOrigin.y,
        pt.x - pOld.x,
        pOld.y - pOrigin.y,
        DSTINVERT);
```

The height of this rectangle is the difference between the original y coordinate, **pOrigin.y**, and the previous y coordinate, **pOld.y**. The width of the rectangle is the difference between the previous x coordinate, **pOld.x**, and the current coordinate, **pt.x**.

The last two statements save the current position:

```
pOld.x = pt.x;
pOld.y = pt.y;
```

3.16 Borrowing Data from the Previous Instance

A new instance of an application can borrow data from the previous instance by using the **GetInstanceData** function. **WinMain** uses this function to copy the data associated with the brush handle **hbrBlack** and the accelerator table **hAccelTable** into the new instance:

```
GetInstanceData(hPrevInstance, (PSTR)&hbrBlack, sizeof(hbrBlack));
GetInstanceData(hPrevInstance, (PSTR)&hAccelTable, sizeof(hAccelTable));
```

The functions are called only if a previous instance exists. The module instance handle **hPrevInstance** indicates the module to borrow from. The address offsets of the variables **hbrBlack** and **hAccelTable** define where the data is to be copied.

Chapter 4
Using Fonts and Dialog Boxes

4.1 Introduction 81
4.2 What FONTTEST Does 81
4.3 The Source File: FONTTEST.C 84
4.4 Reading FONTTEST's Menu 91
4.5 Displaying the Sample String 92
4.6 FONTTEST's Dialog Box 93
4.6.1 Creating and Displaying a Dialog Box 93
4.6.2 Processing a Dialog Box's Messages 93
4.6.3 Initializing the Dialog Box Controls 94
4.6.4 Carrying Out Dialog Commands 95
4.7 The Resource Script: FONTTEST.RC 97
4.8 The Module Definition File: FONTTEST.DEF 100

4.1 Introduction

This chapter explains how to use fonts and dialog boxes in an application. It presents a single application named FONTTEST that prompts for user input through a dialog box and displays a character string using a font whose attributes have been selected by the user.

4.2 What FONTTEST Does

FONTTEST is a test application that lets the user examine the fonts available on a given machine. FONTTEST displays a character string in the current font, then permits the user to invoke a dialog box that can be used to change the attributes of the current font, letting the user compare the same string in two different fonts.

FONTTEST initially looks like this:

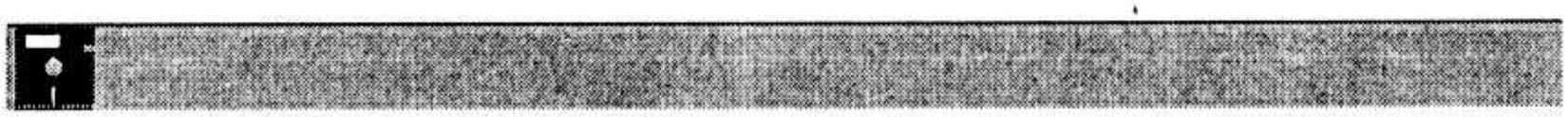


Figure 4.1 FONTTEST Window

The user can invoke the dialog box by selecting the Options... command in the Command menu:

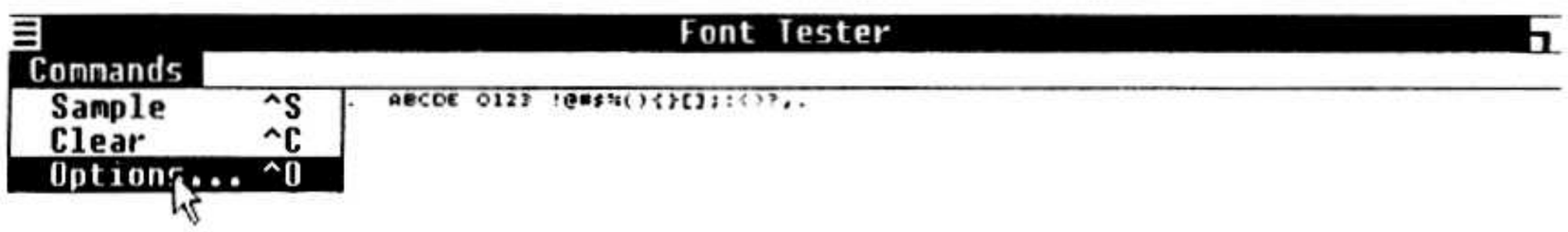


Figure 4.2 Selecting the Options... Command

The dialog box is displayed in the center of the window:

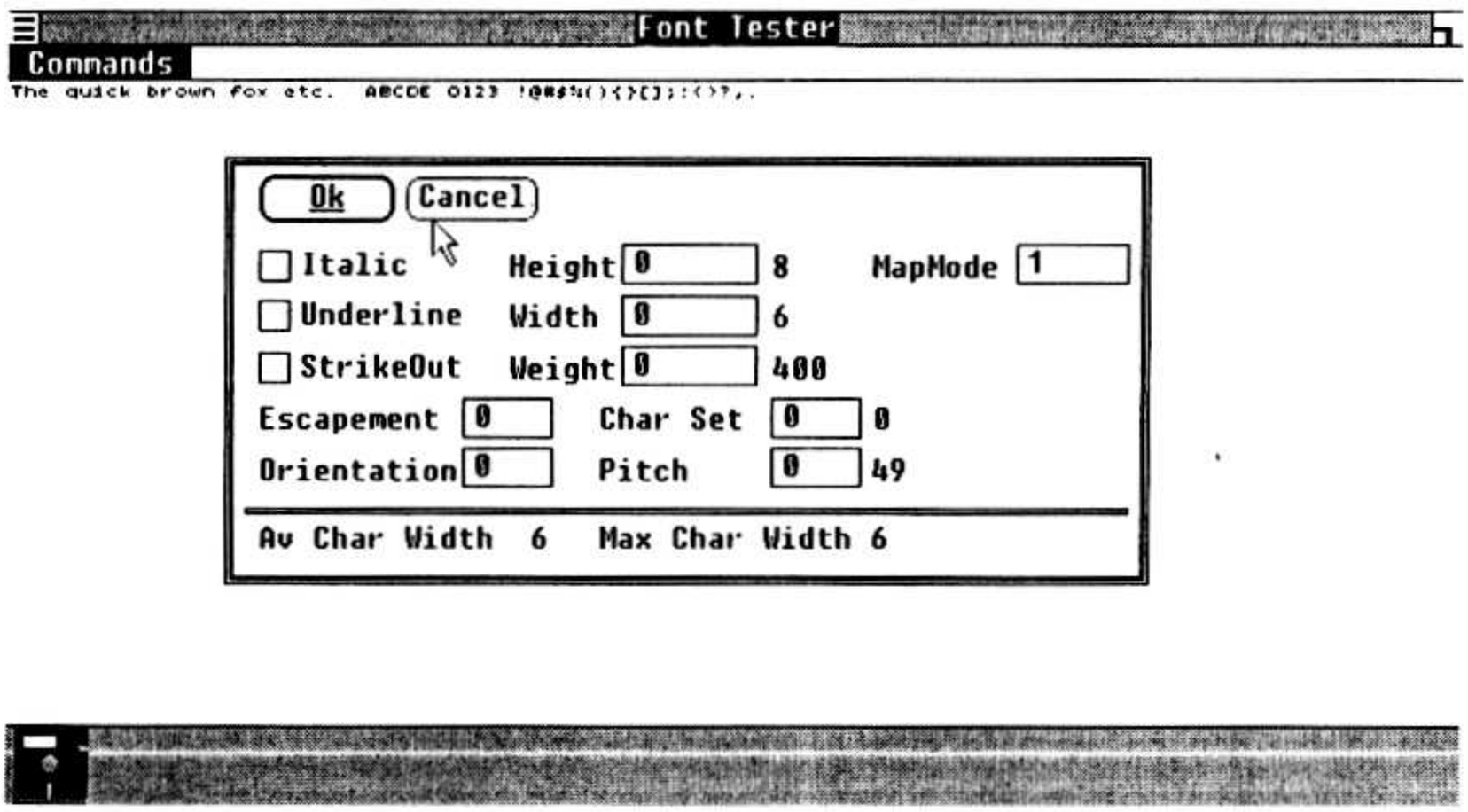


Figure 4.3 Dialog Box

Each labeled entry in the dialog box is a control window in which the user can enter text or select. Each control represents an attribute of the current font. The user can make changes to the attributes by using the mouse to select a control and then using the keyboard to enter a change.

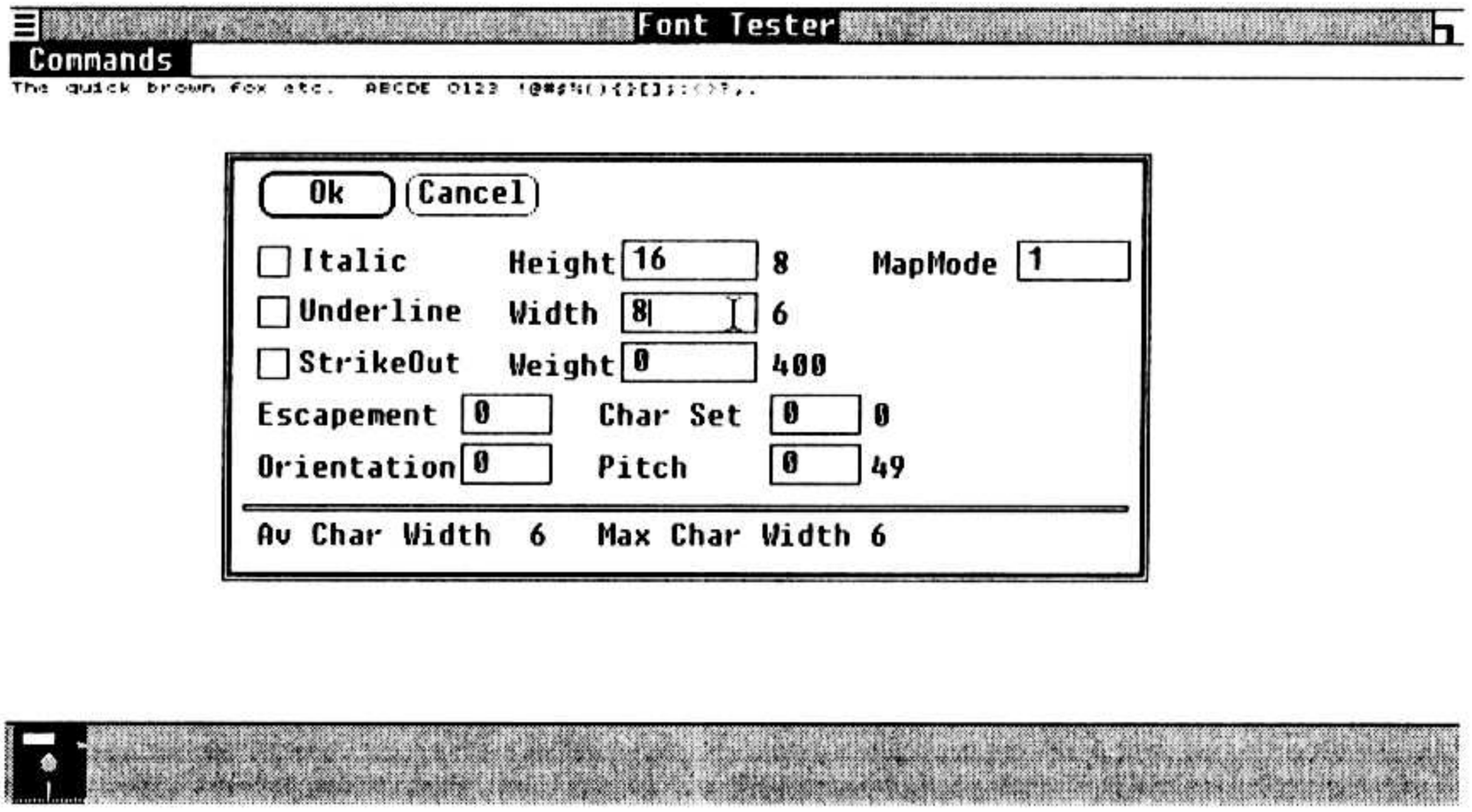


Figure 4.4 Changing A Control

The I-Beam cursor indicates that the mouse is in a control window. A flashing caret appears when the control is ready to accept text from the keyboard.

To initiate the changes made to the font, the user must point to the Ok button and click a mouse button. The dialog box is removed. To display the string, the user can select the Sample command in the Command menu, or press the CNTRL-S key, an accelerator for the Sample command. The same string in the new font appears in the following line:

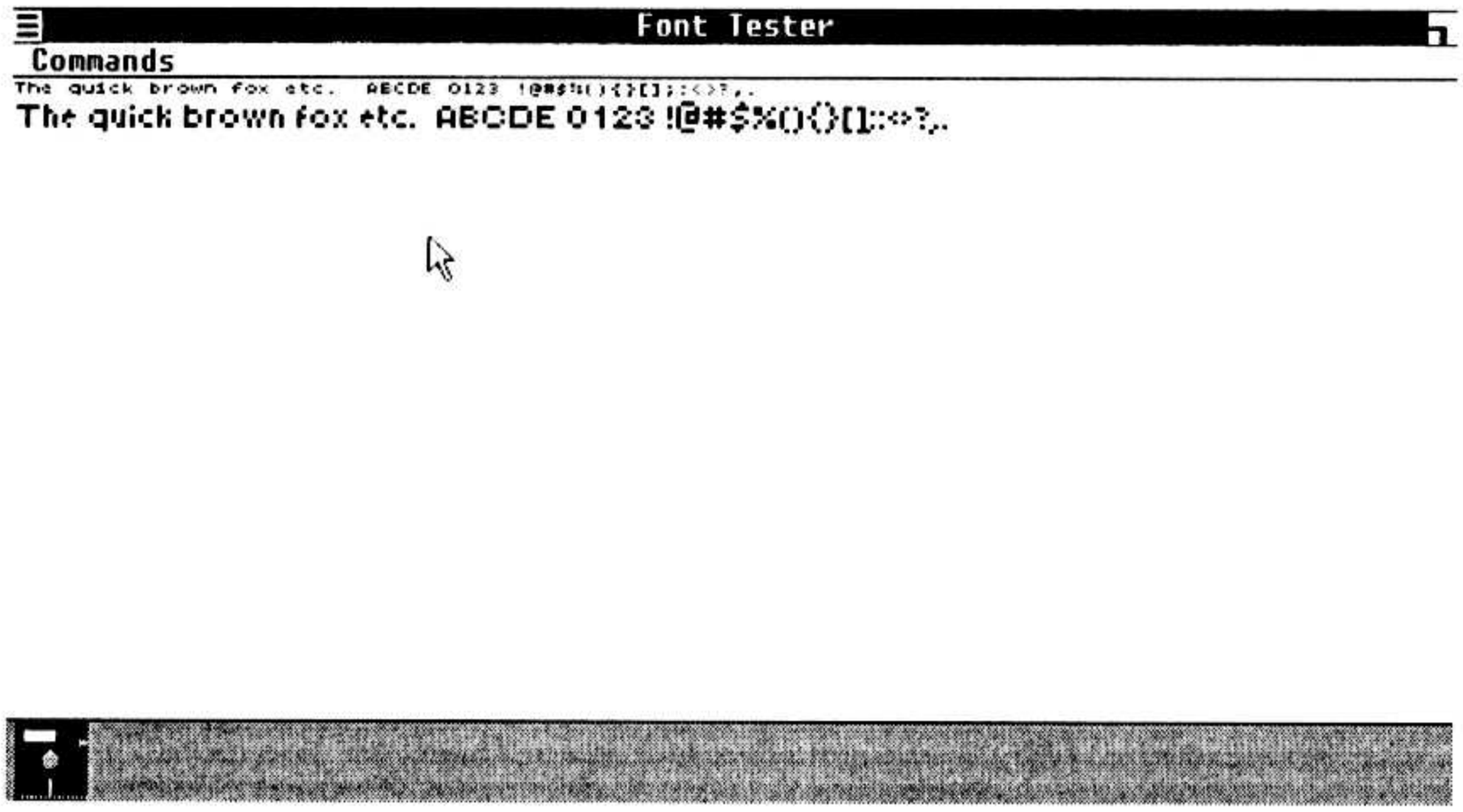


Figure 4.5 Character Height Increased

4.3 The Source File: FONTTEST.C

The FONTTEST.C file defines the following functions:

WinMain	The main function
FonttestInit	A window class initialization function
FonttestWndProc	The window function
FonttestCommand	A local function to process WM_COMMAND
ShowSampleString	A local function to display text
ChangeOption	The dialog input function
InitDlg	A dialog initialization function
DlgCommand	A local function to process dialog commands

The following is a complete listing of FONTTEST.C:

```
/*  Fonttest.c
    Fonttest Application
    Windows Premiere Edition
    Copyright (c) Microsoft 1985        */

#include "windows.h"
#include "fonttest.h"

typedef BOOL FAR * LPBOOL;
```

```
extern long FAR PASCAL lstrcpy();
extern long FAR PASCAL lstrlen();

FARPROC       lpprocOptions;
FARPROC       lpprocAbout;
HANDLE        hInst;
HANDLE        hAccelTable;  /* handle to accelerator table */
int           FontYCoord;
short         nMapMode;
LOGFONT       LogFont;
LPLOGFONT     lpLogFont;
TEXTMETRIC    TM;
LPTEXTMETRIC  lpTM = (LPTEXTMETRIC)&TM;

char *szAppName;
char *szAbout;
char *szWindowTitle;
char *szAlphabet;


BOOL FAR PASCAL About( hDlg, message, wParam, lParam )
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
    if (message == WM_COMMAND) {
        EndDialog( hDlg, TRUE );
        return TRUE;
        }
    else if (message == WM_INITDIALOG)
        return TRUE;
    else return FALSE;
}


ShowSampleString(hDC)
HDC hDC;
{
    HANDLE hLogFont, hLogFontOld;

    hLogFont = CreateFontIndirect(lpLogFont);
    SetMapMode(hDC, nMapMode);
    hLogFontOld = SelectObject(hDC, hLogFont);
    GetTextMetrics(hDC, (TEXTMETRIC FAR *)lpTM);

    TextOut(hDC, 0, FontYCoord, (LPSTR)szAlphabet,
                                (short)lstrlen((LPSTR)szAlphabet));
    FontYCoord += lpTM->tmHeight + lpTM->tmExternalLeading;
    SelectObject(hDC, hLogFontOld);
    DeleteObject(hLogFont);
}
```

```
InitDlg(hDlg)
HWND hDlg;
{
    if (lpLogFont->lfUnderline)
        CheckDlgButton(hDlg, IDUNDERLINE, TRUE);
    if (lpLogFont->lfStrikeOut)
        CheckDlgButton(hDlg, IDSTRIKEOUT, TRUE);
    if (lpLogFont->lfItalic)
        CheckDlgButton(hDlg, IDITALIC, TRUE);
    SetDlgItemInt(hDlg, IDHEIGHT, lpLogFont->lfHeight, FALSE);
    SetDlgItemInt(hDlg, IDTMHEIGHT, lpTM->tmHeight, FALSE);
    SetDlgItemInt(hDlg, IDWIDTH,  lpLogFont->lfWidth, FALSE);
    SetDlgItemInt(hDlg, IDTMWIDTH,lpTM->tmAveCharWidth, FALSE);
    SetDlgItemInt(hDlg, IDWEIGHT, lpLogFont->lfWeight, FALSE);
    SetDlgItemInt(hDlg, IDTMWEIGHT, lpTM->tmWeight, FALSE);
    SetDlgItemInt(hDlg, IDAVCHARWIDTH, lpTM->tmAveCharWidth, FALSE);
    SetDlgItemInt(hDlg, IDMAXCHARWIDTH, lpTM->tmMaxCharWidth, FALSE);
    SetDlgItemInt(hDlg, IDESCAPEMENT, lpLogFont->lfEscapement, FALSE);
    SetDlgItemInt(hDlg, IDORIENTATION,lpLogFont->lfOrientation,FALSE);
    SetDlgItemInt(hDlg, IDPITCH, lpLogFont->lfPitchAndFamily, FALSE);
    SetDlgItemInt(hDlg, IDTMPITCH, 1 + lpTM->tmPitchAndFamily, FALSE);
    SetDlgItemInt(hDlg, IDCHARSET, lpLogFont->lfCharSet, FALSE);
    SetDlgItemInt(hDlg, IDTMCHARSET, lpTM->tmCharSet, FALSE);
    SetDlgItemInt(hDlg, IDMAPMODE, nMapMode, FALSE);
}


DlgCommand (hDlg, wParam, lParam)
HWND hDlg;
WORD wParam;
LONG lParam;
{
    BOOL fValOK;

    switch (wParam)
    {
    case IDOK:
        lpLogFont->lfItalic = IsDlgButtonChecked (hDlg, IDITALIC)
                              ? 1 : 0;
lpLogFont->lfUnderline = IsDlgButtonChecked (hDlg, IDUNDERLINE)
                         ? 1 : 0;
lpLogFont->lfStrikeOut = IsDlgButtonChecked (hDlg, IDSTRIKEOUT)
                         ? 1 : 0;
lpLogFont->lfWeight = GetDlgItemInt
                          (hDlg,
                          IDWEIGHT,
                          (LPBOOL)&fValOK,
                          FALSE);
lpLogFont->lfWidth = GetDlgItemInt
                         (hDlg,
                         IDWIDTH,
                         (LPBOOL)&fValOK,
                         FALSE);
lpLogFont->lfHeight = GetDlgItemInt
                          (hDlg,
                          IDHEIGHT,
```

```
                                    (LPBOOL)&fValOK,
                                    FALSE);
lpLogFont->lfEscapement = GetDlgItemInt
                                    (hDlg,
                                    IDESCAPEMENT,
                                    (LPBOOL)&fValOK,
                                    FALSE);
lpLogFont->lfOrientation = GetDlgItemInt
                                    (hDlg,
                                    IDORIENTATION,
                                    (LPBOOL)&fValOK,
                                    FALSE);
lpLogFont->lfPitchAndFamily = (BYTE)GetDlgItemInt
                                    (hDlg,
                                    IDPITCH,
                                    (LPBOOL)&fValOK,
                                    FALSE);
        lpLogFont->lfCharSet = (BYTE)GetDlgItemInt
                                    (hDlg,
                                    IDCHARSET,
                                    (LPBOOL)&fValOK,
                                    FALSE);
        nMapMode = GetDlgItemInt(hDlg, IDMAPMODE, (LPBOOL)&fValOK, FALSE);
        EndDialog(hDlg, TRUE);
        break;

    case IDCANCEL:
        EndDialog(hDlg, TRUE);
        break;

    case IDITALIC:
    case IDUNDERLINE:
    case IDSTRIKEOUT:
        CheckDlgButton(hDlg,  wParam,
                    (IsDlgButtonChecked(hDlg, wParam) ? FALSE : TRUE));
        break;
    default:
        break;
        }
}


BOOL far PASCAL ChangeOptions(hDlg, message, wParam, lParam)
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
    switch (message)
    {
    case WM_COMMAND:
        DlgCommand(hDlg, wParam, lParam);
        break;
```

```
    case WM_INITDIALOG:
        InitDlg(hDlg);
        break;
    default:
        return FALSE;
    }
    return TRUE;
}


FonttestCommand(hWindow, id)
HWND hWindow;
WORD id;
{
    int i;
    HDC hDC;

    switch (id)
    {
    case IDMSAMPLE:
        hDC = GetDC(hWindow);
        ShowSampleString(hDC);
        ReleaseDC(hWindow, hDC);
        break;
    case IDMCLEAR:
        InvalidateRect(hWindow, (LPRECT)NULL, TRUE);
        UpdateWindow(hWindow);
        break;
    case IDMOPTIONS:
        DialogBox(hInst, MAKEINTRESOURCE(OPTIONSBOX), hWindow, lpprocOptions);
        break;
    default:
        break;
    }
}


long FAR PASCAL FonttestWndProc(hWnd, message,  wParam, lParam)
HWND hWnd;
int  message;
WORD wParam;
long lParam;
{
    PAINTSTRUCT ps;

    switch (message)
    {
    case WM_SYSCOMMAND:
        switch (wParam)
        {
        case IDSABOUT:
            DialogBox(hInst, MAKEINTRESOURCE(ABOUTBOX), hWnd, lpprocAbout);
            break;
        default:
            return(DefWindowProc(hWnd, message, wParam, lParam));
            break;
```

```
        }
        break;

    case WM_PAINT:
        BeginPaint(hWnd, (LPPAINTSTRUCT)&ps);
        EndPaint(hWnd, (LPPAINTSTRUCT)&ps);
        FontYCoord = 0;
        return (long)TRUE;
        break;

    case WM_DESTROY:
        PostQuitMessage(0);
        break;

    case WM_COMMAND:
        FonttestCommand(hWnd, wParam);
        return (long) TRUE;

    default:
        return (DefWindowProc(hWnd, message, wParam, lParam));
        break;
    }
}

int PASCAL WinMain(hInstance, hPrev, lpszCmdLine, cmdShow)
HANDLE hInstance, hPrev;
LPSTR lpszCmdLine;
int cmdShow;
{
    MSG msg;
    HWND hWnd;
    HMENU hMenu;

    /* Loading from string table */
    szAppName = (char *)LocalAlloc( LPTR, 10 );
    szAbout   = (char *)LocalAlloc( LPTR, 10 );
    szWindowTitle = (char *)LocalAlloc( LPTR, 15 );
    szAlphabet = (char *)LocalAlloc( LPTR, 56 );
    LoadString( hInstance, IDSNAME, (LPSTR)szAppName, 10 );
    LoadString( hInstance, IDSABOUT, (LPSTR)szAbout, 10 );
    LoadString( hInstance, IDSTITLE, (LPSTR)szWindowTitle, 15 );
    LoadString( hInstance, IDSALPHABET, (LPSTR)szAlphabet, 56 );

    if (!hPrev)
    {
        if (!FonttestInit(hInstance))
            return(FALSE);
    }
    else
    {
        GetInstanceData(hPrev, (PSTR)&LogFont,     sizeof(LogFont));
        GetInstanceData(hPrev, (PSTR)&lpLogFont,   sizeof(lpLogFont));
        GetInstanceData(hPrev, (PSTR)&hInst,       sizeof(hInst));
        GetInstanceData(hPrev, (PSTR)&TM,          sizeof(TM));
        GetInstanceData(hPrev, (PSTR)&lpTM,        sizeof(lpTM));
        GetInstanceData(hPrev, (PSTR)&nMapMode,    sizeof(nMapMode));
```

```
        GetInstanceData(hPrev, (PSTR)&hAccelTable, sizeof(hAccelTable));
}
        FontYCoord = 0;
        nMapMode = 1;  /* the default mapmode */

hWnd = (HWND)CreateWindow(
        (LPSTR)szAppName,
        (LPSTR)szWindowTitle,
        WS_TILEDWINDOW,
        0, 0, 0, 0,
        (HWND)NULL,
        (HMENU)NULL,
        (HANDLE)hInstance,
        (LPSTR)NULL
    );

ShowWindow(hWnd, cmdShow);
    UpdateWindow(hWnd);

    hInst = hInstance;
    lpprocOptions = MakeProcInstance((FARPROC)ChangeOptions, hInstance);
    lpprocAbout = MakeProcInstance( (FARPROC)About, hInstance);
    hMenu = GetSystemMenu(hWnd, FALSE);
    ChangeMenu(hMenu, 0, NULL, 999, MF_APPEND | MF_SEPARATOR);
    ChangeMenu(hMenu, 0, (LPSTR)szAbout, IDSABOUT, MF_APPEND | MF_STRING);

    while (GetMessage((LPMSG)&msg, NULL, 0, 0)) {
        if (TranslateAccelerator(hWnd, hAccelTable, (LPMSG)&msg) == 0) {
            TranslateMessage((LPMSG)&msg);
            DispatchMessage((LPMSG)&msg);
            }
        }
    exit(msg.wParam);
}

FonttestInit(hInstance)
HANDLE hInstance;
{
    PWNDCLASS   pTemplateClass;

    lpLogFont = (LPLOGFONT)&LogFont;

    pTemplateClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS));

    pTemplateClass->hCursor         = LoadCursor(NULL, IDC_ARROW);
    pTemplateClass->hIcon           = LoadIcon(hInstance, (LPSTR)szAppName);
    pTemplateClass->lpszMenuName    = (LPSTR)szAppName;
    pTemplateClass->lpszClassName   = (LPSTR)szAppName;
    pTemplateClass->hbrBackground   = GetStockObject (WHITE_BRUSH);
    pTemplateClass->hInstance       = hInstance;
    pTemplateClass->lpfnWndProc     = FonttestWndProc;
    pTemplateClass->style           = CS_VREDRAW | CS_HREDRAW;

    if (!RegisterClass((LPWNDCLASS)pTemplateClass))
        return FALSE;
```

```
    LocalFree((HANDLE)pTemplateClass);
    hAccelTable = LoadAccelerators(hInstance, (LPSTR)szAppName);

    return TRUE;
}
```

4.4 Reading FONTTEST's Menu

The **FonttestCommand** function processes command input from FONTTEST's menu. The function recognizes three command values: IDMSAMPLE, IDMCLEAR, and IDMOPTIONS. These values correspond to the menu commands: Sample, Clear, and Options.

FonttestWndProc calls **FonttestCommand** whenever it receives a WM_COMMAND message. The window function passes the **wParam** value of the message to the **id** parameter. **FonttestCommand** uses this parameter to select an action. Its value is equal to the value associated with the command, as defined by the resource script file.

```
FonttestCommand(hWindow, id)
HWND hWindow;
WORD id;
{
    int i;
    HDC hDC;

    switch (id)
    {
    case IDMSAMPLE:
        hDC = GetDC(hWindow);
        ShowSampleString(hDC);
        ReleaseDC(hWindow, hDC);
        break;
    case IDMCLEAR:
        InvalidateRect(hWindow, (LPRECT)NULL, TRUE);
        UpdateWindow(hWindow);
        break;
    case IDMOPTIONS:
        DialogBox(hInst, MAKEINTRESOURCE(OPTIONSBOX), hWindow, lpprocOptions);
        break;
    default:
        break;
    }
}
```

The IDMCLEAR command directs FONTTEST to clear the screen. **FonttestCommand** calls the **InvalidateRect** and **UpdateWindow** functions to update the window immediately. The contents of the windows are cleared.

The IDMSAMPLE command directs FONTTEST to retrieve a display context and call the **ShowSampleString** function. This function uses the currently specified font to display a character string. The display context is released as soon as **ShowSampleString** returns.

4.5 Displaying the Sample String

The **ShowSampleString** function creates a physical font based on the attributes of the currently selected logical font and uses this font to display the character string:

```
The quick brown fox etc.  ABCDE 0123 !@#$%(){}[];:<>?,.
```

ShowSampleString is called by the **FonttestCommand** function, which passes it a display context. Before the string can be displayed, the function uses the **SetMapMode** and **SelectObject** functions to set the current mapping mode and select the created font. The **TextOut** function writes the string.

```
ShowSampleString(hDC)
HDC hDC;
{
    HANDLE hLogFont, hLogFontOld;

    hLogFont = CreateFontIndirect(lpLogFont);
    SetMapMode(hDC, nMapMode);
    hLogFontOld = SelectObject(hDC, hLogFont);
    GetTextMetrics(hDC, (TEXTMETRIC FAR *)lpTM);

    TextOut(hDC, 0, FontYCoord, (LPSTR)szAlphabet,
                                (short)lstrlen((LPSTR)szAlphabet));
    FontYCoord += lpTM->tmHeight + lpTM->tmExternalLeading;
    SelectObject(hDC, hLogFontOld);
    DeleteObject(hLogFont);
}
```

The **GetTextMetrics** function retrieves information about the currently selected font. **ShowSampleString** uses this information, stored in the TEXTMETRIC data structure pointed to by **lpTM**, to adjust the static variable **FontYCoord**.

FontYCoord contains the y coordinate of the next line to receive a string. The variable must be adjusted after each new string to prevent the next string from overwriting the previous string. The **tmHeight** and **tmExternalLeading** fields of the TEXTMETRIC data structure define the vertical distance from the top of the highest character in the current line to the top

of the highest character in the following line. These values are different for each font.

To save space, **ShowSampleString** deletes each font after using it. The **DeleteObject** function frees all memory occupied by the font.

4.6 FONTTEST's Dialog Box

FONTTEST uses a dialog box to prompt the user for changes to the fonts to be displayed. The dialog box contains several predefined control windows. When FONTTEST creates the dialog box, the user can enter text or select any combination of the controls in the box. The dialog box takes control and keeps it until the user selects the Ok or Cancel button.

4.6.1 Creating and Displaying a Dialog Box

The IDMOPTIONS command directs FONTTEST to create and display a dialog box and wait until the box returns before continuing execution. The **DialogBox** function creates a dialog box using the dialog template identified by OPTIONSBOX. This template must be defined in the application's resource script file. The dialog box's parent is given by **hWindow**.

```
case IDMOPTIONS:
    DialogBox(hInst, MAKEINTRESOURCE(OPTIONSBOX), hWindow, lpprocOptions);
    break;
```

The **lpprocOptions** argument defines the dialog box input function. This pointer contains the instance address of the **ChangeOptions** function. The address is created by the **MakeProcInstance** function in **WinMain**. An instance address refers to a specific instance of the application and guarantees that the dialog box input function will use the correct data no matter how many instances of the application are executing.

4.6.2 Processing a Dialog Box's Messages

Every dialog box needs an input function to process messages sent to it by its parent and by Windows. A dialog input function does the same job as a window function, but has a limited scope and does not require use of the **DefWindowProc** function.

The **ChangeOptions** command receives messages sent to it by Windows and by the dialog box's controls. Windows sends an WM_INITDIALOG message when it initializes the dialog box. It sends a WM_COMMAND whenever the user selects or changes a control.

```
BOOL far PASCAL ChangeOptions(hDlg, message, wParam, lParam)
HWND hDlg;
unsigned message;
WORD wParam;
LONG lParam;
{
    switch (message)
    {
    case WM_COMMAND:
        DlgCommand(hDlg, wParam, lParam);
        break;
    case WM_INITDIALOG:
        InitDlg(hDlg);
        break;
    default:
        return FALSE;
    }
    return TRUE;
}
```

ChangeOptions calls the **DlgCommand** function whenever it receives a WM_COMMAND message.

The function calls the **InitDlg** message when it receives an WM_INITDIALOG message.

4.6.3 Initializing the Dialog Box Controls

The **InitDlg** function initializes the control windows in the dialog box when the dialog box is created. The function uses **CheckDlgButton** and **SetDlgItemInt** functions to set the control values. It copies the initial values from the LOGFONT data structure pointed to by **lpLogFont**.

```
InitDlg(hDlg)
HWND hDlg;
{
    if (lpLogFont->lfUnderline)
        CheckDlgButton(hDlg, IDUNDERLINE, TRUE);
    if (lpLogFont->lfStrikeOut)
        CheckDlgButton(hDlg, IDSTRIKEOUT, TRUE);
    if (lpLogFont->lfItalic)
        CheckDlgButton(hDlg, IDITALIC, TRUE);
    SetDlgItemInt(hDlg, IDHEIGHT, lpLogFont->lfHeight, FALSE);
    SetDlgItemInt(hDlg, IDTMHEIGHT, lpTM->tmHeight, FALSE);
    SetDlgItemInt(hDlg, IDWIDTH,  lpLogFont->lfWidth, FALSE);
    SetDlgItemInt(hDlg, IDTMWIDTH,lpTM->tmAveCharWidth, FALSE);
```

```
    SetDlgItemInt(hDlg, IDWEIGHT, lpLogFont->lfWeight, FALSE);
    SetDlgItemInt(hDlg, IDTMWEIGHT, lpTM->tmWeight, FALSE);
    SetDlgItemInt(hDlg, IDAVCHARWIDTH, lpTM->tmAveCharWidth, FALSE);
    SetDlgItemInt(hDlg, IDMAXCHARWIDTH, lpTM->tmMaxCharWidth, FALSE);
    SetDlgItemInt(hDlg, IDESCAPEMENT, lpLogFont->lfEscapement, FALSE);
    SetDlgItemInt(hDlg, IDORIENTATION,lpLogFont->lfOrientation,FALSE);
    SetDlgItemInt(hDlg, IDPITCH, lpLogFont->lfPitchAndFamily, FALSE);
    SetDlgItemInt(hDlg, IDTMPITCH, 1 + lpTM->tmPitchAndFamily, FALSE);
    SetDlgItemInt(hDlg, IDCHARSET, lpLogFont->lfCharSet, FALSE);
    SetDlgItemInt(hDlg, IDTMCHARSET, lpTM->tmCharSet, FALSE);
    SetDlgItemInt(hDlg, IDMAPMODE, nMapMode, FALSE);
}
```

4.6.4 Carrying Out Dialog Commands

The **DlgCommand** function processes the WM_COMMAND messages sent to the dialog input function, **ChangeOptions**. A WM_COMMAND can be the result of the user selecting or changing a control in the dialog box. **DlgCommand** must determine the source of the message and act appropriately.

DlgCommand responses are divided into three cases: a response to the user pushing the Ok button, a response to the user pushing the Cancel button, and a response to a message from a control.

```
case IDOK:
    lpLogFont->lfItalic = IsDlgButtonChecked (hDlg, IDITALIC)
                          ? 1 : 0;
    lpLogFont->lfUnderline = IsDlgButtonChecked (hDlg, IDUNDERLINE)
                             ? 1 : 0;
    lpLogFont->lfStrikeOut = IsDlgButtonChecked (hDlg, IDSTRIKEOUT)
                             ? 1 : 0;
    lpLogFont->lfWeight = GetDlgItemInt
                           (hDlg,
                           IDWEIGHT,
                           (LPBOOL)&fValOK,
                           FALSE);
lpLogFont->lfWidth = GetDlgItemInt
                          (hDlg,
                          IDWIDTH,
                          (LPBOOL)&fValOK,
                          FALSE);
lpLogFont->lfHeight = GetDlgItemInt
                           (hDlg,
                           IDHEIGHT,
                           (LPBOOL)&fValOK,
                           FALSE);
lpLogFont->lfEscapement = GetDlgItemInt
                               (hDlg,
                               IDESCAPEMENT,
                               (LPBOOL)&fValOK,
                               FALSE);
```

```
        lpLogFont->lfOrientation = GetDlgItemInt
                                        (hDlg,
                                        IDORIENTATION,
                                        (LPBOOL)&fValOK,
                                        FALSE);
        lpLogFont->lfPitchAndFamily = (BYTE)GetDlgItemInt
                                        (hDlg,
                                        IDPITCH,
                                        (LPBOOL)&fValOK,
                                        FALSE);
        lpLogFont->lfCharSet = (BYTE)GetDlgItemInt
                                          (hDlg,
                                          IDCHARSET,
                                          (LPBOOL)&fValOK,
                                          FALSE);
    nMapMode = GetDlgItemInt(hDlg, IDMAPMODE, (LPBOOL)&fValOK, FALSE);
    EndDialog(hDlg, TRUE);
    break;
```

In the first case, **DlgCommand** uses the **GetDlgItemInt** and **IsDlgButtonChecked** functions to retrieve the current values of the control windows in the dialog box and to assign the values to the fields in the LOGFONT data structure pointed to by **lpLogFont**. **GetDlgTextInt** converts a character string in an edit control into an integer number. The **IsDlgButtonChecked** function returns a Boolean value specifying whether or not the given button control is checked.

The second case corresponds to the Cancel button.

```
case IDCANCEL:
    EndDialog(hDlg, TRUE);
    break;
```

In this case, **DlgCommand** requires no action since the user has canceled his request. The function calls **EndDialog** to exit the dialog box.

The third case is any message that has been sent by a control window. Although the dialog box has 11 control windows, only three controls return messages of interest. These are check boxes representing the italics, underlined, and strikeout fonts.

```
case IDITALIC:
case IDUNDERLINE:
case IDSTRIKEOUT:
    CheckDlgButton(hDlg,  wParam,
                   (IsDlgButtonChecked(hDlg, wParam) ? FALSE : TRUE));
    break;
```

When a control message is detected, **DlgCommand** uses the **CheckDlgButton** function to place or remove a check, based on its current state. All other messages are ignored.

4.7 The Resource Script: FONTTEST.RC

The FONTTEST.RC file contains the definitions for the menus and dialog box used by FONTTEST. A dialog box definition is a list of the control windows the box will contain when created. Controls are specified by statements that define other attributes such as the location of the control in the box, its initial title or value, and its ID. A control ID is an integer number used to distinguish it from all other controls in the box.

The following is a complete listing of the FONTTEST.RC file:

```
#include "windows.h"
#include "fonttest.h"

#define TABGRP    WS_TABSTOP | WS_GROUP

fonttest  ICON     FontTest.ico

fonttest MENU
BEGIN
    POPUP "Commands"
        BEGIN
        MENUITEM "Sample^S",      IDMSAMPLE
        MENUITEM "Clear^C" ,      IDMCLEAR
        MENUITEM "Options...^O", IDMOPTIONS
        END
END

fonttest ACCELERATORS
BEGIN
    "^S", IDMSAMPLE
    "^C", IDMCLEAR
    "^O", IDMOPTIONS
END
```

```
OPTIONSBOX DIALOG 50, 25, 199, 119
STYLE WS_POPUP | WS_DLGFRAME
BEGIN
DEFPUSHBUTTON "Ok", IDOK,                    5,  2, 30, 14, TABGRP
PUSHBUTTON "Cancel", IDCANCEL,              37,  2, 30, 14, TABGRP

CHECKBOX "Italic", IDITALIC,                 5, 21, 50, 15, TABGRP
CHECKBOX "Underline", IDUNDERLINE,           5, 36, 55, 15, TABGRP
CHECKBOX "StrikeOut", IDSTRIKEOUT,           5, 51, 55, 15, TABGRP

LTEXT "Height", IDNULL,                     60, 25, 30, 12
EDITTEXT IDHEIGHT,                          85, 22, 30, 12, TABGRP
LTEXT "", IDTMHEIGHT,                      118, 25, 20, 12

LTEXT "MapMode", IDNULL,                   140, 25, 30, 12
EDITTEXT IDMAPMODE,                        172, 22, 25, 12, TABGRP

LTEXT "Width", IDNULL,                      60, 40, 30, 12
EDITTEXT IDWIDTH,                           85, 37, 30, 12, TABGRP
LTEXT "", IDTMWIDTH,                       118, 40, 20, 12

LTEXT "Weight", IDNULL,                     60, 55, 30, 12
EDITTEXT IDWEIGHT,                          85, 52, 30, 12, TABGRP
LTEXT "", IDTMWEIGHT,                      118, 55, 20, 12

LTEXT "Escapement", IDNULL,                  5, 70, 40, 12
EDITTEXT IDESCAPEMENT,                      50, 67, 20, 12, TABGRP

LTEXT "Orientation", IDNULL,                 5, 85, 45, 12
EDITTEXT IDORIENTATION,                     50, 82, 20, 12, TABGRP

LTEXT "Char Set", IDNULL,                   80, 70, 35, 12
EDITTEXT IDCHARSET,                        118, 67, 20, 12, TABGRP
LTEXT "", IDTMCHARSET,                     140, 70, 20, 12

LTEXT "Pitch", IDNULL,                      80, 85, 30, 12
EDITTEXT IDPITCH,                          118, 82, 20, 12, TABGRP
LTEXT "", IDTMPITCH,                       140, 85, 20, 12

CONTROL "", IDNULL, STATIC, SS_BLACKFRAME,  2,100, 195, 2

LTEXT "Av Char Width", IDNULL,               5,105, 60, 12
LTEXT "", IDAVCHARWIDTH,                    65,105, 12, 12

LTEXT "Max Char Width", IDNULL,             80,105, 60, 12
LTEXT "", IDMAXCHARWIDTH,                  140,105, 12, 12
END
```

```
ABOUTBOX DIALOG 22, 17, 144, 75
STYLE WS_POPUP | WS_DLGFRAME
BEGIN
    CTEXT "Microsoft Windows"                       -1, 37,  5, 68, 8
    ICON  "FontTest"                                -1,  9, 23,  0, 0
    CTEXT "FontTest application"                    -1,  0, 14,144, 8
    CTEXT "Version 1.01"                            -1, 38, 34, 64, 8
    CTEXT "Copyright © 1985, Microsoft Corp."       -1,  5, 47,132, 9
    DEFPUSHBUTTON "Ok",          IDOK,              53, 59, 32, 14, WS_GROUP
END


STRINGTABLE
BEGIN
    IDSNAME,     "Fonttest"
    IDSABOUT,    "About..."
    IDSTITLE,    "Font Tester"
    IDSALPHABET,"The quick brown fox etc.  ABCDE 0123 !@#$%(){}[];:<>?,."
END
```

The OPTIONSBOX resource is a dialog box definition. The box contains a variety of controls.

The **fonttest** resource is a menu for FONTTEST's window. It has three items: Sample, Clear, and Options. The constants IDMSAMPLE, IDMCLEAR, and IDMOPTIONS define the menu values. These values are defined in the include file FONTTEST.H.

4.8 The Module Definition File: FONTTEST.DEF

The FONTTEST.DEF file is similar to the module definition files of other applications, but includes an EXPORT definition for the dialog input function, **ChangeOptions**. Windows needs this export information to call the function with messages for the dialog box.

```
NAME     FontTest
DESCRIPTION 'Microsoft Windows Sample Font Manipulation Code'

STUB 'WINSTUB.EXE'

CODE     FIXED
DATA     MULTIPLE FIXED

HEAPSIZE     4096
STACKSIZE    4096

EXPORTS
     FonttestWndProc      @1
     ChangeOptions        @2
     About                @3
```

Chapter 5
Creating Windows Applications

5.1 Introduction 103
5.2 Windows Applications 103
5.3 The Main Function 104
5.4 Messages and Queues 105
5.4.1 Pulling Messages 106
5.4.2 Dispatching Messages 106
5.4.3 Translating Messages 106
5.4.4 Translating Accelerator Keys 107
5.4.5 Checking the Queue 108
5.4.6 Sending Vs. Posting Messages 109
5.4.7 Removing Messages 109
5.4.8 Where to Use Message Functions 109
5.4.9 Filtering Messages 110
5.4.10 Filtering Messages for a Window Class 111
5.5 Window Messages 111
5.5.1 Window State Messages 112
5.5.2 Window Input Messages 115
5.5.3 Registering Window Messages 118
5.6 An Application as a Task 119
5.6.1 Starting an Application for the First Time 119
5.6.2 Starting a New Instance 119
5.6.3 Destroying an Instance 120
5.6.4 Multiple Data Segments 120

5.6.5 Application Stack 121
5.6.6 Sharing Data 121
5.6.7 Window, Dialog Box, and Call-Back Functions 121
5.6.8 Making a Function Instance 121
5.6.9 Yielding Control 122
5.6.10 Synchronizing Tasks 122
5.6.11 Loading Modules 122
5.7 Terminating an Application 123
5.7.1 Terminating With Close 123
5.7.2 Trapping a Close Termination 123
5.7.3 Terminating With End Session 124

5.1 Introduction

This chapter explains:

Window Applications
The Main Function
Message Queues
Messages
Tasks and Application Instances

5.2 Windows Applications

A Windows application is any program that uses Windows functions and programming conventions to access the Windows system. Every Windows application has a main function and one or more window functions. The main function is the application's executive. It controls the execution of the program, determines how often input should be processed, and directs the execution of the application's window functions.

The window functions are the application's processing functions. A window function processes any input sent to it by the main function, displays and maintains the windows that belong to it, and provides interaction with other windows or applications.

Windows places a message in the application queue whenever special actions need to be taken that affect a window belonging to that application. The application's main function can then pull the message from the queue, and dispatch it to the appropriate window function.

5.3 The Main Function

All applications have a main function, called **WinMain**, that Windows calls when the user first invokes the application. **WinMain** has the form:

```
int PASCAL WinMain(hInstance, hPrevInstance, lpszCmdLine, cmdShow)
HANDLE hInstance, hPrevInstance;
LPSTR lpszCmdLine;
int cmdShow;
{
    HWND hWnd;
    MSG msg;

    if (hPrevInstance == NULL) {
        /* do first-time initialization, class registration, etc */
    } else {
        /* get data from previous instance */
    }
    /* create application window */
    hWnd = CreateWindow(...);

    /* Make window visible and paint initial client area */
    ShowWindow(hWnd, cmdShow);
    UpdateWindow(hWnd);

    while ( GetMessage((LPMSG)&msg, NULL, 0, 0) )
    {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage((LPMSG)&msg);
    }
    /* exiting from WinMain terminates the application */
    exit(msg.wParam);
}
```

The **hInstance** parameter is the instance handle of the new task. The **hPrevInstance** parameter is the module instance handle of a previous instance of the module. It is NULL if this is the first instance. The **lpszCmdLine** parameter is a long pointer to a null-terminated command line. The **cmdShow** parameter is an integer value to be passed to **ShowWindow** after you have created your application's window.

The **exit** function is the last statement of the function. This function terminates the application and passes a exit return value **msg.wParam** to Windows.

Note

In small model C-language applications, the **WinMain** function must be declared with the NEAR attribute (the default if the small model compiler option is used). The application must be linked with the SLIBW.LIB and SWLIBC.LIB libraries, but **WinMain** does not need to be exported.

In medium model C-language Windows applications, the **WinMain** function must be declared with the FAR attribute (the default if the medium model compiler option is used). The application must be linked with the MLIBW.LIB and MWLIBC.LIB libraries. Again, **WinMain** does not need to be exported.

5.4 Messages and Queues

All input for an application is presented in the form of messages. A message is a data structure that contains a message identifier and message parameters. The content of the parameters varies with the message type.

Windows generates a message on each input event, such as the user moving the mouse or pressing a keyboard key. It also generates messages when it needs an application to carry out special actions, such as to paint its client area or resize its window. Windows places most messages in application queues. The application queues are first-in/first-out queues that belong to individual applications. Windows places messages that belong to an application in the queue, and the application reads the messages using the **GetMessage** function and dispatches them to the appropriate window function using the **DispatchMessage** function.

Windows sends some messages directly to an application's window functions, and does not place them in the application queue. In general, an unqueued message is any message that affects only the window.

For example, the **CreateWindow** function directs Windows to send a WM_CREATE message to the window function of the application and to wait until the message has been processed by the window function. Windows sends this message directly to the function and does not place it in the application queue.

Although most messages are generated by Windows, applications can create their own messages and place them in the application queues of other applications by using the **PostMessage** function.

5.4.1 Pulling Messages

A main function can pull messages from the application queue by using the **GetMessage** function. This function searches the application queue for messages and returns the top message in the application queue if one exists. If the application queue is empty, **GetMessage** waits for a message to be placed in the queue. While waiting, **GetMessage** relinquishes control to Windows allowing other applications to take control and process their own messages.

5.4.2 Dispatching Messages

Once a main function has a message from a queue, it can dispatch it to a window function by using the **DispatchMessage** function. The function directs Windows to call the window function of the window associated with the message, and pass the content of the message as function arguments. The window function can then process the message and carry out any requested changes to the window. When the window function returns, Windows returns control to the main function. The main function can then pull the next message from the queue.

5.4.3 Translating Messages

Windows generates a virtual key message each time the user presses a key on the keyboard. The virtual key message contains a virtual key code that defines which key was pressed, but does not define the character value of that key. To retrieve the character value, the main function must translate the virtual key message using the **TranslateMessage** function. This function puts another message with an appropriate character value in the application queue. The message can then be dispatched to a window function.

The **TranslateMessage** function can translate virtual key codes into ANSI character values or OEM-specific character values. The style of the window associated with the message defines what kind of translation is to take place. This means **TranslateMessage** can convert a virtual key code into a single character value or into special-purpose or multiple character values, such as foreign language characters. If **TranslateMessage** generates multiple characters, it places multiple messages in the application queue.

In general, a main function should use **TranslateMessage** to translate every message, not just virtual key messages. Although **TranslateMessage** has no effect on other types of messages, it guarantees that any keyboard input is translated correctly.

An application can send virtual key messages directly to an application's window function, if desired, by using the **SendMessage** function. If **SendMessage** is used, the message must be translated in the window function.

The following program fragment illustrates the typical loop used by a main function to pull messages from the queues and dispatch them to window functions:

```
int PASCAL WinMain( hInstance, hPrevInstance, lpCmdLine, ShowCmd )
HANDLE hInstance;
HANDLE hPrevInstance;
LPSTR lpCmdLine;
WORD ShowCmd;
{
        .
        .
        .
    while ( GetMessage((LPMSG)&msg, NULL, 0, 0) )
    {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage((LPMSG)&msg);
    }
    exit(msg.wParam);
}
```

5.4.4 Translating Accelerator Keys

Applications that use accelerator keys must load an accelerator table from the resource file using the **LoadAccelerators** function, then translate keyboard messages into accelerator key messages using the **TranslateAccelerator** function. The main loop for applications that use accelerators should have the following form:

```
while (GetMessage((LPMSG)&msg, (HWND)NULL, 0, 0))
    {
    if (TranslateAccelerator(hWindow, hAccel, (LPMSG)&msg) == 0)
        {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage(LPMSG)&msg);
        }
    }
exit(msg.wParam);
```

The **TranslateAccelerator** call must appear before the standard **TranslateMessage** and **DispatchMessage** functions. Furthermore, since **TranslateAccelerator** automatically dispatches the accelerator message to the appropriate window function, the **TranslateMessage** and **DispatchMessage** functions should not be called if **TranslateAccelerator** returns a non-zero value.

5.4.5 Checking the Queue

An application can use the **PeekMessage** function if it needs to check the queues for messages but does not want to pull the message from the queue. The function returns a non-zero value if a message is in the queue and lets the application retrieve the message and process it without going through the application's main loop.

PeekMessage is typically used to periodically check for messages when the application is carrying out a lengthy operation, such as processing input and output. For example, the function can be used to check for messages that abort the operation. In the following program fragment, the application peeks for a message before continuing with a input and output operation:

```
.
.
.

/* Halt I/O operation and check queue for message */
if (PeekMessage((LPMSG)msg, (HWND)NULL, 0, 0) ) {
        if ((msg.type==WM_COMMAND)&&(msg.wParam==IDABORT))
                AbortIO();
}
/* Continue I/O operation */
.
.
.
```

PeekMessage, like **GetMessage**, waits for messages to be placed in a queue. This lets other applications take control and process their own messages if the application queue is empty. Windows returns control to the application when a message is placed in the queue.

5.4.6 Sending Vs. Posting Messages

The **SendMessage** and **PostMessage** functions let applications to pass messages to its windows or to the windows of other applications.

The **SendMessage** function directs Windows to *send* a message directly to the given window function, bypassing the application queue. Windows does not return control to the calling application until the window function receiving the message processes the message. **SendMessage** is typically used to send messages to child windows.

The **PostMessage** function directs Windows to *post* the message by placing it in the application queue. Control returns immediately to the calling application and any action to be carried out as a result of the message does not occur until the message is read from the queue.

5.4.7 Removing Messages

You can remove messages from the application queue by using the **PeekMessage** function. For example, the following removes keyboard messages from the queue:

```
while (PeekMessage((LPMSG)&msg, WM_KEYFIRST, WM_KEYLAST, TRUE))
```

The WM_KEYFIRST and WM_KEYLAST constants can be used to flush all keyboard input, or the WM_MOUSEFIRST and WM_MOUSELAST constants can be used to flush all mouse input.

Not all messages can be removed in this way. In particular, Windows continues to send WM_PAINT messages to an application until the corresponding window is painted or validated.

5.4.8 Where to Use Message Functions

In most applications, only the main function uses the message functions to pull messages from the queues. Any window function can also call the functions if it has a special need for input, such as when it is tracking the mouse, or reading keystrokes in advance. Windows places no restrictions on when or where a message function can be called.

5.4.9 Filtering Messages

Any application can filter messages that are otherwise processed by the main program loop in Windows. To filter messages, install a message filter function that Windows can call any time the application executes a **GetMessage** function. The filter function is called before any processing. Once the filter function is called, it can process the message and return TRUE, or do nothing and return FALSE. If the filter function returns TRUE, Windows does no further processing of the message. If the function returns FALSE, Windows processes the message. The filter function can also modify the message and return FALSE, to let Windows process a modified message.

Windows passes the filter function a code indicating the reason for each call. One filter is installed for each application. This means your filter sees messages that are related to your application only.

You can install the filter function by using the **SetWindowsHook** function. The call has the form:

```
SetWindowsHook(WH_MSGFILTER, fnFilterProc);
```

where *fnFilterProc* is a long pointer to the filter function. The filter function must have the form:

```
BOOL FAR PASCAL FilterProc()
```

The **MakeProcInstance** function must be used to create the correct instance address for the filter function.

If your application uses its own stack (that is, has a STACKSIZE line in the module definition file), *fnFilterProc* should be declared as follows:

```
BOOL FAR PASCAL fnFilterProc(dummy, pMsg, code)
WORD dummy;
PMSG pMsg;
int code;
```

Otherwise, *fnFilterProc* should be declared as follows:

```
BOOL FAR PASCAL fnFilterProc(lpMsg, code)
LPMSG lpMsg;
int code;
```

where *pMsg* or *lpMsg* is a pointer to a **MSG** data structure, and *code* is a value indicating the context in which the filter function is being called. It can be any one of the following:

MSGF_DIALOGBOX	Inside of DialogBox
MSGF_MESSAGEBOX	Inside of MessageBox
MSGF_SIZE	Sizing windows
MSGF_MOVE	Moving window
MSGF_MENU	Menu tracking

5.4.10 Filtering Messages for a Window Class

The **SetClassLong** function is useful for such tasks as filtering messages of a given class. You can use the GCL_WNDPROC value to set the class window function to the address of a filter function. **SetClassLong** returns the address of the default window function, which the application should store and use to process messages it does not want to filter.

The following example illustrates how to filter messages using **SetClassLong**:

```
FARPROC lpOldProc

lpOldProc = SetClassLong(GCL_WNDPROC, (FARPROC)NewProc);
```

The application should restore the original window function address before terminating.

To do the same for a particular window instead of the entire class, use **SetWindowLong**.

5.5 Window Messages

Windows communicates with applications through formatted window messages. The messages are passed (i.e., either sent or posted) to an application's window function to let the function process the messages as desired. Although an application's main function may read and dispatch window messages, in most cases it is only the window function that processes them.

A message consists of three parts: a message type, a word parameter, and a long parameter. Message types are identified by predefined message names. The names begin with "WM_" and suggest the meaning or origin of the message. The word and long parameters, named *wParam* and *lParam*, contain values that depend on the message type. If a given message does not use the parameter, it is set to 0.

The *lParam* parameter often contains more than one piece of information. For example, the high-order word may contain a handle to a window, and the low-order word an integer value. The HIWORD and LOWORD utility macros can be used to extract the high- and low-order words of *lParam*. The HIBYTE and LOBYTE utility macros can also be used with HIWORD and LOWORD to access any of the bytes.

The following sections describe the window messages processed by most applications. For a list of all window messages and their meanings, see the *Microsoft Windows Reference Manual.*

5.5.1 Window State Messages

Windows passes an application a state message in response to actions that directly affect a window, such as creating, destroying, activating, moving, sizing, enabling, or closing a window.

Creating a Window

The first message that most window functions process is the window creation message, WM_CREATE. Windows sends this message to the window function when the window is being created, such as during the execution of the **CreateWindow** function. This message informs the application that it can now perform any initialization desired, such as allocate memory, and prepare data files. The *wParam* parameter is not used, but the *lParam* parameter contains a long pointer to a **CREATESTRUCT** data structure, whose fields correspond to the parameters passed to **CreateWindow**. The WM_CREATE message is sent directly to the window function, bypassing the application queue. This means an application can create a window and process the WM_CREATE message before it enters the main program loop.

Destroying a Window

The last message a window function usually processes is the destroy message, WM_DESTROY. This message occurs whenever the application calls the **DestroyWindow** function or lets the **DefWindowProc** function process a WM_CLOSE message. The *wParam* and *lParam* parameters are not used. When an application receives a WM_DESTROY message, it should call the **PostQuitMessage** function. This function copies a WM_QUIT message to the application's queue. The WM_QUIT message is a signal to the **GetMessage** or **PeekMessage** function in the main program loop to terminate the loop and exit the application.

Activating a Window

A window function receives a WM_ACTIVATE message when the user activates the corresponding window or changes the activation to another window. The user changes the activation by pressing the ALT-TAB key or by clicking in a window. Windows passes the WM_ACTIVATE message to both windows involved, the one becoming active and the one becoming inactive. The *wParam* parameter is TRUE for the window becoming activate; FALSE for the one becoming inactive. The low-order word of *lParam* contains the window handle of the opposite window. The high-order word contains TRUE if the window is iconic. If a window is not iconic when it becomes active, it should change the input focus when it receives the WM_ACTIVATE message such as in the following example:

```
switch (message)
  ...
  case WM_ACTIVATE:
    if (wParam && !HIWORD(lParam))
      SetFocus(hWnd);
    break;
  ....
```

This lets the application process keyboard input that is intended for the active window. Calling **SetFocus** causes a WM_SETFOCUS message to be sent to the window getting the focus and a WM_KILLFOCUS message to the window losing it. Window functions usually do not process these messages unless they wish to do special tasks, such as displaying a flashing caret or saving the current selection.

Moving a Window

A window function receives a WM_MOVE message when the user moves or resizes the window. The *wParam* parameter is not used, but the *lParam* parameter contains the new location of the upper left corner of the window's client area (with the x coordinate in the low-order word and the y coordinate in the high-order word). Window functions that need to know this location usually save the new x and y coordinates so they can use it later.

Sizing a Window

A window function receives a WM_SIZE message when the user resizes or zooms the window or makes it iconic. The *wParam* parameter contains a value defining the type of resizing requested: SIZEICONIC, SIZEFULLSCREEN, or SIZENORMAL. The *lParam* parameter contains

the new width and height of the window's client area (with the width in the low word and height in the high word). Window functions that adjust their displays to make best use of the size of the window usually save the new width and height so they can use it when they paint their windows later.

Note

If *lParam* is 0 and *wParam* is SIZENORMAL, the WM_SIZE message should be ignored. This is an Windows internal message.

Painting a Window

A window function receives a WM_PAINT message when a request to paint the contents of the window has been made. This can occur after the window has been created, moved, sized, or scrolled. An application can also make a specific request to be painted. The *wParam* parameter is not used, but the *lParam* parameter contains a long pointer to a PAINTSTRUCT structure that contains information about the area to paint. Any application that displays text or graphics in its windows needs to process the WM_PAINT message. The message tells the application where to paint; it is the application's responsibility to determine what to paint.

Erasing the Background

A window function receives a WM_ERASEBKGND message when a request to erase the current window background has been made. This usually occurs before a WM_PAINT message, but always arrives immediately after the request has been made, unlike a WM_PAINT message which can be delayed until other messages have been processed. The *wParam* parameter contains a display context that can be used to paint the background. The *lParam* parameter is not used. Many window functions ignore this message, passing it to the **DefWindowProc** function, which erases the background using a default brush. But applications that wish to change the color or texture of the background need to process this message by choosing an appropriate brush and painting.

Preparing for the End of the Session

A window function receives a WM_QUERYENDSESSION message when the user chooses the End Session command in the MS-DOS Executive menu. The message is a request by Windows for the application to state whether or not it can terminate. The application can return TRUE if it is ready to terminate, or FALSE if it is not. Windows will not end the session if any application replies FALSE to the WM_QUERYENDSESSION message. The *wParam* and *lParam* parameters are not used. Most applications prompt the user when they receive a WM_QUERYENDSESSION message. The prompt typically asks if current files should be saved or other last actions should be carried out. Once the user replies, the application carries out the actions and returns TRUE.

If all applications reply TRUE to the WM_QUERYENDSESSION message, Windows sends a WM_ENDSESSION message to each. Most applications pass this message to the **DefWindowProc** function which automatically uses calls the **DestroyWindow** function to destroy the window and terminate the application.

5.5.2 Window Input Messages

Windows passes input messages to an application whenever an input event, such as a keystroke or mouse click, occurs within the application's window. The messages define the message type and contain additional information about the input itself.

Mouse Movement and Buttons

A mouse input message occurs whenever a mouse event has taken place in the application's window. There are the following mouse input messages:

WM_MOUSEMOVE	Mouse movement
WM_LBUTTONDOWN	Left button down
WM_LBUTTONUP	Left button down
WM_LBUTTONDBLCLK	Left button double click
WM_RBUTTONDOWN	Right button down
WM_RBUTTONUP	Right button up
WM_RBUTTONDBLCLK	Right button double click

The *wParam* parameter contains a value indicating whether or not various virtual keys are down. It can be any combination of the following:

MK_LBUTTON	Set if left button is down
MK_RBUTTON	Set if right button is down
MK_SHIFT	Set if shift key is down
MK_CONTROL	Set if control key is down

The low-order word of *lParam* contains the x coordinate of the mouse cursor, and the high-order word of *lParam* contains the y coordinate. These coordinates are always relative to the top left corner of the screen.

To receive double click messages, the window class must be registered with the CS_DBLCLKS style. Windows generates a double click message whenever the user presses, releases, and then presses a mouse button again within a certain number of milliseconds. This actually generates four messages: a down click message, an up click message, the double click message, and another up click message.

Keyboard Input

Windows generates virtual key messages whenever the user presses or releases a key. The virtual messages, WM_KEYDOWN and WM_KEYUP, contain virtual key values that do not necessarily specify the ANSI value of the key pressed. An application's main program loop usually creates ANSI key messages by translating the virtual key messages into WM_CHAR or WM_DEADCHAR messages with the **TranslateMessage** function. After translation, an application's window function receives both the WM_KEYDOWN and WM_KEYUP messages and the translated WM_CHAR and WM_DEADCHAR messages.

A window function receives a WM_KEYDOWN and WM_KEYUP message whenever the user presses or releases a key on the system keyboard and the window has the current input focus. The *wParam* parameter contains a virtual key value that uniquely identifies the key but does not necessarily give the key's ANSI value. Windows has several predefined virtual keynames. They all begin with "VK_" and are defined in the *Microsoft Windows Reference Manual.* The low-order word of the *lParam* parameter contains a repeat count, and the high-order bit of *lParam* specifies the transition state of the key (1 for a WM_KEYUP message, and 0 for a WM_KEYDOWN message). The repeat count is the number of times the keystroke is repeated as a result of the user pressing and holding the key.

The WM_CHAR message, created by the **TranslateMessage** function, contains the ANSI value of a virtual key message. The *wParam* parameter contains the ANSI value of the input character, the low-order word of *lParam* contains a repeat count.

A WM_DEADCHAR message represents a dead key. A dead key is any key, like the umlaut (double dot) character, that is usually combined with another key to form a complete character. To enter umlaut-O, for example, the user types the umlaut key followed by the O key. When umlaut-O is pressed, Windows passes first a WM_DEADCHAR message containing the key's character code, then a WM_CHAR message containing the umlaut-O character code. This means that applications interested in complete characters only can ignore the WM_DEADCHAR message and wait for the WM_CHAR message. The WM_DEADCHAR message is typically used by applications that want to give the user feedback about each key pressed. The *wParam* parameter contains the dead key character value, the low-order word of *lParam* contains a repeat count.

You can determine the state of any key on the keyboard at any time with the **GetKeyState** function.

Timer Input

A window function receives a WM_TIMER message when a timer set with the **SetTimer** function has elapsed. The *wParam* parameter contains the timer ID. The timer ID is an integer value that identifies a timer event. The timer message is typically used by applications that wish to be periodically awakened to carry out certain tasks.

Menu and Control Input

A window function receives a WM_COMMAND message as input from a menu or a control. The *wParam* parameter contains the menu item or control ID. The content of the *lParam* parameter depends on the message source. If the message is from a menu, *lParam* contains 0. If it is from a control, the low-order word of *lParam* contains the window handle of the control; the high-order word contains a control-specific notification code.

Scroll Bar Input

A window function receives scroll bar messages, WM_VSCROLL and WM_HSCROLL, whenever the user clicks on the vertical or horizontal scroll bar of the window. The *wParam* parameter contains a scroll bar code. This code defines the user's scrolling request. It can be any one of the following:

SB_LINEUP	Scroll one line up or left
SB_LINEDOWN	Scroll one line down or right
SB_PAGEUP	Scroll one page up or left
SB_PAGEDOWN	Scroll one page down or right
SB_THUMBPOSITION	Scroll to absolute position
SB_THUMBTRACK	Thumb is being tracked at specified position

The high-order word of *lParam* is the window handle of the scroll bar.

Windows sends SB_THUMBPOSITION whenever the user drags the thumb to a new position. The low-order word of *lParam* contains the new position value.

Windows sends SB_THUMBTRACK while the user is in the process of dragging the thumb. The low-order word of *lParam* contains the current position of the thumb. This message is typically used by applications that give some feedback while the thumb is being dragged.

If an application scrolls the document in the window, it must also reset the position of the thumb by using the **SetScrollPos** function.

5.5.3 Registering Window Messages

Applications can use their own window messages to communicate within the application or with other applications. Windows recognizes three types of messages:

Reserved Messages	These are Window messages, such as WM_PAINT and WM_COMMAND. Windows reserves all message numbers from 0 to WM_USER for Windows messages. The messages must not be redefined.
Local Messages	These messages can be used by an application for sending messages within the application. For example, an application may wish to send private messages to its own controls or child windows. Local messages should not be sent to windows that do not know about the specific message meaning. Different applications may have local messages that have the same value but different meaning.

Registered Messages

These messages are defined and registered by an application for use with any other application that registers the same message using the same name. An application registers a message by using the **RegisterWindowMessage** function. Once one application registers the message, all subsequent applications that register the message receive the same message value.

5.6 An Application as a Task

A task is any application that Windows regards as a separately running program. All Window applications are run as tasks. Each invocation of an application is a separate task.

Windows lets several tasks run at the same time by carefully sharing the resources of the system's CPU between the tasks. An application continues to run until it yields control to another application, or must wait for a message to be placed in its queue. In both cases, Windows passes control to the application having the highest task priority.

5.6.1 Starting an Application for the First Time

When a user starts an application for the first time, Windows creates a task by loading the application from disk into memory, preparing the application's data area, then calling the application's main function. The main function can then define and register the window class (or classes) to be used by the application, create a window for the application and perform any required initializations, such as opening disk files or allocating blocks of memory. The application, its data area, and its windows are called the first instance of the application.

5.6.2 Starting a New Instance

Windows starts a new instance of an application when the user reinvokes the application while the initial invocation is still running. The new instance is a new version of the application that shares the program code of the first instance but has a completely new data area. Windows makes a

new call to the application's main function whenever it starts the new instance. This new call does not affect any previous calls to the function; all instances execute independently.

When Windows calls the main function for a new instance, it passes the handle of the previous instance. This handle lets the new instance use the **GetDataInstance** function to access any resources that the previous instance has already prepared, such as handles to drawing objects, window classes, and other useful items.

Although instances share the same application code, they do not share data areas. If two instances of the same application need to exchange data, they use the same data interchange technique as two unrelated applications.

5.6.3 Destroying an Instance

When an application's main function terminates, Windows immediately frees all data and resources belonging to the instance. This destroys the instance and stops that version of the application. All other instances of the application continue unchanged.

Before destroying an instance, the window function should use the **PostQuitMessage** function to inform Windows that execution is about to terminate. Windows will then place a WM_QUIT message in the application queue. **GetMessage** returns FALSE when it returns WM_QUIT. In this case, *wParam* contains the value passed to **PostQuitMessage**. See the section, "Terminating an Application," for details.

5.6.4 Multiple Data Segments

Windows never loads an application's code more than once, no matter how many instances are created. To ensure proper execution when a Windows application is invoked more than once, the application must have multiple data segments. This is defined in the application's module definition file.

Whenever there are multiple instances of an application, each function in each instance has an unique entry point at a prolog that loads the DS and SS registers with correct data segment values for the instance then jumps to the shared code of the rest of the function.

5.6.5 Application Stack

Every application has a stack that is used for temporary storage of values passed as arguments to functions. When Windows loads the application, it places the stack in the application's data segment if the module definition file defines a stack size. This means that all function arguments placed in the stack are local to the application. The application can pass arguments to its functions without using long pointers.

5.6.6 Sharing Data

Applications can share data by using the clipboard to copy data to and from other applications. Applications must not attempt to share data by passing static data addresses, local data handles, or global data handles to one another.

Applications can share resources, such as bitmaps or fonts, if these resources are loaded as sharable libraries.

5.6.7 Window, Dialog Box, and Call-Back Functions

All Window, dialog box, and call back functions used in a C-language application must be defined with the FAR and PASCAL attributes. This lets Windows, which uses Pascal language calling conventions and far (32-bit) calls, access the functions correctly.

Applications that have moveable code segments must use the **MakeProcInstance** function to create a function instance address for each diaslog box and call-back function. This lets Windows access the correct function address even though the code segment may have moved. Applications with fixed code segments do not need to make a function instance address.

5.6.8 Making a Function Instance

The **MakeProcInstance** function creates an entry point address for the given function in the current instance. The function address returned by **MakeProcInstance** actually points to a prolog that loads the correct data segment value for the current instance before jumping to the actual function code. This ensures that the current instance's data is used each time the function is called from within that instance.

MakeProcInstance should be used in any application that can have multiple instances running at the same time. The function is typically used to create instance-specific function addresses that can be passed to Windows through the **DialogBox** function and other functions that take a call-back function address as a parameter.

5.6.9 Yielding Control

An application automatically yields control to other applications whenever it calls the **GetMessage** and **PeekMessage** functions. In general, if a Windows application is using a window and it is periodically reading messages from the application queue, the application should use **GetMessage** and **PeekMessage** to yield control instead of the **Yield** function.

5.6.10 Synchronizing Tasks

The data interchange function of the Windows clipboard can be used to synchronize the operations of two independent tasks. Tasks can design a communication scheme in which one task, a slave, waits for messages passed to it from another task, the master, through the clipboard. While waiting, the slave can still process other Windows input messages.

5.6.11 Loading Modules

Applications can load other executable modules by using the MS-DOS EXEC system call. When an application uses the EXEC call to load and execute a module, it should pass as a call argument a long pointer to an FCB1 and FCB2.

The value of FCB1 depends on the type of module to be loaded. If the module is a non-Windows executable, FCB1 should point to a one word byte count which is followed by the number of bytes specified in the count. These bytes are passed to the application as invocation parameters. For Windows applications, the byte count should be 2, followed by a one word SHOW_ WINDOW parameter (0, 1, or 3).

If the MS-DOS call is successful when loading a non-Windows executable, it returns the module handle in the AX register.

5.7 Terminating an Application

An application is expected to terminate when the user selects the Close command in the application's system menu, or selects the End Session command in the MS-DOS Executive system menu. The different methods require different responses from the application. The following sections explain these responses and what steps are needed to terminate the application.

5.7.1 Terminating With Close

When the user invokes the Close command on the system menu, Windows sends a WM_SYSCOMMAND message with *wParam* set to SC_CLOSE. An application that wants to terminate should pass this message to the **DefWindowProc** function. It will then receive a WM_CLOSE message, which should also be passed to **DefWindowProc**. The WM_CLOSE message directs **DefWindowProc** to call the **DestroyWindow** function which does some housekeeping and sends a WM_DESTROY message to the application.

The application should trap WM_DESTROY when sent to its principal window and call the **PostQuitMessage** function. This function posts a WM_QUIT message, which when read from the application queue with the **GetMessage** function, causes the function to return FALSE, the signal to exit the main program loop and call the **exit** function. **PostQuitMessage** should not be called by child windows or popups when they receive WM_DESTROY.

5.7.2 Trapping a Close Termination

If an application wants to control termination as a result of the Close command, it should trap and process the WM_CLOSE message. Applications that trap WM_CLOSE typically prompt the user with a dialog box to confirm closing the window or saving files. If the user indicates that termination should not continue, the application can abort the termination by returning from the window function without taking any other action.

If the user indicates that the termination should proceed, the application should save files, free resources, and call the **DestroyWindow** function. **DestroyWindow** sends a WM_DESTROY message to all children and finally the parent. The parent should invoke **PostQuitMessage** when it receives WM_DESTROY.

5.7.3 Terminating With End Session

When the user selects the End Session command of the MS-DOS Executive, Windows asks each application if it is willing to terminate. If all respond TRUE, Windows terminates, the application does not have to. If any application responds FALSE, Windows does not terminate and no applications terminate.

To signal an end session request, Windows sends each application a WM_QUERYENDSESSION message. If an application is willing to terminate, it returns TRUE. Otherwise, it returns FALSE. If an application does not catch WM_QUERYENDSESSION, **DefWindowProc** returns TRUE.

If all applications respond TRUE, all applications are sent a WM_ENDSESSION message with *wParam* set to TRUE. If any application returns FALSE, no more applications receive a query and the applications that returned TRUE are sent a WM_ENDSESSION message with *wParam* set to FALSE.

When an application receives the WM_QUERYENDSESSION message, it can ask the user to confirm termination by posting a dialog box. It should save files, write buffers to disk, and carry out other tasks that prepare for termination but can be undone if the end of session is aborted.

When the application receives the WM_ENDSESSION message with *wParam* set to TRUE, it should free public and private resources, free scratch files, reset baud rates on I/O ports, and free memory, screen space, and brushes. It should then return without destroying the window or calling **PostQuitMessage**. The application finishes processing the WM_ENDSESSION message and returns to Windows, it will never be called again.

If an application gets a WM_ENDSESSION with *wParam* set to FALSE, another application has aborted the termination. The current application should continue running.

Chapter 6
Creating and Using Windows

6.1 Introduction 127
6.2 The Standard Window 127
6.3 Window Classes 128
6.3.1 Window Class Attributes 128
6.3.2 Registering a Window Class 129
6.4 Tiled Windows 130
6.5 Creating a Tiled Window 131
6.6 Child Windows 131
6.6.1 A Child's Parent and Class 132
6.6.2 Creating a Child Window 132
6.6.3 Receiving Input and Messages 133
6.6.4 Moving and Sizing Child Windows 133
6.6.5 The Child-Parent Relationship 133
6.6.6 Overlapping Child Windows 134
6.6.7 Painting with Visible Child Windows 134
6.7 Popup Windows 134
6.7.1 A Popup's Parent and Class 134
6.7.2 Creating a Popup Window 135
6.7.3 Receiving Input and Messages 135
6.7.4 The Popup-Parent Relationship 136
6.7.5 Overlapping Popups 136
6.7.6 Painting Under a Popup Window 136
6.7.7 The Message Box 137

6.8 Window Styles 137
6.9 Painting Windows 138
6.9.1 Receiving a WM_PAINT Message 139
6.9.2 A Window's Mapping Mode 140
6.9.3 The Clipping Region 140
6.9.4 Asynchronous Paint Messages 140
6.9.5 Sizing Requests 140
6.9.6 Zooming a Window 141
6.9.7 Painting the Window Background 142
6.9.8 Using System Colors for Window Background 142
6.9.9 The Display Context 142
6.10 Window Subclassing 144

6.1 Introduction

This chapter explains:

Window Classes
Tiled Windows
Child Windows
Popup Windows
Window Styles
Window Painting
Window Subclassing

6.2 The Standard Window

A window is the standard input and output tool of a Windows application program. Every Windows application uses a window to display text and graphics on the system screen and to process input from the system keyboard, mouse, and timer. In other words, a window provides the same input and output capabilities as a terminal, but it does so without requiring complete control of the system's actual hardware resources.

A window's screen image consists of a client area, a caption bar, special control icons, and optional menus and scroll bars. A window's exact appearance and behavior depend on the window class to which the window belongs. Any number of windows of the same class can be on the screen at the same time. For example, if two applications have the same window class or a given application is invoked twice, identical windows are displayed on the screen. But each window is unique, and any action applied to a given window affects only the application to which it belongs.

A window has an associated icon or symbol that Windows displays on the system screen when the window is iconic. An application can make the window iconic to save screen space when several windows occupy the system screen at the same time. Windows creates the open window whenever the corresponding icon is open, and replaces the open window with the icon when the window is made iconic. When a window is destroyed, Windows removes the window from the screen.

6.3 Window Classes

A window class is a set of attributes that define how a window looks and behaves when running under Windows. A window class defines the shape of the window icon and cursor, the window's module instance, and the default menu. A window class also defines the window function used to process input for the window. Before an application can create and use a window, it must define and register a window class for the window.

6.3.1 Window Class Attributes

The window class attributes are:

Attribute	Purpose
Class Style	A window class can have a variety of styles. The styles define how to update the window after moving or resizing, how to process double clicks of the mouse, and how to allocate space for the display context.
Window Function	A window class must have a specific function that processes messages sent to its windows. The function can call other function within the application to carry out its work.
Class Module	A window class must belong to a specific executable module. Failure to define a module causes an error.
Class Cursor	A window class needs a class cursor if the application does not intend to draw a cursor of its own.
Class Icon	A window class needs an icon to represent the window when iconic.
Background Brush	A window class needs a background brush to paint over images already on the screen when the window is opened. If NULL, the class is responsible for painting the background.

Menu Name	A window class can specify a default menu for the window so the application does not have to create its own. The menu name defines the name of the menu resource in the application's executable file.
Class Name	The window class needs a unique name to distinguish it from other registered classes.

Any number of window classes can be registered. Once a class has been registered, Windows permits the application to create any number of windows belonging to that class. If two classes having the same name are registered, the most recently registered class is recognized, and the other is destroyed.

6.3.2 Registering a Window Class

You can register a window class by passing a data structure containing the class attributes to the **RegisterWindow** function. When Windows registers a window class, it copies the attributes into its own memory area. Any application that later refers to the window class by name causes Windows to use the internally stored attributes, so the application that originally registered the class does not need to keep the structure available.

In the following example, an application registers a window class named "MyClass." The attributes are passed in a data structure that has been allocated in the application's local heap. The structure is destroyed after the class is registered.

```
PWNDCLASS    pMyClass;

pMyClass = (PWNDCLASS)LocalAlloc( LPTR, sizeof(WNDCLASS) );

pMyClass->hCursor      = LoadCursor( hInstance, (LPSTR)"MyIcon" );
pMyClass->hIcon        = LoadIcon( hInstance, (LPSTR)"MyCursor" );
pMyClass->lpszMenuName   = (HMENU) NULL;
pMyClass->lpszClassName  = (LPSTR)"MyClass";
pMyClass->hbrBackground  = GetStockObject( WHITE_BRUSH );
pMyClass->hInstance      = hInstance;
pMyClass->style          = CS_HREDRAW | CS_VREDRAW;
pMyClass->lpfnWndProc = MyWndProc;

RegisterClass( (LPWNDCLASS)pMyClass );

LocalFree( (HANDLE)pMyClass );
```

6.4 Tiled Windows

A tiled window has four parts: the caption bar, the optional menu, the client area, and the optional scroll bars.

The caption bar, at the top of the window, provides space for the name of the window. An application defines this name when it creates the window. It can also change this name at any time using the **SetWindowText** function. The special icons at each end of the caption bar are the system menu box and the size box. These icons let a user move, size, close, and make the window iconic.

The application menu, an optional rectangle just below the caption bar, provides space for an application's menu of commands. An application defines the contents of the menu when it creates the window, or when it registers the window's window class. The application defines the command names or icons used in the menu and the results returned to the application when the user selects a command. An application can read the results from the application queue and pass them to the window. An application can change the contents of the menu at any time, can disable any command in the menu, and even define popup menus to appear whenever the user selects a command.

The horizontal and vertical scroll bars, optional bars on the right and lower sides of a window, provide a graphic image through which a user can request scrolling. An application can have both bars, if desired, or just one. It can choose when it creates the window. If it chooses a bar, it must be prepared to receive WM_HSCROLL and WM_VSCROLL messages.

The client area in the center of the window provides space for all the output generated by the application. The application uses the GDI functions to draw images and write text in this area. Windows will provide a background when needed if the application's window class has a background brush. Whenever Windows rearranges or resizes a window, it sends a WM_PAINT message to the window to let it redraw the area if necessary. Windows passes a description of the client area, called a display context, that the application can use to redraw the window. This means an application is never required to determine where the window is before proceeding with redrawing.

6.5 Creating a Tiled Window

You can create a tiled window by supplying the WS_TILEDWINDOW window style in a call to the **CreateWindow** function. For example, the call

```
hWindow = CreateWindow (
                    (LPSTR)"MyClass",  /* Window Class */
                    (LPSTR)"My Window",          /* Caption */
                    WS_TILEDWINDOW,              /* Window Style */
                    0,                           /* Not used */
                    0,                           /* Not used */
                    0,                           /* Not used */
                    0,                           /* Not used */
                    (HWND)NULL,                  /* Window handle */
                    (HMENU)NULL,                 /* Menu handle */
                    hInstance,                   /* Module instance */
                    (LPSTR)NULL                  /* Additional data */
                    );
```

creates a tiled window that belongs to the previously registered window class "MyClass." The tiled window is made visible using the **ShowWindow** function.

In this example, the tiled window uses the default menu, if any, defined by the window class. The window could have been given any existing menu by supplying the menu's handle in the **CreateWindow** call.

6.6 Child Windows

A child window is an application-defined and maintained window that is displayed within the client area of another window. Child windows are typically used to divide the client area into different functional areas. For example, the MS-DOS Executive window has several child windows. The disk drive icons, the current directory name, and the current directory listing are separate child windows. In general, it is easier to create and combine several child windows than it is to design a tiled window that performs all these tasks.

6.6.1 A Child's Parent and Class

Every child window belongs to some other window. The other window can be a tiled window, a popup, or even another child. The window that owns the child window is called the parent window. The parent window relinquishes a portion of its client area to the child, and the child window receives all input from this area.

A child window, like any tiled window, must belong to a window class. The window class defines the default attributes of the window, including its window function. The child's window class does not have to be same as the parent's window class. This means that the child and parent windows can look and act quite differently.

6.6.2 Creating a Child Window

You can create a child window by supplying the WS_CHILD window style in a call to the **CreateWindow** function. For example, the call

```
hChild = CreateWindow (
                    (LPSTR)"MyClass",  /* Window Class */
                    (LPSTR)"Child 1",  /* Caption */
                    WS_CHILD,          /* Window Style */
                    10, 10,            /* Origin in parent window */
                    200, 100,                  /* Width and height */
                    hTiledWindow,              /* Parent's handle */
                    1,                         /* Child ID */
                    hInstance,                 /* Module instance */
                    (LPSTR)NULL                /* Additional data */
                    );
```

creates a child window that belongs to the previously registered window class "MyClass." The child's parent is given by **hTiledWindow**. The child window, when made visible using the **ShowWindow** function, is displayed in the parent's client area. The upper left corner of the child window is set at the coordinate (10, 10) relative to the origin of this client area. The window is 200 units wide by 100 units high. The units depend on the current mapping mode of the parent when the **CreateWindow** function is called. Usually, units are in pixels.

In this example, the child window sets the child ID value to 1. Each child window should have a unique ID. The child should use this ID in any messages it sends to the parent window. When a parent has several child windows, this is the way the parent can identify the child sending the message.

6.6.3 Receiving Input and Messages

A child window is an independent window that receives its own input and messages. This means that input intended for a child goes directly to the child window function, and is not passed through the parent window function.

When the **EnableWindow** function is used to disable a child window, Windows passes any input that would have gone to the child to the parent instead. This gives the parent an opportunity to examine the input and enable the child if necessary.

6.6.4 Moving and Sizing Child Windows

If a child window has a caption, the mouse may be used to move the window. If the child window has a size box either on a caption or somewhere in the window, the mouse may be used to change the size.

Child window captions do not have a system menu box. Instead, they have an optional close box. This box is on the caption bar in the same place as the system menu box. Clicking this symbol with the mouse closes the child window. To have a close box, the child window must have the WS_CAPTION and WS_SYSMENU styles.

6.6.5 The Child-Parent Relationship

Since every child window belongs to a parent window, actions that affect the parent can also affect the child. The following is a list of actions that can affect child windows:

1. Hiding the parent window or making it iconic also hides the child window. A child window can be visible only when the parent is visible.
2. Destroying a parent with the **DestroyWindow** function also destroys the child. Windows ends a WM_DESTROY message to the child window before sending one to the parent.
3. Sizing a parent does not cause child windows to be resized. Windows sends WM_SIZE messages to tiled windows only. Child windows never receive this message. It is up to the parent to resize its own child windows.

6.6.6 Overlapping Child Windows

You can create child windows that can overlap. Child windows that can overlap can be moved together to form a pile of child windows, in which the top window partially or completely obscures the contents of the windows below it. The order of the windows in the pile can be changed by using the **BringWindowToTop** function.

6.6.7 Painting with Visible Child Windows

When overlapping child windows are specified for a parent window, Windows automatically clips the regions occupied by visible, overlapping child windows when it prepares a a display context for the parent window's client area. This means the parent window can paint in the client area without overwriting the child windows. When non-overlapping child windows are specified for a parent window, Windows does not clip the child windows from the display context.

While child windows are visible, the parent can save painting time by skipping any calculations needed to determine the display under the windows. The parent must repaint this area when a child window is removed from the screen.

6.7 Popup Windows

A popup window is an application-defined and maintained window that can be displayed anywhere on the Windows screen. Popup windows, unlike child windows, lie on top of the other windows on the screen, making their complete contents visible at all times. For this reason, popup windows are typically used as temporary windows that display important information about the current command or request immediate input from the user.

6.7.1 A Popup's Parent and Class

Although every popup window, like every child window, has a parent window, a popup's parent does not restrict the popup to its client area. Windows lets the popup appear anywhere on the system screen. This means a popup can straddle several windows at the same time or even appear to be in another window.

A popup window, like any other window, must belong to a window class. The window class defines the default attributes of the window, including its window function. The popup's window class is almost never the same as the parent's window class. Most popup windows have very different functions from those of their parents.

6.7.2 Creating a Popup Window

You can create a popup window by supplying the WS_POPUP window style in a call to the **CreateWindow** function. For example, the call

```
hPopup = CreateWindow (
                    (LPSTR)"MyClass",  /* Window Class */
(LPSTR)"Popup 1",  /* Caption */
WS_POPUP,          /* Window Style */
10, 10,            /* Origin on screen */
200, 100,                  /* Width and height */
hTiledWindow,              /* Parent's handle */
(HMENU)NULL,               /* Menu handle */
hInstance,                 /* Module instance */
(LPSTR)NULL                /* Additional data */
);
```

creates a popup window that belongs to the previously registered window class "MyClass." The popup's parent is given by **hTiledWindow**. The popup window, when made visible using the **ShowWindow** function, is displayed at the screen coordinates (10, 10). The window is 200 units wide by 100 units high. The units depend on the current mapping mode of the parent when the **CreateWindow** function is called. Usually, units are in pixels.

In this example, the popup window uses the default menu, if any, defined by the window class. The popup could have been given any existing menu by supplying the menu's handle in the **CreateWindow** call.

6.7.3 Receiving Input and Messages

A popup window is an independent window that receives its own input and messages. This means that input and messages intended for a popup go directly to the popup, and are not passed through the parent.

When the **EnableWindow** function is used to disable a popup window, Windows passes any input that would have gone to the popup to the parent instead. It does not pass the input to the window that appears under the popup. This gives the parent an opportunity to examine the input and enable the popup if necessary.

6.7.4 The Popup-Parent Relationship

Since every popup window belongs to a parent window, actions that affect the parent can also affect the popup. The following is a list of actions that can affect popup windows:

1. Hiding the parent window or making it iconic does not hide the popup window. A popup window can be visible even when the parent is not.
2. Destroying a parent also destroys the popup. Windows sends a WM_DESTROY message to the popup window before sending one to the parent.
3. Sizing or moving a parent does not affect a popup window. Popup windows never receive a WM_SIZE message.

6.7.5 Overlapping Popups

Popup windows overlap if two or more are displayed in the same portion of the screen. The last popup to be displayed always appears on top. To avoid overlapping popups, the **AnyPopup** function can be used to determine whether or not a popup window is visible. If popups are overlapped, the **BringWindowToTop** function can be used to bring a popup to the top of the pile.

6.7.6 Painting Under a Popup Window

When any window retrieves a display context for its client area, Windows automatically clips the regions occupied by visible, overlapping popup windows from the context. This means the parent window can paint in the client area without overwriting any popup windows.

While popup windows are visible, a window can save painting time by skipping any calculations needed to determine the display under the windows. Since painting to the area behind a popup is clipped, there is no need to keep the area up-to-date.

6.7.7 The Message Box

The message box is a special predefined popup window that any application can use to display messages. To create and display the message box, the application must call the **MessageBox** function. The function displays the popup window at the center of the Window screen. The box contains a specified message, and waits for the user to make a selection from its menu.

In the following example, the **MessageBox** function creates a popup window containing the message: "Cannot find the file!":

```
MessageBox ( hWindow, (LPSTR)"Cannot find the file!",
                    (LPSTR)NULL, MB_CANCEL );
```

6.8 Window Styles

Windows provides several different window styles that can be combined to form different kinds of tiled, child, and popup windows. The following is a list of the window styles:

Style	Purpose
WS_TILED	Creates a tiled window.
WS_POPUP	Creates a popup window.
WS_CHILD	Creates a child window.
WS_ICONIC	Directs Windows to create a window that is initially iconic (closed). This style can be used with WS_TILED only.
WS_VISIBLE	Directs Windows to create a window that is initially visible (open).
WS_DISABLED	Directs Windows to create a window that is initially disabled. Disabled windows do not receive input.
WS_CLIPSIBLINGS	Directs Windows to exclude all child and popup windows from a parent window's clipping region.
WS_CLIPCHILDREN	Directs Windows to exclude all child windows from a parent window's clipping region.

WS_BORDER	Creates a border around a window.
WS_DLGFRAME	Creates a double border. It must not be used with WS_CAPTION.
WS_CAPTION	Creates a caption bar for a window. This style implies WS_BORDER.
WS_VSCROLL	Creates a vertical scroll bar for a window.
WS_HSCROLL	Creates a horizontal scroll bar for a window.
WS_SIZEBOX	Creates a size box for a window. The size box appears in the right corner of the caption bar. If the window has no caption bar, the size box request is ignored.
WS_SYSMENU	Creates a system menu in the caption.
WS_TILEDWINDOW	Creates a tiled window that has a border, caption, system menu, and size box.
WS_POPUPWINDOW	Creates a popup window that has a border and a system menu.
WS_CHILDWINDOW	Creates a child window.

6.9 Painting Windows

Any application can draw images, write text, and paint patterns in the client area of its window. To paint in a window, an application needs a display context for the window that it can pass to an appropriate GDI output function. An application can get a display context in one of two ways. It can call the **UpdateWindow** function and wait for Windows to send a WM_PAINT message, or it can retrieve a display context using the **GetDC** function. An application that has received a WM_PAINT message must use the **BeginPaint** function to retrieve a data structure containing a display context and other information. An application that uses the **GetDC** function gets just a display context.

Windows also sends a WM_PAINT message to a application whenever the application's window needs updating; for example, after a resizing operation.

6.9.1 Receiving a WM_PAINT Message

Whenever an application receives a WM_PAINT message, it must retrieve additional information about the painting request from Windows by calling the **BeginPaint** function. This function returns a data structure that includes a display context for the window and a flag indicating whether or not Windows erased the contents of the window.

Once the application has the display context, it can use the GDI graphics functions to draw images or write text in the window. When the window is painted, the application must signal the end of the operation by using the **EndPaint** function.

In the following example, the **BeginPaint** and **EndPaint** functions retrieve and release a painting structure that contains information used by **MyPaint** to paint the window.

```
PAINTSTRUCT        ps;
                .
                .
                .
case WM_PAINT:
    BeginPaint(hWindow, (LPPAINTSTRUCT)&ps);
    MyPaint(hWindow, ps.hdc, ps.fErase);
    EndPaint(hWindow, (LPPAINTSTRUCT)&ps);
    break;
                .
                .
                .
```

Windows sends WM_PAINT messages to the application until the **BeginPaint** and **EndPaint** functions have been used, or until the **ValidateRect** function has been used to validate the window's client area. The **BeginPaint** and **EndPaint** function should only be used when the application receives WM_PAINT messages and wishes to carry out the paint operation. The **ValidateRect** function can be used at any time to remove WM_PAINT messages from the application queue without actually processing them. If you pass the function a NULL rectangle pointer, Windows assumes that the entire window has been repainted and WM_PAINT messages are discontinued.

You can check a window to see if it is up-to-date by using the **UpdateWindow** function. This forces an immediate WM_PAINT message if the window needs painting. **UpdateWindow** is very fast if no painting is needed.

6.9.2 A Window's Mapping Mode

A window is initially given a text mapping mode. In a text mode, the origin is the upper left corner. The positive x direction is to the right. The positive y direction is down. An application can use the **SetMapMode**, **SetViewportOrg**, and **SetViewportExt** function to change a window's mapping mode to make it more convenient for graphics. For example, an application can set the origin at the lower left corner, make the positive x direction to the right, and the positive y direction up.

6.9.3 The Clipping Region

Windows maintains its own set of clipping regions for the windows on the system display. This means an application never needs to worry about overwriting another window. Windows also provides the dimensions of the screen area that needs updating as part of the WM_PAINT message. Thus, an application can also determine what part of its output it can clip.

6.9.4 Asynchronous Paint Messages

To reduce the number of times a window function is requested to update its window, Windows places WM_PAINT messages, generated by a window function or the user, at the end of the application queue and keeps the message at the end until it is the last message in the queue. This means that as new messages arrive, Windows places them in front of the WM_PAINT message.

If a new WM_PAINT message arrives while another is still in the queue, Windows combines the two messages into one. In this way, the application receives a single request instead of several.

6.9.5 Sizing Requests

Windows places a WM_SIZE message in the application queue whenever the user sizes or moves the application's window. Sizing messages, unlike paint messages, are not held.

Windows places separate sizing and painting requests in the queue. Since paint messages are withheld until all other messages are processed, the size message may occur long before the corresponding paint message.

For this reason, window functions should carry out sizing calculations as soon as they receive a sizing message, and must not wait for the paint message. If the function does not carry out sizing calculations immediately, messages causing actions that depend on the current window size may arrive before the paint message, and the subsequent actions will cause errors.

6.9.6 Zooming a Window

Windows sends a WM_SHOWWINDOW message to all windows when the user zooms a window and when the user restores a zoomed window to its previous state. If an application does not process the WM_SHOWWINDOW message, the **DefWindowProc** function automatically hides all windows when the user zooms, and shows all windows when the user restores the zoomed window.

Since Windows does not keep a record of which windows were hidden before the user zooms, **DefWindowProc** shows all windows, even if they were previously hidden.

Applications that do not wish to hide popups associated with the zoomed window, or show windows that were previously hidden must catch and process the WM_SHOWWINDOW message. The *wParam* parameter is TRUE if the window is to be shown, and FALSE if it is to be hidden. The low-order word of *lParam* is one of the following:

Parameter	Meaning
0	A tiled, child, or popup window is opening as a result of the **ShowWindow** function.
SW_PARENTCLOSING	A tiled window is closing into an icon. Also, a popup window is being hidden.
SW_PARENTOPENING	A tiled window is opening for the first time or from an icon. Also, a popup window is becoming visible for the first time or after being hidden.
SW_OTHERZOOM	A tiled window is zooming.
SW_OTHERUNZOOM	A tiled window is unzooming.

6.9.7 Painting the Window Background

Applications that paint their own window backgrounds should not wait for a WM_PAINT message. Waiting can cause undesirable results, especially if the WM_PAINT message does not arrive immediately after the window needs painting. Applications should instead process the WM_ERASEBKGND message, painting the background whenever the message arrives.

The WM_ERASEBKGND message is sent immediately after a window client area needs repainting, such as after moving or sizing a window. The *wParam* parameter has an handle to a display context for drawing. This is the same handle returned by the **BeginPaint** function. You can actually do any drawing you want when you get this message, although the application should not take too much time.

6.9.8 Using System Colors for Window Background

Applications can paint the window background with one of the standard system colors by registering the window class with a color number instead of a background brush. When a number is registered, the **DefWindowProc** function uses the indicated color when painting the window background.

System colors can be specified using the COLOR_* constants (see the *Microsoft Windows Reference Manual*). You must add one to a constant to use it. The following example illustrates how to set the color.

```
pClass->hbrBackground = (HBRUSH) (COLOR_BACKGROUND + 1);
```

6.9.9 The Display Context

Since all windows share the same system display device, Windows must carefully control the display to keep applications from overwriting and destroying windows that do not belong to them. To do this, Windows maintains a display context for each window. The display context contains information that defines the location and size of the window and is intended to be used in GDI output functions. Any application, however, can choose to let a window or window class maintain its own display context by explicitly defining the display context type when it creates the window class.

The CS_CLASSDC and CS_OWNDC class styles specify whether or not a display context is permanently allocated for the given window or window class. If CS_CLASSDC is given, one display context is allocated for all windows of the given class. If CS_OWNDC is given, one display context is allocated for each window. In both cases, it is up to the window using the display context to maintain the context.

An application that uses a class or private display context does not have to reset the attributes of the context each time it uses the context. Applications that use the default display context have to reset the attributes, because Windows restores a display context to its original values after the application releases it (with the **ReleaseDC** function). When using a class or private display context, the application is responsible for remembering the context's current state.

Unlike a default display context, Windows does not automatically update a class or private display context when the window is sized or moved. This means the application must watch for changes to the window and use the **ReleaseDC** and **GetDC** functions to update the display context before painting the window.

A window has changed when an application receives the WM_SIZE message. The window must use the **ReleaseDC** function to release the display context associated with the window, then use **GetDC** to retrieve an updated display context. The **ReleaseDC** function informs Windows that the application has released the display context for updating. The **GetDC** function retrieves the updated display context. The display context has a new clipping region and origin to match the window's current state.

Applications that use a class display context must update the display context each time a new window uses it. Since Windows cannot know when a new window acquires the display context, the application must develop its own techniques for using and releasing the context. Note that while a window is using a class display context, Windows prevents any second window from getting the context by locking a **GetDC** call until the display context becomes free through a **ReleaseDC** call.

By default, child windows use a default display context and must get a fresh display context each time they wish to paint. If an application wants its child windows to use class or private display context, it must specify the correct argument in the child's window class.

Applications that use the default display context must release the context as soon as possible after using it. When the default display context is in use, all other requests to use it must wait.

An application that uses a private display context does not need to call **ReleaseDC** until it receives a WM_SIZE message.

When a window is about to be obscured (e.g. popup window), Windows ignores any display context locks; it updates the display context immediately.

Warning

Windows maintains a 5-item cache for display contexts retrieved by the **GetDC** function. If applications retrievemore than 5 simultaneous display contexts, unpredictable system behavior may result.

6.10 Window Subclassing

All windows of a class share the same window input function. Given a handle to a window, it is possible to access and replace the input function for a class or for a window. Replacing the input function for a window is a technique called subclassing. It allows a programmer to intercept input for a predefined window class, such as an edit control, and alter the behavior of the window.

The following is an example of subclassing code:

```
LONG FAR PASCAL MyInputProc( hwnd, message, wParam, lParam )
HANDLE hwnd;
WORD message;
WORD wParam;
LONG lParam;
{
    switch ( message ) {
    case 1:
         /* Some code. */
         break;
    default:
         CallWindowProc( fp, hwnd, message, wParam, lParam );
    }
}
```

```
int FAR PASCAL WinMain( hInstance, hPrevInstance, lpCmdLine, CmdShow )
HANDLE hInstance;
HANDLE hPrevInstance;
LPSTR lpCmdLine;
BOOL CmdShow;
{
    MSG msg;
    FARPROC fp;
    HANDLE hwnd;  /* Handle to window from create window. */

    fp = GetClassLong(hwnd, GCL_WNDPROC );
    SetWindowLong(hwnd, GWL_WNDPROC, MyInputProc );

    while ( GetMessage( (LPMSG)&msg, 0 , 0, 0 ) ) {
         TranslateMessage( (LPMSG)&msg );
         DispatchMessage( (LPMSG)&msg );
         }
}
```

The default behavior for the window described by *hwnd* is defined by the class input function. The class input function is obtained by the **GetClassLong** call and placed in *fp*. The default behavior for the window is changed by the **SetWindowLong** call. The input function inherited from the class is replaced by **MyInputProc**. Messages that **MyInputProc** does not want to handle are passed back to the default input window function through the **CallWindowProc**.

By getting the default window function from the class, any previous subclassing using *hwnd* is ignored. Using **CallWindowProc** in the default messaging case ensures that **DefWindowProc** is called only once – from the bottom-most input function in the subclassing chain.

Chapter 7
Controls and Dialog Boxes

7.1 Introduction 149
7.2 Control Windows 149
7.3 Button Controls 150
7.3.1 Button Control Styles 150
7.3.2 Button Notification Messages 151
7.3.3 Button Control Messages 152
7.3.4 Button Control Features 152
7.3.5 Defining a Group Box 152
7.3.6 Designing Push buttons 153
7.4 Edit Controls 153
7.4.1 Edit Control Styles 153
7.4.2 Choosing a Height for Edit Controls 154
7.4.3 Entering Text in an Edit Control 155
7.4.4 Edit Control Keys 155
7.4.5 Printing Edit Control Text 156
7.4.6 Using Multiline Edit Controls 157
7.4.7 Using Word Wrapping in Edit Controls 157
7.4.8 Using Scroll Bars With Edit Controls 158
7.4.9 Edit Control Notification Messages 158
7.4.10 Edit Control Messages 159
7.4.11 Data Interchange and Edit Controls 161
7.5 Static Controls 161
7.5.1 Static Control Styles 161
7.5.2 Making Icon Controls 162

7.6 List Boxes 162
7.7 List Box Control Styles 163
7.7.1 List Box Notification Messages 163
7.7.2 List Box Messages 163
7.7.3 List Box Examples 166
7.7.4 Sending Messages to List Boxes 166
7.7.5 List Box Space Limitations 166
7.8 Scroll Bar Controls 167
7.8.1 Scroll Bar Styles 167
7.8.2 Scroll Bar Notification Messages 167
7.8.3 Scroll Bar Messages 168
7.8.4 Scroll Bar Alignment 168
7.9 WS_TABSTOP and WS_GROUP Styles 168
7.10 Dialog Boxes 169
7.10.1 Creating the Dialog Box 169
7.10.2 Creating a Dialog Function 169
7.10.3 Starting the Dialog Box 170
7.10.4 Using Modeless Dialog Boxes 170
7.10.5 Dialog Box Memory Requirements 170
7.10.6 Dialog Box Menu - Mouse Input 171
7.10.7 Changing a Dialog Box's Caption 171
7.10.8 Dialog Box Units of Measure 171
7.10.9 Scrolling Dialog Boxes 171
7.10.10 Default Response in Dialog Boxes 172
7.10.11 Creating Pretty Dialog Borders 172
7.10.12 Return Values From a Dialog Box 172
7.10.13 Accelerators And Modeless Dialog Boxes 173
7.10.14 Modeless Dialog Boxes and the Keyboard Interface 173
7.11 Default Responses in Message Boxes 174

7.1 Introduction

This chapter explains:

Control Windows
Dialog Boxes

7.2 Control Windows

Control windows, or just controls, are predefined child windows that any application can use for input and output. There are the following types of controls:

Class	Definition
BUTTON	These controls consist of buttons and boxes that the user can select with the mouse or keyboard.
EDIT	These controls consist of one or more lines of text that the user can edit.
STATIC	These controls are simple items such as icons, text, and boxes that do not respond to keyboard or mouse events. They are primarily used inside dialog boxes.
ListBox	These controls consist of a box containing one or more text strings from which the user can select zero or more.
ScrollBar	These controls consist of window scroll bars that let the user make a request to scroll the contents of an associated window.

Each user-defined class has one or more specific styles. The following sections describe each class and define the styles.

7.3 Button Controls

A button control is a small rectangular child window that represents a button that the user can turn on or off by clicking it with the mouse. Button controls can be used alone or in groups, and can either be labeled or appear without text. Button controls typically change appearance when the user clicks them.

7.3.1 Button Control Styles

There are the following styles:

BS_PUSHBUTTON	A push button is a box that contains a string. When you push a button, the parent window is notified. You push a button by clicking the mouse on it or pressing the SPACEBAR when the button is active.
BS_DEFPUSHBUTTON	A default push button is a thick bordered box that contains a string. When you push a button, the parent window is notified. You push a button by clicking the mouse on it, by pressing the SPACEBAR when the button is active, or by pressing the RETURN key.
BS_CHECKBOX	A check box is a little square with a character string to the right. If it is checked, a small black box appears inside the little square. When you click the box or string, the checkbox changes state and the parent window is notified. You click the box by clicking on it with the mouse or pressing the SPACEBAR when it is active.
BS_RADIOBUTTON	A radio button is the same as a check box, but is typically used in groups in which only one button at a time is checked. When a radio button is clicked or a cursor key is pressed to move within the group, it notifies its parent window. It is then up to the parent window to check the clicked radio button and uncheck all the rest, if necessary.

BS_AUTOCHECKBOX	An automatic check box automatically toggles its state whenever the user clicks on it.
BS_3STATE	A 3-state check box is identical to a check box control except that its character string can be grayed as well as the box being checked or unchecked. The check box is typically used whenever the application needs to show that the check box has been disabled.
BS_AUTO3STATE	An automatic 3-state check box automatically toggles its state when the user clicks on it.
BS_GROUPBOX	A group box control is a rectangle that has an identifying text string in its upper left corner. Group boxes are used to collect a group of radio buttons or other controls in a single unit.
BS_USERBUTTON	A user button is a user-definable button that notifies the parent when it is pushed. The notification includes a message to paint the button and highlight or disable it.

7.3.2 Button Notification Messages

Button controls use the WM_COMMAND message to notify the parent window of actions. In all cases, the *wParam* parameter contains the control ID. For BS_USERBUTTON styles, the high-order word of the *lParam* parameter contains a display context for the button, and the low-order word contains one of the following:

BN_CLICKED	The button has been clicked or the SPACEBAR is pressed when the button is active.
BN_PAINT	Request to repaint the button.
BN_HILITE	Request to highlight the button.
BN_UNHILITE	Request to remove the highlight.
BN_DISABLE	Request to draw a disabled button.

7.3.3 Button Control Messages

Button control messages are sent to buttons to control or modify them. There are the following button control messages:

BM_GETCHECK	Returns TRUE if a radio button or check box is checked. Otherwise, it returns FALSE. It always returns FALSE for a push button.
BM_GETSTATE	Returns TRUE if the mouse cursor is over the button and the the user is pressing a mouse button. Otherwise, it returns FALSE. For push buttons, it returns TRUE if the button is highlighted and FALSE if not.
BM_SETCHECK	Checks or unchecks a radio button or check box. If *wParam* is TRUE, a check is placed beside the button or box. If FALSE, the check (if any) is removed. This has no effect on push buttons.
BM_SETSTATE	Highlights a button or check box. If *wParam* is TRUE, the button is blackened, or the check box is emboldened. If FALSE, the highlight is removed. Push buttons cannot be highlighted.

7.3.4 Button Control Features

You can set the state of a 3-state check box using the **CheckDlgButton** function. The state can be any one of the following:

0 – Checked
1 – Unchecked
2 – Grayed

7.3.5 Defining a Group Box

You can define a group box by using the BUTTON class in a **CONTROL** statement. The following example illustrates how to define the box in a dialog template:

```
CONTROL "Group Box", 5, BUTTON, BS_GROUP|WS_VISIBLE|WS_GROUP, 5,5, 75,20
```

7.3.6 Designing Push buttons

A good rule of thumb for the width of a push button is: the number of characters in the push button name plus 2. A good push button height is 14.

7.4 Edit Controls

An edit control is a rectangular child window in which the user can enter text from the keyboard. The user selects the control and gives it the input focus by clicking the mouse inside it. The user can enter text when the control displays a flashing caret. The mouse can be used to move the cursor and select characters to be replaced, or position the cursor for inserting characters. The backspace key can be used to delete characters.

7.4.1 Edit Control Styles

There are the following edit control styles:

Style	Meaning
ES_LEFT	Creates an unframed field containing left-justified text.
ES_RIGHT	Creates an unframed field containing right-justified text. The text is justified as the user enters it.
ES_CENTER	Creates an unframed field containing centered text. The text is centered as the user enters it.
ES_MULTILINE	Creates a multiple-line edit control. The default control is a single-line control. The user can type any number of characters on a line. For multiple-line edit controls, use the ES_AUTOVSCROLL style and a scroll bar. The edit control shows as many lines as possible, and automatically scrolls when a carriage return/linefeed combination (CR/LF) is typed. Lines in a multiple-line edit control must be delimited with the carriage return/linefeed combination (CR-LF).

ES_AUTOVSCROLL	Creates an edit control that scrolls the control's contents up one page when the user types past the bottom of the control.
ES_AUTOHSCROLL	Creates an edit control that scrolls the control's contents right by 10 characters when the user types past the right edge of the control, and scrolls back to the left when the user presses the carriage return.

The WS_HSCROLL, WS_VSCROLL, and WS_BORDER window styles can also be used. Styles can be combined using the OR operator.

7.4.2 Choosing a Height for Edit Controls

When defining an edit control in a resource script file, use 8 units if the control has no border, and 12 units if the control has a border. Edit control units specified in the resource script file are always assumed to be 1/8 of a character high and 1/4 of a character wide.

When creating an edit control using the **CreateWindow** function, compute the minimum height by doing the following:

1. Determine the height of the font to be used in the control.
2. Use the **GetSystemMetrics** to determine the height and width of the lines used for the border.
3. Compute the minimum height by adding the font height to the height of the border.

When using a static text control as a label to an edit control, remember that the static control usually has no border. If you want the top of the text for a static control to line up with the text inside the edit control, the vertical position of the static text should be two units higher than the edit control.

Important

Edit controls will not display text if any part of the text does not fit within the control's border. Always make sure the edit control is set to at least the minimum height.

7.4.3 Entering Text in an Edit Control

The user can enter text by typing at the keyboard. All edit controls use the fixed pitch ANSI font to display characters. A multiple-line edit control does not supply an automatic carriage return if it encounters a line feed character.

An edit control expands tab characters into spaces. The control expands tabs into enough spaces to move the cursor to the next tab stop. Tab stops are assumed to be at every eighth character position.

If the user attempts to enter more text in an edit control than specified by the text limit set by the EM_LIMITTEXT message, the edit control indicates the error by beeping and does not accept the characters.

7.4.4 Edit Control Keys

Edit controls let the user move the caret and make selections using the following keys:

Key	Result
LEFT	Moves caret one character left
RIGHT	Moves caret one character right
UP	Moves caret one line up
DOWN	Moves caret one line down
SHIFT-LEFT	Extends selection one character left
SHIFT-RIGHT	Extends selection one character right
SHIFT-UP	Extends selection one line up
SHIFT-DOWN	Extends selection one line down
HOME	Moves caret to top left corner of edit control
END	Moves caret to the end of the line
BACKSPACE	Deletes the character to the left of the caret and resets the current selection
SHIFT-BACKSPACE	Deletes character to the right of caret and resets current selection

DELETE	Cuts current selection to clipboard
SHIFT-DELETE	Deletes current selection, but does not copy it to the clipboard
INSERT	Replaces current selection with text contents of the clipboard

---

Note

When caret moves off right or left edge of the edit control, the control automatically scrolls right or left by 1/3 of the control width.

7.4.5 Printing Edit Control Text

Applications must be careful when printing the contents of edit controls that use word wrapping. Word wrap text does not contain the end-of-line characters needed to reproduce the line breaks seen by the user in the edit control.

One way to print the contents of the edit control is to use the EM_GETLINECOUNT message to get the number of lines, then use the EM_GETLINE message to retrieve each line and send it to the printer.

Another method is to use the EM_FMTLINES message to add the end-of-line characters to the text in the edit controls. This method is typically used to prepare the text for copying to a file that will later be sent to printer.

The following example illustrates how to format text and copy it for later printing:

```
/* format the text */
        fFormatted = SendMessage(hwndEdit, EM_FMTLINES, TRUE, 0L);
/* get the text length */
        cch = SendMessage(hwndEdit, EM_GETTEXTLENGTH, 0, 0L);
/* allocate space for the text */
        pch = LocalAlloc(LPTR, cch+1);
/* get the text and put it into the allocate space */
        SendMessage(hwndEdit, EM_GETTEXT, cch, (LPSTR)pch);
/* remove the format  */
        if (fFormatted)
                SendMessage(hwndEdit, EM_FMTLINES, FALSE, 0L);
```

7.4.6 Using Multiline Edit Controls

You can create a multiline edit control by specifying the ES_MULTILINE control style. Multiline edit controls let the user type the end-of-line character.

The following messages can be used with multiline edit controls only:

EM_SETRECT
EM_SETRECTNP
EM_SCROLL
EM_LINESCROLL
EM_GETLINECOUNT
EM_LINEINDEX
EM_SETHANDLE
EM_GETHANDLE
EM_GETTHUMB

Applications that want to repaint a multiline edit control immediately after sending a WM_SETREDRAW message to the control must invalidate the control's client area and update the window explicitly.

```
SendMessage(hwndEdit, EM_SETREDRAW, NULL, (long)NULL);
InvalidateRect(hwndEdit, (LPRECT)NULL, TRUE);
UpdateWindow(hwndEdit);
```

7.4.7 Using Word Wrapping in Edit Controls

Multiline edit controls that do not have ES_AUTOHSCROLL style support automatic word wrapping. As the user types, the edit control automatically moves words to the next line if they extend beyond the right margin of the control. Word breaks are any sequence of one or more spaces. The maximum line length in a word wrap edit control is determined by the size of the control's client area.

Applications should use the the EM_LIMITTEXT message for word wrapping edit controls to prevent users from appending an unlimited number of spaces to the end of a line. Edit controls do not display leading or trailing spaces. Edit controls that have ES_AUTOHSCROLL style will not word wrap.

7.4.8 Using Scroll Bars With Edit Controls

An edit control can be given scroll bars when it is created. An edit control with scroll bars processes its own scroll bar messages. Edit controls without scroll bars process scroll messages that are sent by the parent window, but do not allow user scrolling input.

Applications that use scroll bars with their edit controls can use the EN_VSCROLL and EN_HSCROLL notification codes sent by an edit control to update the scroll bar whenever the user has caused an automatic scroll through typing. Edit controls send the the notification codes in a notification message whenever the user has caused an automatic vertical or horizontal scroll. The message is sent before the scrolling is actually carried out.

When an edit control receives a page up or page down command, the control scrolls one screenful of text.

7.4.9 Edit Control Notification Messages

Edit controls send a notification message to their parent window whenever a specific event occurs, such as receiving the focus or changing the text of the control. A notification message is a WM_COMMAND message in which the *wParam* parameter is the control ID, the low-order word of *lParam* is a window handle to the edit control, and the high-order word of *lParam* is one of the following notification codes:

Code	Meaning
EN_SETFOCUS	The edit control received the focus.
EN_KILLFOCUS	The edit control lost the focus.
EN_CHANGE	The user changed the content of the edit control, for example, as typed a character.
EN_VSCROLL	The user requested a vertical scrolling, but the scrolling has not yet been carried out.
EN_HSCROLL	The user has requested a horizontal scrolling, but the scrolling has not yet been carried out.
EN_ERRSPACE	The edit control ran out of space for text.

7.4.10 Edit Control Messages

An application can send messages to an edit control using the **SendMessage** function. Edit controls recognize the following messages:

Message	Purpose
EM_GETSEL	Returns the start and end character positions of the current selection. The long value returned has the start position in the low-order word and the end position in the high-order word.
EM_SETSEL	Selects all characters in the current text that are within the starting and ending character positions given by the *lParam* parameter. The starting position is in the low-order word, and the ending position is in the high-order word. The position values 0 to 32767 select the entire string.
EM_GETRECT	Retrieves the formatting rectangle of the control. The *lParam* parameter must be a long pointer to a data structure having **RECT** type. The control copies the dimensions to the structure.
EM_SETRECT	Sets the formatting rectangle for a control. The text is reformatted and redisplayed to reflect the changed rectangle. The *lParam* parameter must be a data structure having **RECT** type specifying the new dimensions of the rectangle.
EM_SETRECTNP	Same as EM_SETRECT, except that the control is NOT repainted. Any subsequent repaints cause the control to be repainted to reflect the changed formatting rectangle. This message is used when the field is to be repainted later.
EM_GETHANDLE	Returns the data handle of the buffer used to hold the contents of the control window. The *wParam* and *lParam* parameters are not used.

EM_SETHANDLE	Establishes the text buffer used to hold the contents of the control window. The *wParam* contains a handle to the buffer. The edit control uses this buffer to store the currently displayed text instead of allocating its own buffer. If necessary, the control reallocates this buffer. The *lParam* parameter is not used.
EM_GETLINECOUNT	Returns the number of lines of text in the edit control.
EM_LINEINDEX	Returns the number of characters to the beginning of a given line. The *wParam* parameter contains the desired line number.
EM_LINESCROLL	Scrolls the contents of the control by the given number of lines. The high-order word of *lParam* contains the number of lines; the low-order word contains the number of pixels to scroll horizontally.
WM_CUT	Sends the current selection to the clipboard in CF_TEXT format, then deletes the selection from the control window. The *wParam* and *lParam* parameters are not used.
WM_COPY	Sends the current selection to the clipboard in CF_TEXT format. The *wParam* and *lParam* parameters are not used.
WM_PASTE	Inserts the data from the clipboard into the control window at the current cursor position. Data is inserted only if the clipboard contains data in CF_TEXT format. The *wParam* and *lParam* parameters are not used.

The following messages window messages can also be used with edit controls.

WM_HSCROLL	WM_VSCROLL
WM_SETFOCUS	WM_KILLFOCUS
WM_SETTEXT	WM_GETTEXT
WM_GETTEXTLENGTH	WM_SETREDRAW
WM_ENABLE	

The following messages must not be used with single-line edit controls:

EM_SETRECT
EM_SCROLL
EM_GETLINECOUNT
EM_SETHANDLE
EM_GETTHUMB
EM_SETRECTNP
EM_LINESCROLL
EM_LINEINDEX
EM_GETHANDLE

7.4.11 Data Interchange and Edit Controls

You can use the **SendMessage** function to send WM_CUT, WM_COPY, and WM_PASTE messages to an edit control. All edit controls read and respond to these messages by copying the currently selected data to or from the clipboard.

7.5 Static Controls

Static controls are simple text fields, boxes, and rectangles that can be used to label, box, or separate other controls. Static controls take no input and provide no output.

7.5.1 Static Control Styles

There are the following static control styles:

Style	Meaning
SS_LEFT	Creates a left-justified text field. The text is formatted before it is displayed. Words that would extend past the end of a line are automatically wrapped to the beginning of the next line.
SS_RIGHT	Creates a right-justified text field. The text is formatted before it is displayed. Words that would extend past the end of a line are automatically wrapped to the beginning of the next line.
SS_CENTER	Creates a centered text field. The text is formatted before it is displayed. Words that would extend past the end of a line are automatically wrapped to the beginning of the next line.

SS_ICON	Draws an icon.
SS_BLACKRECT	Creates a black-filled rectangle.
SS_GRAYRECT	Creates a gray-filled rectangle.
SS_WHITERECT	Creates a white-filled rectangle.
SS_BLACKFRAME	Creates a box with a black frame.
SS_GRAYFRAME	Creates a box with a gray frame.
SS_WHITEFRAME	Creates a box with a white frame.
SS_USERITEM	Creates a user-defined item.

7.5.2 Making Icon Controls

You can make icon controls with the CONTROL statement

```
CONTROL "ResourceName", id, STATIC, SS_ICON, x, y, 0, 0
```

where *ResourceName* is the resource name of the icon resource (it must be enclosed in double quotation marks), *id* is the control ID, *x* and *y* are the horizontal and vertical positions of the top left corner of the icon. The last two parameters are ignored, but should be set to 0.

Windows automatically sizes the icon to the appropriate size for your display device.

7.6 List Boxes

A list box is a list of character strings. A list box is used whenever the application wishes to present a list of names, such as filenames, that the user can view and select.

A list box can contain any number of strings. When the user selects a string, the list box passes a notification message to the parent window. If a string is in the list but does not appear in the box, the user can use the list box's vertical scroll bar to scroll the string into view. The parent can highlight any string in the list and retrieve the currently highlighted string.

7.7 List Box Control Styles

There are the following list box control styles:

Style	Meaning
LBS_NOTIFY	The parent receives an input message whenever the user clicks or double clicks a string.
LBS_MULTIPLESEL	The string selection is toggled each time the user clicks or double clicks on the string. Any number of strings can be selected.
LBS_SORT	The strings in the list box are sorted alphabetically.
LBS_NOREDRAW	The list box display is not updated when changes are made. This style can be changed at any time by sending a WM_SETREDRAW message.

You can combine styles by using the OR operator (|). The WS_VISIBLE window style can also be used.

7.7.1 List Box Notification Messages

A list box having LBS_NOTIFY style notifies its parent whenever the user releases the mouse button over a string. The list box sends a message to the parent in which the *hWindow* parameter is the window handle of the parent, *Message* is WM_COMMAND, *wParam* is the ID of the list box control, the low-order word of *lParam* is the window handle of the list box, and the high-order word of *lParam* is either 1 for a single click, or 2 for a double click.

If the user releases the mouse over anything but a string, the state of the list box remains unchanged.

7.7.2 List Box Messages

You can send messages to a list box by using the **SendMessage** function. List boxes recognize the following messages:

LB_ADDSTRING	Adds a string to the list box. The *lParam* parameter is a long pointer to the string to be added. It must be null-terminated. If the list box is not sorted, the string is appended to the end of the list. Otherwise, it is inserted into the list after sorting. The list box returns the index to the string.
LB_INSERTSTRING	Inserts a string into the list box. The *lParam* parameter is a long pointer to the string; *wParam* is an index to the position to receive the string. If *wParam* is –1, the string is appended to the end of the list. No sorting is performed. The list box returns *wParam*.
LB_DELETESTRING	Deletes a string from the list box. The *wParam* parameter is an index to the string to be deleted. The list box returns a count of strings remaining in the list.
LB_RESETCONTENT	Deletes all entries in the list box if the array address is NULL.
LB_SETSEL	Sets the selection state of a string. The low-order word of *lParam* is an index that specifies which string to set; *wParam* is a Boolean value specifying how to set the selection. If it is TRUE, the string is highlighted; if FALSE, any highlight is removed. If *lParam* is –1, all strings in the list are selected. This message should be used only with multiple-selection type list boxes.
LB_SETCURSEL	Selects a string and scrolls it into view, if necessary. The *wParam* is the index of the string to select. If *lParam* is –1, no new selection is made. If a new string is selected, the list box removes the highlight from the previously selected string. This message should only be used with single-selection type list boxes.
LB_GETSEL	Returns the selection state of a string. The *wParam* parameter is an index to the string.

LB_GETCURSEL	Returns the index of the currently selected item, if any. It returns –1 if no item is selected or the list box has multiple-selection type.
LB_GETTEXT	Copies a string from the list into a buffer. The *wParam* parameter is the index of the string to be copied; *lParam* is a long pointer to the buffer to receive the string. The buffer must have sufficient space for the string and a terminating null character. The list box returns the length of the string (excluding the null character). It returns –1 if *wParam* is not a valid index.
LB_GETTEXTLEN	Returns the length of a string in the list (excluding the terminating null character). The *wParam* parameter is the index to the string.
LB_GETCOUNT	Returns a count of the number of items in the list.
LB_SELECTSTRING	Changes the current selection to the first string having the specified prefix. The *lParam* parameter is a long pointer to the prefix string. The string must be null-terminated. A string is selected only if its initial characters match the characters in the prefix string. The list box returns the index of the selected item. This message must not be used with list boxes having multiple-selection type.
LB_DIR	Adds a list of the files from the current directory to the list. The *lParam* parameter is a long pointer to a file specification string. Only files whose names match this string are listed. The string can contain wildcard characters (e.g., *.*). The *wParam* parameter is an MS-DOS attribute value. Items from the directory are added to the current list box contents. The list box returns a count of items displayed.

WM_SETREDRAW	Sets the redraw flag so that changes to the list are displayed immediately. If *wParam* is TRUE, the flag is set and redrawing is carried out after each change to the list. If *wParam* is FALSE, the flag is cleared and no redrawing is done.

A list box returns the value LB_ERR if the requested action cannot be carried out. It returns the value LB_ERRSPACE if there is not sufficient memory to store the new string.

7.7.3 List Box Examples

The following example illustrates how to make a major set of changes to a list box:

```
SendMessage(hwndCtl, WM_SETREDRAW, FALSE, (LPSTR)0);
        /* create the new contents */
SendMessage(hwndCtl, LB_ADDSTRING, 0, (LPSTR)"Start");
SendMessage(hwndCtl, LB_ADDSTRING, 0, (LPSTR)"End");
SendMessage(hwndCtl, WM_SETREDRAW, TRUE, (LPSTR)0);
```

A list box control can be created in a dialog box by adding the entry

```
LISTBOX    "",id, x, y, width, height, style
```

to the dialog box definition.

7.7.4 Sending Messages to List Boxes

Applications should always use the **SendMessage** function to send messages to list box controls. **SendMessage** temporarily suspends execution of the application while the control processes the message. This permits the control to return values to the parent if errors are detected. In particular, applications must not attempt to use **PostMessage** for add and insert functions since out-of-space errors that may occur will not be detected.

7.7.5 List Box Space Limitations

List box controls store their strings in the Windows global heap. This means that the total number of strings you may add to a list box control depends on the space available in this heap, and not on the space available in the application.

7.8 Scroll Bar Controls

A scroll bar control is a special control window that has the same appearance and function as the scroll bars used in ordinary windows. Unlike scroll bars, scroll bar controls can be positioned anywhere in a window and used whenever needed to provide scrolling input for a window.

A scroll bar is a rectangle containing a thumb and direction arrows at both ends. The scroll bar sends a notification message to its parent whenever the user clicks the mouse in the control. The parent is responsible for updating the thumb position, if necessary.

7.8.1 Scroll Bar Styles

There are the following scroll bar control styles:

Style	Meaning
SBS_VERT	Creates a vertical scroll bar.
SBS_LEFTALIGN	Creates a vertical scroll bar that is aligned along the left edge of a window.
SBS_RIGHTALIGN	Creates a vertical scroll bar that is aligned along the right edge of a window.
SBS_HORZ	Creates a horizontal scroll bar.
SBS_TOPALIGN	Creates a horizontal scroll bar that is aligned along the top edge of a window.
SBS_BOTTOMALIGN	Creates a horizontal scroll bar that is aligned along the bottom edge of a window.

7.8.2 Scroll Bar Notification Messages

A scroll bar control sends a WM_VSCROLL or WM_HSCROLL message to its parent whenever the user clicks the mouse in the control. The *wParam* and *lParam* parameters contain the same values as defined for the scrolling messages for a standard window. The high-order word of *lParam* contains the window handle of the scroll bar control.

7.8.3 Scroll Bar Messages

There are no predefined scroll bar control messages. Instead, the **SetScrollPos**, **GetScrollPos**, **SetScrollRange**, and **GetScrollRange** functions can be used to modify the appearance of the scroll bar. The functions take the window handle of the scroll bar control to be modified. The SB_CTL argument must also be supplied to direct Windows to operate on a scroll bar control.

In the following example, the current thumb position of the scroll bar control given by **hScroll** is set to 23:

```
SetScrollPos( hScroll, SB_CTL, 23, TRUE );
```

7.8.4 Scroll Bar Alignment

If you give no alignment bits in the style, the scroll bar is stretched to fill the given rectangle. If you pass in alignment bits such as SBS_LEFTALIGN, SBS_RIGHTALIGN, or others, the scrollbar is given a standard size and is aligned against the specified side of the given rectangle.

7.9 WS_TABSTOP and WS_GROUP Styles

The WS_TABSTOP and WS_GROUP styles are special window styles that are typically used with controls in a dialog box to provide a method to move the input focus from one control to the next using the keyboard.

The WS_TABSTOP style indicates that input focus can be transferred to the control by the user pressing the TAB key. Some controls, such as buttons and edit controls, have this style by default. For others, such as list boxes, the style must be explicitly given.

A control group is a collection of controls in which the user can press the DIRECTION keys to move the input focus from one control to the next. The WS_GROUP style indicates which controls belong to the group. You must assign the style to the first and last control in the group. All controls between the first and last controls belong to the group.

In applications that provide full keyboard support, the WS_GROUP and WS_TABSTOP styles are often used together. This lets the user press the TAB key to move the nput focus to the beginning of a group, then use the DIRECTION keys to move from item to item in the group.

7.10 Dialog Boxes

A dialog box is a popup or child window that contains one or more window controls. A dialog box is typically used whenever an application must prompt the user for additional information about a current command selection. In other words, a dialog box is a temporary window that is created for special-purpose input and destroyed immediately after use. For example, the MS-DOS Executive creates a dialog box when the user selects the End Session command. The dialog box asks for confirmation of the selection.

7.10.1 Creating the Dialog Box

The complete dialog box consists of a dialog template and a dialog function. The dialog template defines the attributes and window style of the dialog box. The dialog function serves the same purpose as a window function. It receives and processes input to the dialog box.

You can create a dialog box by preparing a dialog template in the application's resource script. For example, the following template defines a dialog box that contains a static control and a push button:

```
sample DIALOGBOX 10, 10, 40, 15
STYLE WS_POPUP|WS_BORDER
BEGIN
        CONTROL "", 1, STATIC, SS_GRAYFRAME, 1, 1, 38, 12
        PUSHBUTTON "Power", 2, 10, 4, 20, 10
END
```

All units in the dialog template are assumed to be in a coordinate system where a character is 4 units wide and 8 units high.

A complete definition of the dialog template format is given in the *Microsoft Windows Reference Manual.*

7.10.2 Creating a Dialog Function

A dialog function is simply a window function for a dialog box. The function is no different from other window functions except that it can receive predefined window messages intended especially for dialog boxes.

When a dialog box is initialized, Windows sends the dialog function a WM_INITDIALOG message. If the box has one or more edit controls, the *wParam* parameter contains the window handle of the control having the input focus. The *lParam* parameter is not used.

The dialog function must have the FAR and PASCAL attributes. For multiple instance applications, the **MakeProcInstance** function must be used to create a new instance of a function address.

7.10.3 Starting the Dialog Box

You can start the dialog box by using the **CreateDialog** or **DialogBox** function. The functions load the dialog template resource from the application's executable file, then create a popup or child window that matches the template's specifications. The **DialogBox** function creates a modal dialog box that takes control from the application and keeps control until the box's dialog function calls the **DialogEnd** function. The **CreateDialog** function creates a modeless dialog box that runs as an independent window.

7.10.4 Using Modeless Dialog Boxes

A modeless dialog box is simply a dialog box that receives and processes input independently. The parent window creates the dialog box and leaves the box running while it continues other processing.

In general, the dialog function of a modeless dialog box must post a message to the parent when it has input for the parent. It must also destroy the dialog box when it is no longer needed. A dialog box can be destroyed using the **DestroyWindow** function.

7.10.5 Dialog Box Memory Requirements

The control windows contained in a dialog box allocate memory from the application's local heap. Applications that use large or complex dialog boxes should have least 1024 bytes of heap space in addition to any other space requirements.

7.10.6 Dialog Box Menu - Mouse Input

Windows passes mouse clicks in a dialog box's menu to the dialog box's input function. Note that in a dialog box, there must be no conflict between menu item IDs and control IDs. All IDs for all items in the dialog box must be unique.

7.10.7 Changing a Dialog Box's Caption

You can use the **SetWindowText** function to change the caption of a dialog box. The function is typically used when the dialog box input function receives the WM_ INITDIALOG message.

7.10.8 Dialog Box Units of Measure

The units of measure used in dialog box templates are as follows:

X coordinate
: Each unit is 1/4 the width of a character.

Y coordinate
: Each unit is 1/8 the width of a character.

Width
: Each unit is 1/4 the width of a character.

Height
: Each unit is 1/8 the width of a character. This height does not include the character's external leading value.

Character widths are a function of the default system font.

7.10.9 Scrolling Dialog Boxes

In dialog boxes, the cursor keys have specific functions that depend on the controls in the box. For example, the keys move the input focus from control to control in group boxes, move the cursor in edit controls, and scroll the contents of list boxes. This means the cursor keys cannot be used to scroll the contents of any dialog box that has its own scroll bars. If a dialog box has scroll bars, it is the application's responsibility to provide an appropriate keyboard interface for the scroll bars. Note that the mouse interface for scrolling is always available.

7.10.10 Default Response in Dialog Boxes

Windows sends a default response message to the dialog input function whenever the user presses the RETURN, CANCEL, or ESCAPE key when in a dialog box other than a message box.

If the dialog box has a default button, the RETURN key sends a WM_COMMAND message along with the ID associated with the default button. If the box has no default button, the predefined value IDOK is sent with WM_COMMAND instead. The CANCEL or ESCAPE key sends a WM_COMMAND and the IDCANCEL value whether or not a default button exists.

7.10.11 Creating Pretty Dialog Borders

You can create pretty borders for modeless dialog boxes by using the WS_DLGFRAME style in the STYLE statement of your dialog template.

```
mydialog DIALOG 10, 10, 100, 50
STYLE WS_DLGFRAME | WS_POPUP | WS_VISIBLE
BEGIN
        ...
END
```

7.10.12 Return Values From a Dialog Box

You can return values from a dialog box to the parent window's window function by setting the return value using the **SetWindowLong** function. The third argument is the long value to be returned by the window function.

For a dialog box, the **GetWindowLong** and **SetWindowLong** functions can be used to get or set the address of the dialog function. Applications can use the **GetWindowWord**, **GetWindowLong**, **SetWindowWord**, and **SetWindowLong** functions to set and retrieve 4 bytes of additional information associated with a dialog box's window. These 4 bytes, at offset 8 in the window structure, are not used by the dialog manager and are therefore free for use by the application. The bytes are initialized to zero.

7.10.13 Accelerators And Modeless Dialog Boxes

If the application uses accelerators and has a modeless dialog box, then the WinMain function should test in the following order:

```
while (GetMessage((LPMSG)&msg, (HWND)NULL, 0, 0))
    {
    if (!(hwndModelessDlg != NULL
                && IsDialogMessage(hwndModelessDlg, (LPMSG)&msg))
                && TranslateAccelerator(hWnd, hAccel, (LPMSG)&msg) == 0)
        {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage(LPMSG)&msg);
        }
    }
exit(msg.wParam);
```

This assumes that **hwndModelessDlg** is non-NULL only when the dialog box exists.

7.10.14 Modeless Dialog Boxes and the Keyboard Interface

If the application has a modeless dialog box, the following lets the modeless dialog box use the keyboard interface:

```
while (GetMessage((LPMSG)&msg, (HWND)NULL, 0, 0))
    {
    if (!(hwndModelessDlg != NULL
                && IsDialogMessage(hwndModelessDlg, (LPMSG)&msg)))
        {
        TranslateMessage((LPMSG)&msg);
        DispatchMessage(LPMSG)&msg);
        }
    }
exit(msg.wParam);
```

This assumes that **hwndModelessDlg** is non-NULL only when the dialog box exists.

7.11 Default Responses in Message Boxes

Message boxes can have one of the following combination of buttons:

a. OK
b. OK CANCEL
c. ABORT RETRY IGNORE
d. YES NO CANCEL

The application defines which combination is to be used. Each combination has a predefined default button.

Message boxes can pass any one of the following values with a WM_ COMMAND message:

IDMBYES
IDMBNO
IDMBCANCEL
IDMBABORT
IDMBRETRY
IDMBIGNORE

The CANCEL and ESCAPE keys invoke the Cancel button if there is one, otherwise it is ignored. The RETURN and EXECUTE keys invoke the default button.

Chapter 8
Using System Resources

8.1 Introduction 177
8.2 Memory Management 177
8.2.1 Freeing Memory 177
8.2.2 Locking Global and Local Handles 177
8.2.3 Locking the Data Segment 177
8.2.4 Discardable Memory 178
8.2.5 Changing Discardable Memory 178
8.3 Resources 179
8.3.1 Changing the WIN.INI File 179
8.3.2 Changing Font Resources 179
8.3.3 Resource Storage 180
8.4 Dynamic Linking 180
8.5 Changing Menus 181
8.6 Using the System Menu 185
8.6.1 The WM_SYSCOMMAND Message 185
8.6.2 The Default System Menu 185
8.6.3 Changing the System Menu 185
8.6.4 Adding Commands to the System Menu 186
8.6.5 Locking the System Menu 187
8.6.6 System Keys Without the Input Focus 187
8.6.7 System Menu Accelerators 188
8.7 Using the Mouse Cursor When No Mouse Is Available 188
8.8 Rules About Disk Files 190

8.8.1 Creating and Using a File 191
8.8.2 Prompting for a File Before Opening 191
8.8.3 Opening an Existing File 191
8.8.4 Creating Temporary Filenames 192
8.8.5 Filename Syntax 192
8.8.6 Filename Conversions 192
8.8.7 International Characters in Strings 192
8.8.8 Finding Directory Names 192
8.8.9 Checking For File Existence 193
8.8.10 Displaying Errors When Files Are Open 193
8.8.11 TEMP Environment Variable 193
8.9 Using a Printer 194
8.9.1 How to Print 194
8.9.2 How to Print Using Banding 194
8.9.3 Print Abort Function 195
8.9.4 Aborting a Print Operation With ABORTDOC 196
8.9.5 How to Abort a Print Operation 196
8.9.6 Errors During Printing 199
8.9.7 Setting Up the Printer 200
8.10 Miscellaneous Rules 201
8.10.1 Holding Handles 201
8.10.2 Word Aligned Stack 201
8.10.3 How Windows Displays Icons 201

8.1 Introduction

This chapter presents some tips on how to use the Windows system resources. In particular, it describes:

Memory Management
Resources
Dynamic Linking
Menus
System Menu
Diskette Files
Printing

8.2 Memory Management

This section provides tips for using the Windows memory management system.

8.2.1 Freeing Memory

Applications are responsible for freeing any global memory they may have allocated before terminating. Windows automatically frees any local memory the application may have allocated.

8.2.2 Locking Global and Local Handles

Applications should use the **GlobalLock** and **LocalLock** functions to lock global and local handles before assigning values to the specified memory locations. Locking the handles prevents the memory object associated with the handles from moving while being used.

8.2.3 Locking the Data Segment

You can use the **LockData** and **UnlockData** functions to lock and unlock an application's data segment. Data segment locking is required whenever a long pointer to data in the data segment is passed to Windows functions that perform memory allocation. Locking the segment prevents the segment from being moved and the given pointer from becoming invalid.

Windows never locks the data segment automatically.

8.2.4 Discardable Memory

Whenever an application attempts to allocate more memory than is available, Windows satisfies the request by removing discardable memory until the specified number of bytes have been freed. If the application is trying to allocate discardable memory, Windows stills discards existing discardable memory. If sufficient memory is still not available after discarding, Windows returns an error.

Discardable memory cannot be discarded until it has been allocated.

An application can guarantee that an allocation request succeeds by allocating a moveable, discardable object of size zero. Windows does not discard memory if its current lock count is not zero.

The **GlobalLock** function returns NULL if the object has been discarded. Discarded objects always have a lock count of zero.

8.2.5 Changing Discardable Memory

You can make a discardable object non-discardable, or a non-discardable object discardable, by using the **GlobalReAlloc** function and the GMEM_MODIFY flag.

Example

```
GlobalReAlloc( hMem, 0L, GMEM_MODIFY );
```

This call changes a discardable object to a non-discardable object.

8.3 Resources

This section provides tips on how to use application resources such as fonts, icons, and menus.

8.3.1 Changing the WIN.INI File

Applications that make changes to the WIN.INI file are responsible for informing all other applications of the change by sending a WM_ WININICHANGE message to the applications. The **EnumWindows** function can be used to retrieve the window handles of each each.

The following example illustrates how to signal a font change:

```
int FAR PASCAL EnumCallBack(hWnd,   )
HWND hWnd;
{
        SendMessage(hWnd, WM_WININICHANGE ..);
}

        .
        .
        .
while (EnumWindows((FARPROC)EnumCallBack))
        ;
        .
        .
        .
```

8.3.2 Changing Font Resources

Applications that add or remove fonts from the system using the **AddFontResource** and **RemoveFontResource** functions are responsible for informing all other applications of the change by sending a WM_FONTCHANGE message to the applications. The **EnumWindows** function can be used to retrieve the window handles of all windows.

The following example illustrates how to signal a font change:

```
int FAR PASCAL EnumCallBack(hWnd,   )
HWND hWnd;
{
        SendMessage(hWnd, WM_FONTCHANGE ..);
}
        .
        .
```

```
.
while (EnumWindows((FARPROC)EnumCallBack))
        ;
        .
        .
        .
```

8.3.3 Resource Storage

Bitmaps are GDI objects, so Windows does not free them from memory when an application terminates. It is the application's responsibilty to free any bitmaps it has created.

Cursors and icons are not GDI objects and Windows does free these resources.

8.4 Dynamic Linking

Applications can link dynamically to functions in Windows libraries. The application can either specify the desired function as imported functions in the application's module definition file, or use the **GetProcAddress** function to search for and load the function directly.

Applications must not attempt to link to functions in other applications.

When entering the names of exported or imported functions in the module definition file, you must spell the function names using the same case as they are spelled in the object file. The object file contents depend on the calling convention used to compile it.

When compiling with the Pascal calling convention, you should spell function names in uppercase letters. When compiling with other conventions, spell them as they appear in the source file.

When you import a function using the **GetProcAddress** function, you must spell the function name exactly as it appears in the module definition file of the module containing the function.

8.5 Changing Menus

You can use the **ChangeMenu** function to change the items in a menu. **ChangeMenu** can be used to append, insert, modify, and delete an item. You specify which operation you wish to perform by specifying one of the following values in the **ChangeMenu** call:

MF_APPEND	Append to the end of the menu
MF_INSERT	Insert in the menu
MF_CHANGE	Modify an item
MF_DELETE	Delete an item

To carry out a change, you need to identify the item you wish to change. You can identify items in one of two ways: by position (MF_BYPOSITION) or by command (MF_BYCOMMAND). When you identify the item by position, you specify where the item is relative to the left end of the menu. The item at the left end is item 0. The third item from the left is item 2. If you identify the item by command, you specify the command value that the menu item returns. For example, if the menu item returns 100 when selected, the item is item 100.

The following example illustrates how to change the third item in a static menu to a bitmap. The item is a command item that returns the command value 100.

```
hBitmap = LoadBitmap ( ... );

ChangeMenu(hMenu,              /* handle to static menu */
                   2,                    /* identifies menu item (by position) */
                   (LPSTR)(DWORD)hBitmap,  /* handle to bitmap */
                   100,                     /* item's command value */
                   MF_CHANGE                /* modify item */
                   | MF_BYPOSITION          /* identify by position */
                   | MF_BITMAP);            /* use a bitmap */
```

The following example illustrates how change an item in a static or popup menu to a bitmap. The item is a non-popup menu item that returns the command value 100.

```
ChangeMenu(hMenu,              /* handle to static or popup menu */
                   100,                  /* item's command value */
                   (LPSTR)(DWORD)hBitmap,  /* handle to bitmap */
                   100,                     /* item's command value */
                   MF_CHANGE           /* modify item */
                   | MF_BYCOMMAND           /* identify by command */
                   | MF_BITMAP);            /* use a bitmap */
```

In this example, **ChangeMenu** looks at all items in the specified menu to find the one item that has the command value 100. If this menu contains any popup menus, the function looks at all menu items in the popup too.

When changing menus, remember the following:

1. Popup menus do not have command values, so you cannot identify them by command.
2. Use the **DrawMenuBar** function to repaint the menu whenever you make changes while the menu is being displayed.
3. The MF_SEPARATOR value, used to insert a horizontal dividing line between menu items, can only be used with popup menus.

The following example illustrates how to create a popup menu and add it to another menu:

```
/* create a handle for a popup menu */
    hPopupMenu = CreateMenu();

/* build the popup from scratch by appending a menu item */
    ChangeMenu(hPopupMenu,  /* handle to popup menu */
                    NULL,                   /* use NULL when appending */
                    (LPSTR)"Alpha", /* item name */
                    100,                    /* item command value */
                    MF_APPEND);             /* append mene item */

/* append the new popup menu to the window's menu bar */
    ChangeMenu(GetMenu(hwnd),               /* handle to window's menu bar */
                    2,                      /* third item from left */
                    (LPSTR)"PopUp", /* item's new name */
                    hPopupMenu,             /* handle to popup menu */
                    MF_CHANGE/* modify menu */
                    | MF_POPUP);            /* use a popup menu */
```

You can also change the status of a menu item by using the **ChangeMenu** function. The status of a menu item can be enabled or disabled, checked or unchecked, highlighted or not highlighted. To set the status, specify combinations of the following:

Status	Meaning
MF_ENABLED	Enables the menu item for user selection.
MF_DISABLED	Disables the menu item to prevent user selection. The menu item looks enabled but does not respond to user selection.

MF_GRAYED
: Disables the menu item to prevent user selection. The menu item looks greyed.

MF_CHECKED
: Places a checkmark to the left of the menu item. Checkmarks can be used with popup menu items only.

MF_UNCHECKED
: Removes the checkmark (if any) from beside the menu item.

MF_HILITE
: Highlights the menu item. Menu item is displayed in reverse video.

MF_UNHILITE
: Removes the highlight (if any) from the menu item.

When the user selects a menu item, Windows automatically highlights it.

You can also use the **ChangeMenu** function to add additional rows or columns to a menu. To do so, specify the one of the following:

MF_MENUBREAK
: Starts a new row in menu bars, or starts a new column in popup menus.

MF_MENUBARBREAK
: Starts a new row in menu bars, or starts a new column in popup menus. In popup menus, the new column is separated from the previous column with a vertical bar.

Note

If an application changes the state of an item on the menu bar (for example, disables it), then it should call **DrawMenuBar** to update the display of the menu bar.

The following call appends "File" to a menu. Note that the *wIDChangeItem* parameter is zero because the item is being appended.

```
ChangeMenu(hMainMenu, 0, (LPSTR)"File", IDFILE, MF_APPEND | MF_STRING);
```

The following example shows two different ways to replace the "File" menu item with the string "Files Open". In the first call, the menu item is

identified by command value; in the second, it is identified by position. (The "File" item is assumed to be the third item on the menu.)

```
ChangeMenu(hMainMenu, IDFILE, (LPSTR)"Files Open", IDFILESOPEN,
           MF_BYCOMMAND | MF_CHANGE | MF_STRING);


ChangeMenu(hMainMenu, 2, (LPSTR)"Files Open", IDFILESOPEN,
           MF_BYPOSITION | MF_CHANGE | MF_STRING);
```

The following example shows a sequence of calls which create a popup menu, add items to the menu (separating the items into columns and drawing a horizontal dividing line), and associate the new popup menu with the "Files Open" menu item on the main menu.

```
/* create a popup menu */
      hPopupMenu = CreateMenu();
/* append an item to the popup menu */
      ChangeMenu(hPopupMenu, 0, (LPSTR)"classic.c", IDEXAMPLE,
                                          MF_APPEND | MF_STRING);
/* draw a horizontal dividing line */
      ChangeMenu(hPopupMenu, 0, (LPSTR)0, 0, MF_SEPARATOR);
/* append another item */
      ChangeMenu(hPopupMenu, 0, (LPSTR)"fun.c", IDFUN,
                                          MF_APPEND | MF_STRING);
/* start a new column for the next item */
      ChangeMenu(hPopupMenu, 0, (LPSTR)"today.c", IDTODAY,
                        MF_MENUBARBREAK | MF_APPEND | MF_STRING);
/* draw another horizontal dividing line */
      ChangeMenu(hPopupMenu, 0, (LPSTR)0, 0, MF_SEPARATOR);
/* Append another item */
      ChangeMenu(hPopupMenu, 0, (LPSTR)"guitar.c", IDGUITAR,
                                          MF_APPEND | MF_STRING);
/* associate the popup menu with the "Files Open" menu item */
      ChangeMenu(hMainMenu, IDFILESOPEN,
                        (LPSTR)"Files Open", (WORD)hPopupMenu,
                        MF_BYCOMMAND | MF_STRING | MF_POPUP | MF_CHANGE);
```

8.6 Using the System Menu

This section explains how to use the system menu.

8.6.1 The WM_SYSCOMMAND Message

When the user selects an item in the system menu, Windows sends a WM_SYSCOMMAND message to the application. The *wParam* parameter contains a predefined constant that identifies the command selected. An application can initiate its own system command by passing a WM_SYSCOMMAND message to the **DefWindowProc** function as in the following example:

```
DefWindowProc(hWindow, WM_SYSCOMMAND, SC_ICON, 0L);
```

There is also a SC_PREVWINDOW item that can be added to the system menu.

If the user is using the keyboard interface, Windows sends a WM_SYSCHAR, WM_SYSKEYDOWN, or WM_SYSKEYUP message to the application. An application can carry out requests by passing all WM_SYSCHAR, WM_SYSKEYDOWN, and WM_SYSKEYUP messages to the **DefWindowProc** function.

8.6.2 The Default System Menu

For each window, Windows creates a default system menu that contains the commands that can be used with that given type of window. For example, in a popup window, the Icon command is grayed since popup windows cannot be made iconic. The **GetSystemMenu** function returns a handle to this default system menu. An application can, if desired, change or disable any item in the default system menu. This is typically done when the application initializes its window or receives a WM_INITMENU message.

8.6.3 Changing the System Menu

To change the system menu, you must retrieve a handle to the system menu by using the **GetSystemMenu** function, then use the **ChangeMenu** function to make changes. You can identify system menu items by command if you use the following values:

SC_SIZE
SC_ICON
SC_NEXTWINDOW
SC_VSCROLL
SC_MOUSEMENU
SC_CLOSE
SC_MOVE
SC_ZOOM
SC_PREVWINDOW
SC_HSCROLL
SC_KEYMENU

If you have changed the system menu, you must process any WM_SYSCOMMAND messages corresponding to the menu items you added or changed. Windows sends a WM_SYSCOMMAND message to the application when the user selects commands for the system menu. WM_SYSCOMMAND messages for unchanged commands must be sent to the **DefWindowProc** function.

You can restore the system menu to its initial state by calling **GetSystemMenu** with the second parameter set to TRUE.

You can check or gray system menu items by processing the WM_INITMENU message. Windows sends the WM_INITMENU message to the application whenever the user selects the system command menu (or any other popup), but before Windows actually displays the menu. Note that under some circumstances, Windows automatically grays one or more system commands.

8.6.4 Adding Commands to the System Menu

When the user invokes commands that an application has added to the system command, Windows passes a WM_SYSCOMMAND message to the window function along with the ID of the menu item. Menu items not on the system menu arrive as WM_COMMAND.

Applications that do not process WM_SYSCOMMAND messages directly must pass the message along to the **DefWindowProc** function for processing.

For example, the following code adds a separator and a menu item "About..." to the system menu:

```
#define IDABOUT ...

HMENU hMenu = GetSystemMenu(hWnd, FALSE);

ChangeMenu(hMenu, 0, NULL, id, MF_APPEND | MF_SEPARATOR);
ChangeMenu(hMenu, 0, (LPSTR)"About ...", IDABOUT,MF_APPEND | MF_STRING);
```

The application's window function processes this menu item as follows:

```
switch(message) {
          ...
          case WM_SYSCOMMAND:
              switch (wParam) {
                    case IDABOUT:
                              ...
                              break;
                    default:
                              return (DefWindowProc(hWnd, message, wParam,
                                        lParam);
               }
               break;
          ...
          }
return (OL);
```

The low-order byte of *wParam* must be greater than any of the SC_* values.

8.6.5 Locking the System Menu

You can lock a system menu by trapping the system menu input to your application's window function. For example, in the following window function fragment, system menu input is trapped and ignored.

```
switch(message) {
case WM_SYSCOMMAND:
          switch(wParam) {
          case SC_KEYMENU:
          case SC_MOUSEMENU:
                    if (fNoMenus)
                              return;     /* DON'T call DefWindowProc */
          .....
          }
}
```

8.6.6 System Keys Without the Input Focus

To prevent keyboard input from being lost when no application has the input focus, Windows sends special no-focus system key messages to the active application when the user types at the keyboard. The no-focus messages, WM_SYSKEYUP, WM_SYSKEYDOWN, and WM_SYSCHAR, are similar to system key messages created when the user presses the ALT key, but the two forms are distinguished by bit 29 (0x20000000L) in the *lParam* parameter. If this bit is set, the message was created by the user pressing the ALT key. If not set, the message is a no-focus message.

Applications can use no-focus messages as accelerators. The **TranslateAccelerator** function treats a no-focus message as a normal key message. This means the application can process accelerator keys even if it does not have the input focus, such as when it is an icon.

8.6.7 System Menu Accelerators

Applications can associate keyboard accelerators with system menu commands. When the user presses such an accelerator, Windows sends a WM_SYSCOMMAND message to the application. For all other accelerators, Windows sends a WM_COMMAND message.

You can associate accelerators with system menu commands by creating an accelerator table in your resource script file that uses the system command values (SC_* constants) with the accelerator keys. The following example illustrates how to create an accelerator table that associates accelerators with the system menu commands:

```
acceltab ACCELERATORS
BEGIN
        ^M SC_MOVE
END
```

Note

Accelerators for regular menu items do not generate WM_COMMAND messages if the window is iconic.

Accelerators that are not associated with regular or system menu items generate WM_COMMAND messages, even if the the window is iconic.

8.7 Using the Mouse Cursor When No Mouse Is Available

Applications that want to use the mouse cursor to track keyboard motion can use the **SetCursor**, **SetCursorPos**, **GetCursorPos**, **ClipCursor**, and **ShowCursor** functions to display and move the cursor. When no mouse is available, it is the application's responiblity to display and move the cursor in response to keyboard actions. For example, an application can use the following code in its window function to the cursor shown when the window is active:

```
case WM_ACTIVATE:
if (!GetSystemMetrics(SM_MOUSEPRESENT)) {
    /* no mouse installed */
    if (!HIWORD(lParam)) {
        if (wParam) {
         SetCursor(...);
         SetCursorPos(...);
         }
    ShowCursor(wParam);
    }
if (wParam && !HIWORD(lParam))
    SetFocus(hwnd);
break;
```

Applications must be careful when using the mouse cursor when the system has no mouse installed. In general, applications must hide the cursor when the window is closed, destroyed, or relinquishes control. Failure to hide the cursor prevents subsequent windows from using the mouse cursor. For example, if an application sets the cursor to the hourglass, displays the cursor, then relinquishes control to a dialog box, the cursor remains on the screen (possibly in a new shape), but cannot be used by the dialog box.

8.8 Rules About Disk Files

To prevent problems with file operations, applications must not keep files open longer than it takes to write to the file. A Windows application should open the file, write to the file, then close it immediately. If an attempt to open a file fails, the application knows that the disk has been changed, and therefore can prompt the user to return the correct disk to the drive. To speed up this kind of disk I/O, applications should write information in large pieces.

To prevent problems, applications should observe the following rules about disk files:

Files must be left closed when not in use.
: Many applications require the user to frequently change disks to access different data. Problems occur when an application writing to a file on one disk attempts to write to that file after the disk has been changed. Since MS-DOS does not check to see if the same disk is in the drive before writing, an application can unwittingly write data on the wrong disk, possibly overwriting an existing file.

Application should limit the number of open files.
: Problems occur when applications have many open files when the system has only a fixed number available. Since applications can be invoked any number of times, the number of available open files can quickly be exhausted if all instances attempt to open files at the same time.

Applications should prompt for files that cannot be opened.
: Since disks are frequently changed, applications must not assume that inability to find a desired file is an unrecoverable error.

The **OpenFile** function can be used to help make file operations easier. It does full pathname creation, path searching, and automatic prompting for disk insertion. The following sections explain how to use **OpenFile**.

8.8.1 Creating and Using a File

You can use the **OpenFile** function to create a file. In your intialization code, the working file should be opened as follows:

1. Set *lpFileName* to point to a null-terminated filename.
2. Set *lpReOpenBuff* to point to a buffer having OFSTRUCT type.
3. Set *Style* to OF_CREATE.

8.8.2 Prompting for a File Before Opening

You can automatically prompt the user to insert the correct disk before opening a file by using the **OpenFile** function and the open style. Follow these steps:

1. Set *lpFileName* to point to a null-terminated prompt string. For example, if the string is "Word.doc," the prompt message is

   ```
   Insert Word.doc disk in drive x:
   ```

2. Set *lpReOpenBuff* to point to the buffer having OFSTRUCT type.
3. Set *Style* to your choice for the low byte. The upper byte should be a combination of OF_REOPEN and OF_PROMPT.

If you reopen a file read only, Windows will check that the date and time match the file when it was first opened.

8.8.3 Opening an Existing File

You can open an existing file by using the **OpenFile** function and the open style. In your intialization code, the existing file can be opened as follows:

1. Set *lpFileName* to point to a null-terminated filename.
2. Set *lpReOpenBuff* to point to a buffer having OFSTRUCT type.
3. Set *Style* to OF_EXIST and use the logical OR operator (|) to combine it with a low order byte of your choosing.

If the file fails to open, you can put up a dialog box indicating the file was not found, or use **OpenFile** to prompt for the file.

8.8.4 Creating Temporary Filenames

Since multiple instances of one application may be running at the same time, errors can result if an application uses the same names for temporary or scratch files. These names must be unique to each instance, otherwise, one instance is likely to overwrite the data generated by another. The **GetTempFilename** function can be used to create temporary filenames.

8.8.5 Filename Syntax

Applications should make sure that filenames contain no spaces. Under MS-DOS, a filename can have from 1 to 8 characters; a filename extension can have from 0 to 3 characters.

8.8.6 Filename Conversions

Applications that use the MS-DOS system calls to carry out actions on files must use the **AnsiToOem** function to convert the filenames to appropriate OEM characters before passing them to the system. Applications that use the **OpenFile** function do not need to convert filenames. **OpenFile** does it automatically.

Applications that use international filenames must be prepared to handle filenames containing characters with ANSI values above 127.

8.8.7 International Characters in Strings

Applications should use the **AnsiNext** and **AnsiPrev** functions to move forward and backward in a string. These functions correctly handle strings that contain characters that are not one byte in length, such as strings in machines that are using Japanese.

8.8.8 Finding Directory Names

Applications should use the following method to locate directory names from a string containing a full pathname:

1. Move to the end of the string by scanning for a NULL character.
2. Use the **AnsiPrev** function to move backwards through the string one ANSI character at a time.
3. Check for the "\" character.

When "\" is found, you have found the last character in the directory name.

8.8.9 Checking For File Existence

Applications that test for the existence of files should consider using the MS-DOS system calls instead of the **OpenFile** function if they do not want an overwrite message if the file exists, or do not want an int 24 error if a floppy disk drive has no disk in it.

The following example illustrates how to check for an existing file using C-language functions that implement the MS-DOS system calls.

```
if ((fh = _lopen((LPSTR)pchFileName, READ)) > -1) {
   _lclose(fh);
   PromptForOverWrite();
}
```

8.8.10 Displaying Errors When Files Are Open

Since an application should not relinquish control while it has open files, applications that need to display an alerts or errors with the **MessageBox** function should either make the message box system modal, or close the files before displaying the message box. If the message box is not system modal or the files are not closed, the user can move to another application taking control away from the application with open files.

8.8.11 TEMP Environment Variable

To print under Windows or carry out any function that creates temporary files, make sure that the TEMP environment variable contains a valid path-name. If the name is not valid, errors will result.

8.9 Using a Printer

This section explains how to send data to a printer from an application.

8.9.1 How to Print

Applications that print must use the **Escape** function to frame their output with STARTDOC and ENDDOC escape codes. Furthermore, each new page should begin with a NEWFRAME escape code.

The following example illustrates the correct way to send output to a printer:

```
Escape(hPr, STARTDOC, strlen(name), (LPSTR)name, 0);

for (i = 0; i < numberofpages; i++) {
        do_one_page();
        Escape(hPr, NEWFRAME, 0, (LPPROC) aborthandler, 0);
}

Escape(hPr, ENDDOC, 0, 0, 0);
```

8.9.2 How to Print Using Banding

Banding is a printing technique in which an image is printed by dividing it into several bands (or slices) and sending each band to the printer separately. Banding lets applications print complex graphics images without first creating the complete image in memory. This can reduce the memory requirements for printing and enhance system performance while printing operations are in effect. Banding can be used on any printing device that has banding capability.

To print using banding, follow these steps:

1. Use **GetDC** to retrieve a display context for the printer.
2. Use the **GetDeviceCaps** function to make sure the printer is a banding device.

   ```
   if(GetDeviceCaps(hPrinterDC, RASTERCAPS) & RC_BANDING)
           /* it's a banding device */
   ```

3. Use the **Escape** function and the NEXTBAND code to retrieve the coordinates of a band.

   ```
   Escape(hPrinterDC, NEXTBAND, 0, (LPSTR)NULL, (LPRECT)&tRect);
   ```

The function sets **tRect** to the coordinates of the current band. Coordinates are in device units, and all subsequent GDI calls are clipped to this rectangle.

4. Check **tRect** to see if it is an empty rectangle. The empty rectangle marks the end of the banding operation. If it is empty, terminate the banding operation.

5. Use the **DPtoLP** function to translate the **tRect** points from device units to logical units.

```
LPtoDP(hPrinterDC, (LPRECT)&tRect, 2).
```

6. Use GDI output and other functions to draw in the band. To save time, the application should carry out only those GDI calls that affect the current band. If an application does not wish to save time, GDI will clip all output that does not appear in the band, so no special action is required.

7. Repeat steps 4 through 6.

Once the banding operation is done, use the **DeleteDC** function to remove the printer display context.

8.9.3 Print Abort Function

Applications that print are required to pass an abort function to GDI to handle unusual situations during printing operations. The most common situation occurs when a printing operation causes disk space to be filled before the spooler can copy the data to the printer. Since the spooler can continue to print even though disk space is full, GDI calls the abort function to see if the application wishes to abort the print operation or simply wait until disk space is free.

An application sets the abort function by using the **Escape** function.

```
Escape(hDC, SETABORTPROC, 0, (FARPROC) myAbortHandler, 0)
```

GDI will then call the abort function during spooling. The function must have the form:

```
int FAR PASCAL myAbortHandler(hDC, code)
HDC hDC;
short code;
```

The **hDC** argument is a handle to the printer display context, and the **code** argument specifies the nature of the call. It can be one of the following:

SP_OUTOFDISK
: Spooler has run out of disk space while spooling the data file. The printing operation will continue if the application waits for disk space to become free.

0
: Spooler operation continuing without error.

Once the abort function has been called, it can return TRUE to to continue the spool operation immediately, or return FALSE to abort the printing operation. Most abort functions call the **PeekMessage** function to temporarily yield control, then return TRUE to continue the print operation. Yielding control typically gives the spooler enough time to free some disk space.

If the abort function returns FALSE, the printing operation is aborted and an error value is returned by the application's next call to the **Escape** function.

Important

If an application encounters a printing error or an aborted print operation, it must not attempt to terminate the operation by using **Escape** with ENDDOC or ABORTDOC. GDI automatically terminates the operation before returning the error value.

8.9.4 Aborting a Print Operation With ABORTDOC

You can use the **Escape** function and the ABORTDOC value to abort a print operation. In applications that do not have an abort function, ABORTDOC can be used to abort the operation at any time. In applications that have abort functions, ABORTDOC can only be used the first NEWFRAME or NEXTBAND call.

8.9.5 How to Abort a Print Operation

Applications should always give the user a chance to abort a printing operation. To do this, the application needs to create a modeless Abort dialog box when it begins a print operation, and to define an abort function that processes messages for application while the printing operation is in effect.

To create the dialog box and define the abort function, follow these steps:

1. Place the names of the abort procedure and the input function for the Abort dialog box under the EXPORT sections in your application's module definition file:

```
EXPORT
    fnAbortDlgProc  @7
    fnAbortProc     @8
```

2. Add an Abort dialog box template definition to your application's resource script file:

```
DTABORT     DIALOG 20,20,136,50
STYLE       WS_POPUP | WS_DLGFRAME | WS_VISIBLE
BEGIN
    DefPushButton "Cancel" IDCANCEL, 52, 28, 32, 14
    Ctext "Select Cancel to cancel printing", -1, 4,8, 128,12, WS_GROUP
END
```

3. Use the **MakeProcInstance** function to create a function instance address for each function for each instance of the application:

```
FARPROC lpfnAbortProc;
FARPROC lpfnAbortDlgProc;

lpfnAbortProc = MakeProcInstance(fnAbortProc, hAppIntance);
lpfnAbortDlgProc = MakeProcInstance(fnAbortDlgProc, hAppInstance);
```

4. Use the **Escape** function and the SETABORTPROC value to define the abort function to be used during the print operation.

5. Use the **CreateDialog** function to display the Abort dialog box, a modeless dialog box.

6. Use the **EnableWindow** function to disable your parent window.

7. Start the normal print operation, but check the return value from the **Escape** function after each NEWFRAME call. If the value is less than zero, the user has aborted the opertion or an error has occurred.

8. Use the **DestroyWindow** function to destroy the Abort dialog box if necessary. The box destroys itself if the user aborts the print operation.

The following example illustrates how to carry out a printing operation that the user can abort:

```
HWND hAbortDlgWnd;
int fAbort;

/* create the printer display context */
    hPrintDC = CreateDC( /* printer parameters */ );

/* clear the abort flag */
    fAbort = FALSE;

/* create the Abort dialog box (modeless) */
    hAbortDlgWnd = CreateDialog(hAppInstance, (LPSTR)DTABORT,
                        hAppWnd, lpfnAbortDlgProc);

/* disable the main window to avoid reentrancy problems */
    EnableWindow(hAppWnd, FALSE);

/* define the abort function */
    Escape(hPrintDC, SETABORTPROC, 0, lpfnAbortProc, (LPSTR)0);
    while (/* printing */) {
/* print one page at a time */

/* check Escape for error or abort */
        if (Escape(hPrintDC, NEWFRAME, 0, (LPSTR)NULL, (LPSTR)0) < 0)
            break;
    }

/* if the user did not abort the print */
    if (!fAbort) {
/* End the print operation */
        Escape(hPrintDC, ENDDOC, 0, (LPSTR)0, (LPSTR)0);
/* destroy the Abort dialog box */
        DestroyWindow(hAbortDlgWnd);
    }
/* reenable the main window */
    EnableWindow(hAppWnd, TRUE);
    DeleteDC(hPrintDC);
}
```

The abort function retrieves messages from the application queue and dispatches the messages if they are messages for the abort dialog box. The function continues to loop until the WM_QUIT message (generated by **DestroyWindow**) is encountered or the print operation is complete.

```
int FAR PASCAL fnAbortProc(hPrintDC, iReserved)
HDC hPrintDC;       /* for multiple printer display contexts */
int iReserved;      /* for future use */
{
    MSG msg;
```

```
/* process messages intended for the abort dialog box */
    while (PeekMessage((LPMSG)&msg, NULL, NULL, NULL, TRUE))
        if (!IsDialogMessage(hAbortDlgWnd, (LPMSG)&msg)) {
            TranslateMessage((LPMSG)&msg);
            DispatchMessage((LPMSG)&msg);
        }
/* fAbort is TRUE (return is FALSE) if the user has aborted */
    return(!fAbort);
}
```

The input function for the Abort dialog box processes the WM_INITDIALOG and WM_COMMAND messages. To let the user select the Cancel button with the keyboard, the function takes control of the input focus when the dialog box is initialized. It then ignores all messages until a WM_COMMAND message appears. Command input causes the function to destroy the window and set the abort flag to TRUE.

```
int FAR PASCAL fnAbortDlgProc(hwnd, msg, wParam, lParam)
HWND hwnd;
unsigned msg;
WORD wParam;
LONG lParam;
{
/* watch for Cancel button, RETURN key, ESCAPE key, or SPACE BAR */
    if (msg == WM_COMMAND) {
/* user has aborted operation */
        fAbort = TRUE;
/* destroy Abort dialog box */
        DestroyWindow(hwnd);
        return(TRUE);
    }
    else if (msg == WM_INITDIALOG) {
/* need input focus for user input */
        SetFocus(hwnd);
        return(TRUE);
    }
    return(FALSE);
}
```

8.9.6 Errors During Printing

Although, GDI and the Spooler application attempts to report all printing errors to the user, applications must be prepared to report out-of-disk and out-of-memory conditions.

Any time the **Escape** function returns a value less than 0, an error has occurred. If the SP_NOTREPORTED bit is clear, GDI has already notified the user. If set, the application needs to notify the user. The bit is typically set for general failure, out-of-disk space, and out-of-memory errors.

```
status = Escape(hPrDC, NEWFRAME, 0, (LPSTR)NULL, (LPSTR)0);
/* any errors? */
if (status < 0) {
          if (status & SP_NOTREPORTED) {
/* error occurred, but user doesn't know yet */
               switch (status) {
                    /* user cancellation */
               case SP_OUTOFDISK:
                    OutOfDiskAlert();
                    break;
               case SP_OUTOFMEMORY:
                    OutOfMemoryAlert();
                    break;
               default:
                    GeneralFailureAlert();
                    break
               }
          else
/* user has seen the error - we might care what it was */
               switch (status | SP_NOTREPORTED) {
               case SP_OUTOFDISK:
                    NoteOutOfDisk();
                    break;
               case SP_OUTOFMEMORY:
                    NoteOutOfMemory();
                    break;
               }
}
```

8.9.7 Setting Up the Printer

Applications can prepare a printer driver for operation with a particular printer and port by using the **DeviceModes** function for the driver. In this case, the application must retrieve a function address for **DeviceModes** by using the **GetProcAddress** function. The application can then use the address to set up the printer.

The following example illustrates how to access the DeviceMode function :

```
FARPROC lpfn;

lpfn = GetProcAddress( /* entry name */ );

(*lpfn) ( (HWND)hWnd,                    /* handle to parent window */
          (HANDLE)hModule,  /* handle to driver module */
          (LPSTR)szPrinter,              /* printer name */
          (LPSTR)szPort);   /* port name */
```

The parent window handle should identify the application's tiled window. The driver uses the module handle as its instance handle. The printer name is required if the driver can access more than one printer model. The port

name specifies the computer port to which the printer is connected. If no printer is connected, the port name should be set to the same as the printer name.

8.10 Miscellaneous Rules

This section lists a few special rules that Windows applications should observe.

8.10.1 Holding Handles

A handle is a unique, fixed identifier for a window, data block, module instance, or other object. Since a handle's value remains fixed throughout the lifetime of the object, an application can safely hold a handle in a global variable.

8.10.2 Word Aligned Stack

Windows requires a word aligned stack. Applications that alter the alignment of the stack can cause fatal errors. Old applications (non-Windows applications) that do not have word aligned stacks cannot be run in a window.

8.10.3 How Windows Displays Icons

When an icon is the active or selected icon, Windows places a white border or margin around it. Windows creates the margin by drawing a white rectangle, then drawing the icon inside the rectangle. The size of the rectangle is the same for all icons. If the icon is small the margin is large; if the icon is large, the margin is small.

Chapter 9
The Clipboard

9.1 Introduction 205
9.2 Copying Data to the Clipboard 205
9.3 Reading Data From the Clipboard 206
9.4 Rendering Data on Request 207
9.5 Rendering Formats Before Terminating 207
9.6 Standard Data Formats 208
9.7 The Clipboard Viewer Application 209
9.8 Controlling the Display of Data in the Clipboard Viewer 209
9.8.1 Displaying Private Datatypes as Standard Data Format Objects 209
9.8.2 Taking Full Control of the Clipboard Viewer Display 210
9.9 Clipboard Viewer Chain 211
9.9.1 Additional Actions Required of Chain Members 211
9.10 Metafile Pictures 212
9.10.1 Standards for Metafile Contents 213
9.10.2 Specific GDI Calls in Metafile Pictures 214
9.11 Standards for Playing Metafile Pictures 216

9.1 Introduction

The clipboard is the data exchange feature of Windows. It is a universally accessible pool of data handles through which applications can exchange formatted data.

From the user's point of view, the clipboard holds only one piece of data at a time, clearing out previous data whenever a new Cut or Copy operation is done. Internally, the clipboard holds any number of different data formats and corresponding data handles, all representing the same data but in as many different formats as the application is willing to supply.

For example, a pie chart might be held in the clipboard as both a metafile picture and a bitmap. An application pasting the pie chart would have to decide which representation it wanted to accept, following the general rule that the representation containing the most information would be the most desirable.

9.2 Copying Data to the Clipboard

To copy data to the clipboard, an application must:

1. Open the clipboard for alteration by calling the **OpenClipboard** function.
2. Clear out previously set data handles, and take ownership of the clipboard, by calling the **EmptyClipboard** function.
3. Pass data handles for each of the data formats the application wishes to supply to the clipboard using the **SetClipboardData** function.
4. Signal that it is done changing the clipboard by calling the **CloseClipboard** function.

Applications have the choice of formatting the data before sending it to the clipboard or waiting until a request for the data is made by another application.

To send formatted data to the clipboard, the application must:

1. Format the selected data using either a predefined or private format.
2. Allocate a block of global memory and copy the formatted data to it.
3. Pass the global memory handle and the format number to the clipboard using the **SetClipboardData** function.

To postpone the formatting until some application asks for it, the application must:

- Pass a NULL handle and the format number to the clipboard using the **SetClipboardData** function.

The application must be prepared to format the data on request. See the section "Rendering Data on Request."

9.3 Reading Data From the Clipboard

To read data from the clipboard, an application must:

1. Open the clipboard for examination by calling the **OpenClipboard** function.
2. Retrieve a data handle by calling the **GetClipboardData** function, specifying the format that is desired.
3. Copy the information in the handle to local storage, formatting it in a way that the application can use.
4. Signal that it is done examining the clipboard by calling the **CloseClipboard** function.

GetClipboardData returns NULL if no handle for the specified format exists in the clipboard. It is possible to determine in advance what formats are available in the clipboard by repeatedly calling the **EnumClipboardFormats** function.

A data handle returned by **GetClipboardData** is for temporary use only. It belongs to the clipboard, not to the application requesting data. Accordingly, it should not be freed or left locked down. The application should not rely on the handle remaining valid indefinitely. In general, the application should copy its contents and leave it alone after that.

9.4 Rendering Data on Request

Applications that know many data formats can save formatting time by passing NULL data handles to **SetClipboardData** instead of generating all of the data handles at the time of a Cut or Copy. The application does not actually have to generate a handle to the data until and unless someone requests a handle by calling **GetClipboardData**.

When **GetClipboardData** is called with a request for a format for which a NULL data handle has been set, a WM_RENDERFORMAT message will be sent to the clipboard owner.

When an application that has set a NULL data handle into the clipboard receives a WM_RENDERFORMAT message, it must:

1. Format the data that it last copied to the clipboard in the format requested (wParam is the format being requested).
2. Allocate a block of global memory and copy the formatted data to it.
3. Pass the global memory handle and the format number to the clipboard using the **SetClipboardData** function.

In order to accomplish the above steps, the application must keep information that tells itself what was last copied to the clipboard. The application may get rid of this information when it receives the WM_DESTROYCLIPBOARD message. The WM_DESTROYCLIPBOARD message is sent to the clipboard owner whenever the clipboard is emptied by a call to **EmptyClipboard**.

9.5 Rendering Formats Before Terminating

When an application is destroyed, the knowledge of how to render data that it has copied to the clipboard is also destroyed. Accordingly, a special message, WM_RENDERALLFORMATS, is sent when the application that owns the clipboard is being destroyed. Upon receiving this message, an application should follow the steps described in the section "Rendering Data on Request" for all formats that the application is capable of generating.

9.6 Standard Data Formats

Windows has the following predefined data formats for use in data interchange:

Format	Handle Contains
CF_TEXT	Null-terminated text
CF_METAFILEPICT	Metafile picture structure (see section on Metafile Pictures)
CF_BITMAP	A bitmap
CF_SYLK	SYLK standard data interchange format
CF_DIF	DIF standard data interchange format

In addition, an application may create and use private formats, or even new public ones. To create and use a new data interchange format, an application must:

1. Register the name of the new format by calling **RegisterClipboardFormat**.
2. Use the value returned by **RegisterClipboardFormat** as the code for the new format when calling **SetClipboardData**.

Registering the format name ensures that the application is using a unique format number. In addition, it allows the clipboard viewer application (CLIPBRD.EXE) to display the correct name of the data being held in the clipboard. For more information about displaying private data types in the clipboard viewer application, see the section "Controlling the Display of Data in the Clipboard Viewer."

If two or more applications register formats with the same name, they will all receive the same format code. This allows applications to create their own public data types. If two or more applications register a format called WORKSHEET, for example, they will all have the same format number when calling **SetClipboardData** and **GetClipboardData**, and will have a common basis for transferring WORKSHEET data between them.

9.7 The Clipboard Viewer Application

The clipboard viewer application, CLIPBRD.EXE, provides a way for the user to view the contents of the clipboard. It lists the names of all of the formats for which handles (NULL or otherwise) exist in the clipboard, and displays the contents of the clipboard in one of these formats.

The clipboard viewer knows how to display all of the standard data formats. If there are handles for more than one standard data format, the clipboard viewer will choose from the following list, in decreasing order of priority: CF_TEXT, CF_METAFILEPICT, CF_BITMAP, CF_SYLK, and CF_DIF.

9.8 Controlling the Display of Data in the Clipboard Viewer

There are two reasons why an application might wish to control the display of information in the clipboard viewer. The application may have a private data type that requires special knowledge to display. Alternatively, the application may have a data type which is private but which can be displayed in the clipboard as one of the standard types.

9.8.1 Displaying Private Datatypes as Standard Data Format Objects

This is the simplest way for an application to display a private data format. In order to use it, the desired display must be expressable as one of the data formats: text, bitmap, or metafile picture.

For example, an application might use the private data format "Psychological Evaluation" for transferring Rorschach test results. The internal format of the "Psychological Evaluation" data type might consist of numeric tables and so forth. When there is a "Psychological Evaluation" in the clipboard, the application might wish to show the company logo in the clipboard viewer (a bitmap), or a listing of the contents of the evaluation (text).

The following description assumes that the application has already followed the steps described in the section "Copying Data to the Clipboard" to take ownership of the clipboard and set data handles.

To force the display of a private data type in a standard data form, the application must:

1. Open the clipboard for alteration by calling **OpenClipboard**.
2. Create a global handle containing text, a bitmap, or a metafile picture, containing the information that should be displayed in the clipboard viewer.
3. Set the handle into the clipboard by calling **SetClipboardData**. The format code passed should be CF_DSPTEXT if the handle is to text, CF_DSPBITMAP for a bitmap, and CF_DSPMETAFILEPICT for a metafile picture.
4. Signal that it is done altering the clipboard by calling **CloseClipboard**.

9.8.2 Taking Full Control of the Clipboard Viewer Display

An application can take complete responsibility for the display and scrolling of information in the clipboard. This is useful when the application has a sophisticated private data type that only it knows how to display. WRITE uses this facility for displaying formatted text.

The following description assumes that the application has already followed the steps described in the section "Copying Data to the Clipboard" to take ownership of the clipboard and set data handles.

To take control of the display of information in the clipboard viewer, the application must:

1. Open the clipboard for alteration by calling **OpenClipboard**.
2. Call **SetClipboardData**, using CF_OWNERDISPLAY as the data format, with a NULL handle.
3. Signal that it is done modifying the clipboard by calling **CloseClipboard**.

Subsequently, the clipboard owner will receive special messages associated with the display of information in the clipboard. The messages are:

Message	User Action
WM_PAINTCLIPBOARD	Paint specified portion of window
WM_SIZECLIPBOARD	Take note of window size change
WM_VSCROLLCLIPBOARD	Scroll window vertically
WM_HSCROLLCLIPBOARD	Scroll window horizontally
WM_ASKCBFORMATNAME	Supply name of displayed format

Full descriptions of these messages can be found in the *Microsoft Windows Reference Manual.*

9.9 Clipboard Viewer Chain

The chaining of clipboard viewer windows provides a way for applications to specify that they will be notified whenever a change is made to the clipboard. The form of the notification is the WM_DRAWCLIPBOARD message, which is passed down the viewer chain whenever **CloseClipboard** is called. It is the responsibility of the recipient of WM_DRAWCLIPBOARD to determine the nature of the change (Empty, Set, etc.) by calling **EnumClipboardFormats**, **GetClipboardData**, etc., as desired.

9.9.1 Additional Actions Required of Chain Members

Any window that has made itself a member of the viewer chain must be prepared to:

1. Remove itself from the chain before it is destroyed.
2. Pass along WM_DRAWCLIPBOARD messages to the next member of the chain.

The code for this looks like:

```
case WM_DESTROY:
      ChangeClipboardChain( hwnd, my_save_next );
      /* rest of processing for WM_DESTROY */
      break;
case WM_DRAWCLIPBOARD:
      if (my_save_next != NULL)
            SendMessage( my_save_next, WM_DRAWCLIPBOARD,
                         wParam, lParam );
       /* rest of processing for WM_DRAWCLIPBOARD */
       break;
```

where "my_save_next" is the value returned from **SetClipboardViewer.** These clipboard viewer chain actions should be the first steps taken by the switch statement branches that process WM_DESTROY and WM_DRAWCLIPBOARD.

9.10 Metafile Pictures

The CF_METAFILEPICT format provides an orderly mechanism for passing pictures between applications. The type is an alternative to bitmaps, which are not readily scaled, and ordinary metafiles, which fail to provide certain highly useful pieces of information about the picture.

A data handle to a CF_METAFILEPICT object is a global handle to a structure of the following type:

```
typedef struct {
          int mm;
          int xExt, yExt;
          HANDLE hMF;
} METAFILEPICT;
```

The fields have the following meaings:

Field	Meaning
mm	This field gives the mapping mode in which the picture is drawn.
xExt, yExt	These fields give the size of the metafile picture for all modes except MM_ISOTROPIC and MM_ANISOTROPIC. The extext gives the box within which the picture is drawn. The coordinates are in units appropriate to the mapping mode. For MM_ISOTROPIC and MM_ANISOTROPIC modes, which are scalable, *xExt* and *yExt* contain an optional

suggested size in MM_HIMETRIC units. For MM_ANISOTROPIC pictures, *xExt* and *yExt* may be set to 0 if no suggested size is supplied. For MM_ISOTROPIC pictures, an aspect ratio must be conveyed even if a suggested size (which implicitly conveys an aspect ratio) is not given. To supply the aspect ratio without implying a suggested size, set *xExt* and *yExt* to negative values whose ratio is the appropriate aspect ratio. The magnitude of *xExt* and *yExt* do not matter; only the ratio of their absolute values.

hMF
: This field is a handle to the metafile. This should be a handle to a memory metafile, so that its contents can be copied by applications wishing to retain a private copy of the picture.

There are the following variants of metafile pictures:

1 MM_ANISOTROPIC
: May be scaled in both dimensions. The aspect ratio may be changed.

2 MM_ISOTROPIC
: May be scaled in one dimension. The aspect ratio does not change.

3 Absolute measurements (twips, inches, millimeters)
: May not be scaled.

4 MM_TEXT
: May not be scaled. Size is device-dependent.

9.10.1 Standards for Metafile Contents

The metafiles passed as part of metafile pictures must adhere to standards regarding their content. The recipient of a metafile picture has no way to verify whether these standards are being adhered to. An application which sets metafile pictures into the clipboard must guarantee that the standards appropriate to the picture type (mapping mode) are followed.

Here are some rules:

1. The player will set the mapping mode and viewport for the metafile picture to be played into. The metafile should therefore not contain calls to **SetViewportOrg** or **SetViewportExt**. The metafile may optionally contain calls to **SetMapMode**. **SetMapMode** calls in the metafile will generally not be necessary, because the metafile player sets the map mode specified in the METAFILEPICT structure prior to playing the metafile.

2. The metafile picture must set the window origin and extent for MM_ISOTROPIC and MM_ANISOTROPIC picture types. It may optionally set the window origin for other picture types. Setting the window extent for other picture types is not a supported Windows feature.

3. So that the player can achieve display of selected portions of the metafile picture by setting the clip region and viewport, metafile pictures may not use the **SetClipRgn** or **OffsetRgn** calls. The **IntersectClipRect** and **ExcludeClipRect** calls may be used. The metafile picture may achieve clipping effects by **SaveDC**, then one of the allowed calls, then drawing operations into the clipped area, and finally the **RestoreDC** to restore the previous clip region.

9.10.2 Specific GDI Calls in Metafile Pictures

There are additional restrictions on particular GDI calls used within metafile pictures. The following calls can be used in any metafile picture:

MoveTo	GetCurrentPosition
LineTo	PolyLine
Rectangle	RoundRect
Polygon	Ellipse
Arc	Pie
FloodFill	InvertRect
TextOut	PatBlt

Since regions are stored in device-dependent units, any metafile picture using the following calls has a device-dependent size and therefore has type MM_TEXT.

PaintRgn
InvertRgn
FrameRgn
FillRgn

Although the following calls may be used by any metafile picture, they use pattern brushes which are device-dependent.

FrameRect
FillRect

The following calls may be used by any metafile picture. **SaveDC** and **RestoreDC** calls should be exactly balanced within a metafile; that is, there should be as many save as restore calls, and each restore must restore a DC that was saved within the metafile.

SaveDC
RestoreDC

The following call is device-dependent and may have drastic side effects. It is not permitted in any metafile picture.

Escape

The following calls work in logical units and can be used in any type of metafile picture.

SetTextCharacterExtra
SetTextJustificationExtra

In the following calls, the first two are permitted, the second two are not. The metafile may set clip regions that will be a subset of the one set by the player, but may not override the player's clip region.

ExcludeClipRect
IntersectClipRect
SelectClipRgn
OffsetClipRgn

Both of the following calls must be called within MM_ ISOTROPIC and MM_ ANISOTROPIC metafile pictures before any drawing is done. **SetWindowOrg** may optionally be used within MM_ TEXT and exact measurement metafile pictures.

SetWindowOrg
SetWindowExt

In most cases, there should be no need for the map mode to be set within a metafile picture; the metafile picture player sets the map mode before playing the picture. **SetMapMode** is permitted in metafile pictures, however, so that special effects involving mixing of map modes may be achieved.

The following calls work in device units, and represent the alteration by the metafile of its permitted drawing area on the screen. They are prohibited.

SetViewportOrg
SetViewportExt

The following calls set DC drawing attributes and do not have any device-dependence or scaling issues involved. They may be used in any type of metafile picture.

SetPolyFillMode
SetStretchBltMode
SetROP2
SetColorTable
SetBkColor
SetTextColor
SetBkMode
SetRelAbs

The **SelectObject** function can be used in any metafile picture, but care should be taken when selecting device-dependent objects such as bitmaps and pattern brushes.

The **BitBlt** function can be used in metafile pictures having the MM_TEXT type only. Since **BitBlt** does not scale, it introduces a device-dependent component to the size of the picture. The **StretchBlt** function should be used in place of **BitBlt** in scalable or absolute measurement metafile pictures. **StretchBlt** can be used in any metafile picture.

9.11 Standards for Playing Metafile Pictures

In order to receive the benefits of the standardization provided by metafile pictures, the recipient of a metafile picture must adhere to certain rules when playing the metafile. By following these rules, it is possible for the player to:

- Scale MM_ISOTROPIC and MM_ANISOTROPIC pictures
- Display selected portions of the picture, clipping what is not desired

The following is the sequence that should be followed for playing a metafile picture.

1. Call **SaveDC** to save the properties of the DC into which the picture will be played. (optional)
2. Compute the size, in pixels, of the rectangle enclosing the intended display.
   a. For MM_ANISOTROPIC pictures, this is a matter of arbitrary choice (unless one is following the suggested size).
   b. For MM_ISOTROPIC pictures, this is true as well, but the player may wish to take the aspect ratio into account. An MM_ISOTROPIC picture will always play into a rectangle with its given aspect ratio, even if this must be obtained by subsetting the region provided to it.
   c. For exact measurement pictures, the display size must be computed from the *xExt* and *yExt*, and from the ratio of exact measurement units to pixels for the device, as obtained from **GetDeviceCaps**.
   d. For MM_TEXT pictures, the display size is exactly the *xExt* and *yExt* provided.
3. Set the clip region so as to show the desired portion of the picture.
4. Set the map mode to the value given in the *mm* field of the picture.
5. Set the viewport origin so that the desired portion of the picture will appear in the clip region.
6. For MM_ISOTROPIC pictures, set the window extent to {xExt, yExt}. Although the metafile will itself set the window extent, setting it here tells **SetViewportExt** the shape (i.e., aspect ratio) of the rectangle into which the picture will play. This is important because **SetViewportExt** in MM_ISOTROPIC mode gives the viewport the same aspect ratio as the window, even if it must subset the passed extents to accomplish this.
7. For MM_ISOTROPIC and MM_ANISOTROPIC pictures, set the viewport extent to be the size determined in step 1. Do not set the viewport extent for other picture types.
8. Play the metafile.
9. Restore the DC.

Appendixes

A Creating Windows Libraries 221
B Creating A Font Resource 225
C GDI Definitions 233

Appendix A
Creating Windows Libraries

A.1 Windows Libraries 223
A.2 Initializing the Local Heap For Libraries 223
A.3 Calling Routines in a Library 224

A.1 Windows Libraries

A library module is an .EXE file that is not directly executable. For example, the GDI.EXE and USER.EXE files are both library files. A library module contains routines that can be linked to an application during execution rather than when the application is compiled and linked. A library also uses its own data segment rather than the application's.

To make a library module, follow these steps:

1. Write the library source code.
2. Create a module definition file that uses the LIBRARY statement instead of the NAME statement. Also, include an EXPORT statement for each routine in the library to be exported.
3. Compile the library source file.
4. Use the **Link4** command to create the .exe file. You must specify the SWINLIBC.LIB or MWINLIBC.LIB file in the command line, for small or medium model libraries respectively. This special file is used in place of the standard SWLIBC.LIB or MWLIBC.LIB files. The SLIBW.LIB or MLIBW.LIB libraries should be specified after SWINLIBC.LIB or MWINLIBC.LIB. It is used by **Link4** to create a library module.
5. Use the **Implib** command to create the "dummy" .LIB file. Any application that uses procedures in the library must specify this .LIB file in the **Link4** command line when linking the application.

A.2 Initializing the Local Heap For Libraries

If the library allocates memory from the local heap and therefore uses the HEAPSIZE statement in the module definition file, the initialization procedure must call the **LocalInit** routine to initialize the local heap. For example, the call

```
LocalInit(0, 0, HeapSize);
```

correctly initializes the heap for subsequent use.

When a library's initialization procedures is first called, Windows passes the following information to it:

```
DI = hInstance
ES:SI = not used
DS = Data Segment value
CX = Heap Size
```

The heap size value in the CX can be used in the **LocalInit** call.

A.3 Calling Routines in a Library

To call a procedure in a Windows library, an application just makes a normal procedure call. Windows loads the corresponding library, if it is not already loaded, and completes the call to the procedure.

Appendix B
Creating A Font Resource

B.1 Font Resources 227
B.2 Creating Font Files 227
B.3 Adding Fonts to Executable Files 227
B.4 Creating Font Resource-Only Files 228
B.4.1 Creating the Dummy Code Module 229
B.4.2 Creating the Module Definition File 229
B.5 Compiling and Linking the Font Resource 231

B.1 Font Resources

GDI maintains a pool of physical fonts which are available to all applications. Each physical font is a collection of character bitmaps that have a unique combination of height, width, facename, character set, and other attributes. An application uses a font to display text on a display surface.

A font resource is a group of individual fonts representing characters in a given character set that have various combinations of heights, widths, and pitches. For example, the system font resource contains the ANSI character set and the OEM character set used by the Windows system. Font resources can be added or deleted as needed from GDI's pool using the **AddFontResource** and **RemoveFontResource** routines. An application can have up to 253 entries in the system font table.

B.2 Creating Font Files

Before creating a font resource, you need one or more font files. You can create font files by using the **Fontedit** program described in the *Windows Programmer's Utility Guide.* You can also create files from scratch using the font file format given in the appendix, "Font Files," in the *Microsoft Windows Reference Manual.* The number, size, and type of font files in a font resource is up to you. In most cases, you should include enough fonts to reasonably satisfy most logical font requests for display devices having common aspect ratios. Common ratios are 1 to 1, 1.5 to 1, and 2 to 1. When planning font sizes, remember that GDI can scale device-independent raster fonts by 1 to 8 vertically and 1 to 5 horizontally. GDI can also simulate bold, underlined, strikeout, and italic fonts. Although scaled or simulated fonts do not look as nice as actual fonts, they can save valuable memory resources.

B.3 Adding Fonts to Executable Files

Once you have the font files, the next step is to create a font resource by adding the font files to a Windows library, a device driver, or a resource-only file. (A resource-only file contains icons, cursors, fonts, and other resources but no code). You should not add fonts to application modules, since an application's resources are available to the application only, which defeats the purpose of the font resource.

You add the resources to the file by using the **FONT** statement of the resource compiler, **Rc**. That is, you place these statements in the resource script file of the executable module you are creating. The statement has the form:

number FONT *filename*

One statement is required for each font file to be placed in the resource. The *number* must be unique since it is used to identify the font later. The following is a typical resource script file for a font resource:

```
1   FONT FntFil01.FNT
2   FONT FntFil02.FNT
3   FONT FntFil03.FNT
4   FONT FntFil04.FNT
5   FONT FntFil05.FNT
6   FONT FntFil06.FNT
```

Fonts can be added to modules which contain other resources by merely adding them to the existing resource script. This means you can have icon, cursor, menu, and dialog box definitions in the resource script file as well as FONT statements.

B.4 Creating Font Resource-Only Files

To create a resource-only module that contains only fonts, you must:

1. Create a dummy code module.
2. Create a module definition file that describes the fonts and the devices that use the fonts.
3. Compile and link the sources.

B.4.1 Creating the Dummy Code Module

The dummy code module is used to provide the object file from which the resource file is made. The following assembly code illustrates the dummy code module:

```
        TITLE   FONTRES - Stub file to build FONTRES.EXE

.xlist
include cmacros.inc
.list

sBegin  CODE

sEnd    CODE

end
```

B.4.2 Creating the Module Definition File

The module definition file for the dummy executable should have the form:

```
LIBRARY FontRes

DESCRIPTION 'DDRV ...'
            .
            .
            .
DESCRIPTION 'FONTRES ...'
            .
            .
            .

STUB 'WINSTUB.EXE'

DATA NONE
```

The format for the DESCRIPTION statements in a font resource are as follows:

DESCRIPTION 'DDRV *DeviceTypeList* **:** *FontTypeList* **:** *descriptive-text*'
DESCRIPTION 'FONTRES *FontTypeList* **:** *descriptive-text*'

The *DeviceTypeList* is one of the following:

DISPLAY
DeviceTypeItem. [, *DeviceTypeItem*] ...

where *DeviceTypeItem* is a device type recognized by the device. This is second parameter to the **CreateDC** function.

The *FontTypeList* is one or more of the following, separated by semicolons (;):

Aspect , *LogPixelsX*, *LogPixelsY*
CONTINUOUSSCALING
DEVICESPECIFIC *DeviceTypeGroup*

Aspect is the value (100**AspectY*)/*AspectX* rounded to an integer.

AspectX, *AspectY*, *LogPixelsX*, and *LogPixelsY* are the same values as in the device's GDIINFO structure (values accessible using the **GetDeviceCaps** function).

Note

The maximum length of a DESCRIPTION line is 127 characters.

To avoid loading unneeded fonts, it is recommended that each font resource contain only one aspect ratio or resolution.

Examples

```
DESCRIPTION 'DISPLAY : 240,96,48 : IBM Color/Graphics Adapter'
DESCRIPTION 'DDRV Epson FX-80 : 83,60,72; 120,72,60 :.EPSON 60 X 72 dpi'
DESCRIPTION 'DDRV HP 7470A, HP 7475A : : HP 7470 series plotters'

DESCRIPTION 'FONTRES 133,96,72 : System, Terminal (Set #3)'
DESCRIPTION 'FONTRES 200,96,48; 133,96,72; 83,60,72; 167,120,72 : Helv'
DESCRIPTION 'FONTRES CONTINUOUSSCALING : Modern, Roman, Script'
DESCRIPTION 'FONTRES DEVICESPECIFIC HP 7470A, HP 7475A: HP 7470 plotters'
```

B.5 Compiling and Linking the Font Resource

The following make file lists the commands required to compile and link the resource-only file:

```
fontres.obj: fontres.asm
    masm fontres;

fontres.exe: fontres.def fontres.obj fontres.rc fontres.exe \
                FntFil01.FNT FntFil02.FNT FntFil03.FNT \
                FntFil04.FNT FntFil05.FNT FntFil06.FNT
    link4 fontres.obj, fontres.exe, NUL, /NOD, fontres.def
    rc fontres.rc
    command /c rename fontres.exe fontres.fon
```

By convention, all font resource files have the .FON filename extension. The last line in the make file renames the executable file to FONTRES.FON.

Appendix C
GDI Definitions

C.1 Coordinate Systems 235
C.2 Transformation Equations 235
C.3 GDI Window and Viewport Functions 237
C.4 Mapping Modes 237
C.5 Completely Constrained 238
C.6 Partially Constrained: MM_ISOTROPIC 239
C.7 Unconstrained: MM_ANISOTROPIC 239
C.8 When to Use the Mapping Modes 239
C.9 GDI Transformation Defaults Values 240
C.10 Logical Screen Size 240
C.11 Default System Font 241
C.12 ANSI Fixed Font 242
C.13 Selecting Fonts 242
C.14 How the Font Mapper Works 243
C.14.1 Scaling Device Drivers 245

C.1 Coordinate Systems

To maintain device independence in graphic operations, GDI uses an internal or logical coordinate system to draw all images before mapping them to the actual or physical coordinate system of the display device.

The logical coordinate system is a conceptual system that exists within GDI. It consists of x and y axes, has its origin at the center, and has maximum coordinate values ranging from -32,768 to 32,767.

The physical coordinate system is the system used by individual device drivers to access the picture elements (pixels) of the display surface. GDI requires that a physical coordinate system has x and y axes, and has its origin at the upper left corner of the display surface. The x axis must increase to the right and the y axis must increase down the display surface.

To draw images on the display surface of a device, an application uses a GDI output function to draw in the logical coordinate system. GDI then maps the image to the desired device. The mapping consists of taking the individual points of the image in the logical coordinate system and placing them on the display surface as pixels. The location of the pixel depends on the transformation equations that define the mapping. The mapping result is an image on the display surface that represents the image drawn by the application.

C.2 Transformation Equations

The transformation equations define the mapping of individual points in the logical coordinate system to points in the physical coordinate system. GDI uses the transformation equations in all mappings from logical to physical spaces.

The equations contain the following 12 variables:

Variable	Meaning
xWO	the x coordinate of the window origin
yWO	the y coordinate of the window origin
xWE	the x component of the window extent
yWE	the y component of the window extent
xVO	the x coordinate of the viewport origin
yVO	the y coordinate of the viewport origin
xVE	the x component of the viewport extent
yVE	the y component of the viewport extent
Lx	the x coordinate in the logical coordinate system
Ly	the y coordinate in the logical coordinate system
Dx	the x coordinate in the physical coordinate system
Dy	the y coordinate in the physical coordinate system

The purpose of the transformation equations is to compute either a device point (Dx, Dy) from a logical point (Lx, Ly) or a logical point (Lx, Ly) from a device point (Dx, Dy). To this end, values for the window and viewport variables are set, a logical or device point is selected, and a new point is computed.

The transformation equations are:

```
Dx = (Lx - xWO)*xVE/xWE + xVO
Dy = (Ly - yWO)*yVE/yWE + yVO

Lx = (Dx - xVO)*xWE/xVE + xWO
Ly = (Dy - yVO)*yWE/yVE + yWO
```

The first pair of equations computes a device point from a logical point. The second pair computes a logical point.

In each equation, the ratio of extents is called the scale and determines the amount of stretching to be done. For example, the scale in the first equation is xVE/xWE. The subtraction and addition of the window and viewport origins is called the translational component of the equation and determines the amount of sliding to be done.

Note

There is no implied clipping at the viewport as in many other graphics systems. Clipping is done through a separate interface.

C.3 GDI Window and Viewport Functions

The eight variables which define the transformation can be set to absolute values using the **SetWindowOrg**, **SetWindowExt**, **SetViewportOrg**, and **SetViewportExt** functions. Their current values can be retrieved using the **GetWindowOrg**, **GetWindowExt**, **GetViewportOrg**, and **GetViewportExt** functions. The values can be modified relative to current values using the **ScaleWindowExt**, **ScaleViewportExt**, **OffsetWindowOrg**, and **OffsetViewportOrg**.

Note

All the viewport functions use device units (pixels).

All the window functions use logical coordinates.

For example, if you wish to slide a picture over by a few pixels you should offset the viewport origin. If you want to slide it over by a few logical units, offset the window origin.

C.4 Mapping Modes

The mapping modes are accelerators for setting the window and viewport extents into commmon configurations. You set the mapping mode by using the **SetMapMode** function. This is equivalent to changing the scale component of the transformation equations. **SetMapModes** never changes the origins of the window or viewport.

Some mapping modes prevent changes to the extents. For example, after setting the mapping mode to MM_ TEXT, all calls to **SetWindowExt** and **SetViewport** are ignored.

The mapping modes fall into three classes:

Completely constrained
Partially constrained
Unconstrained

C.5 Completely Constrained

The following mapping modes are completely constrained:

MM_ TEXT
MM_ LOMETRIC
MM_ HIMETRIC
MM_ LOENGLISH
MM_ HIENGLISH
MM_ TWIPS

In these modes, all changes to window or viewport extents are ignored. The MM_ TEXT mapping, the default mapping mode for display contexts, is completely constrained.

The purpose of all constrained modes, except MM_ TEXT, is to provide a way to draw undistorted images in fixed physical units, such as inches or centimeters. These modes guarantee that a one by one square in logical coordinates is mapped in physical coordinates to one by one square in the desired units on any device. The MM_ TEXT mode provides a way to draw images using knowledge about the display surface. In this mode, each logical unit is mapped to a single device pixel. Unlike the other constrained modes, MM_ TEXT does not guarantee constant images on different devices.

The modes are constrained to ensure that the scale cannot be changed unless the mode is changed. Thus inches remain inches and centimeters remain centimeters. The modes, except MM_ TEXT, assume right-handed coordinate systems, which means that x increases to the right and y increases going up. The MM_ TEXT mode assumes a left-handed coordinate system with x increasing to the right and y increasing going down.

C.6 Partially Constrained: MM_ISOTROPIC

In the MM_ISOTROPIC mode, you can change the scale but the ratio between scales for computing the x and y coordinates is kept constant at a value that keeps the x and y units the same. This means that squares in logical units come out as squares on the device. Since an arbitrary change to window or viewport extents does not maintain this constant ratio, GDI always adjusts the viewport extent by making it smaller in one dimension. Thus, if you adjust the window extent, GDI shrinks the viewport in one direction to make it match the window in shape. If you adjust the viewport in both x and y extents, GDI changes one of these values to make it match the window. The amount of change depends on the requested values and the aspect ratio of the device.

C.7 Unconstrained: MM_ANISOTROPIC

In the MM_ANISOTROPIC mode, you are free to set the window and viewport extents to any values. The mode directs GDI to set an internal flag to indicate that extents can be changed, but unlike other modes, the current extents are left unchanged. You must use the **SetWindowExt SetViewportExt** functions to set the extents to desired values if you do not wish to use the previous values. For example, if you use the MM_LOMETRIC mode and then set the mode to MM_ANISOTROPIC, your coordinate system at that moment will be the same as defined by MM_LOMETRIC.

C.8 When to Use the Mapping Modes

You can use the other completely constrained modes when the application wishes to work in physical units, or where the actual size is to be fixed. For example, if you need to create rectangles that are 1 cm by 2 cm, use the MM_LOMETRIC mode.

You can use the MM_ISOTROPIC mode to ensure a 1:1 aspect ratio when actual size is not important. For example, use this mode if you need to draw a square in logical coordinates and have it come out as a square of any size on the output device.

You can use the MM_ANISOTROPIC mode when the x and y coordinates may be independently adjusted. For example, use the mode when you want to map a square in logical coordinates to any size rectangle on the output device.

You can use the MM_TEXT mode when you need to draw using the individual pixels of the output device, such as in justifying text, or using bitmap animation.

C.9 GDI Transformation Defaults Values

Whenever you retrieve a display context for a particular device, it has the following default transformation values:

Mapping mode	MM_TEXT
Window origin	(0,0)
Window extents	(1,1)
Viewport origin	(0,0)
Viewport extent	(1,1)

This default setting maps one logical unit onto one device unit. Since there is no clipping associated with the transformation, points that lie outside the window will be mapped to points outside the viewport and thus the entire device may be addressed.

C.10 Logical Screen Size

Every system display has a logical screen size in which an arbitrary width and height in inches is assigned to the display. This logical width and height, usually different from the physical width and height, represents the actual width or heigth a full screen image will have when printed on another device. The logical screen size is used by applications when writing text or drawing graphics to make better use of the screen's resolution.

GDI expresses the logical screen size in terms of pixels per logical width. An application can retrieve these values by using the **GetDeviceCaps** function. Since most word processing applications work with paper that is 8.5 inches across and assume one inch margins on the left and right, the

logical width of most system displays is 6.5 inches and the number of pixels per logical inch in the X direction for the display is equal to the total number of physical horizontal pixels divided by 6.5. A system display's logical height is usually a function of the device's aspect ratio, but may be arbitrarily selected to keep common fonts legible. To prevent roundoff errors in conversions from pixels to points, both the number of pixels per logical inch in both the X and Y directions are rounded to divisors of 1440. This corresponds to the twip (twentieth of a point), which at 1/1440 inch is the smallest unit of measure in GDI.

For example, the logical screen size of the IBM Color Graphics Adaptor (640x200 pixels) is 6.5 by 4 inches. The pixels per logical inch in the X and Y directions are 96 and 48, respectively. The logical screen size of the IBM Enhanced Graphics Adaptor (640x350 pixels) is 6.5 by 5 inches. The pixels per logical inch in the X and Y directions are 96 and 72, respectively.

Not all applications should use the logical screen size. For example, to display graphics in exact physical sizes, applications should use the MM_LOENGLISH, MM_HIENGLISH, MM_LOMETRIC, or MM_HIMETRIC map mode. To display undistorted graphics (for example, perfectly round circles), applications should use an MM_ISOTROPIC map mode with the X extents of the viewport set to the display's logical width.

C.11 Default System Font

Windows uses the default system font, ANSI_SYSTEM_FIXED, to draw text in menus, captions, and buttons. The font is designed for best appearance on the system display, and has features that may not be suitable for printer output, such as wide stems to make it more readable when greyed.

GDI selects this font by default when you retrieve a display context. Applications that do not care about the appearance of text when copied from a screen to a printer can use the default system font for all text operations. This means the application never needs to select a font.

The font is in fixed memory and therefore is always available for text operations.

C.12 ANSI Fixed Font

Applications that wish to present text on the screen as it may appear when printed should use the ANSI_FIXED font. This font is designed to represent a 10 pitch Courier font, a font common to many printers. To use the font, the application retreives a handle to it using the **GetStockObject** function, then selects it using the **SelectObject** function. The ANSI_FIXED font is available as a stock object.

Although the ANSI_FIXED font looks like a 10 pitch Courier on the screen, it may not have the same appearance when printed on an actual printer. In other words, GDI does not guarantee that ANSI_FIXED maps too the given printer's 10 pitch Courier. To use the printer's 10 pitch Courier, an application should create and select that font.

In systems without sufficient memory, the ANSI_FIXED font and the ANSI_SYSTEM_FIXED font are identical.

C.13 Selecting Fonts

To maintain device-independence, GDI lets applications choose a logical font in which to draw text. It then maps this logical font to a physical font that can be displayed on the device's display surface. Although an application can choose a wide variety of logical fonts, there is typically a limited set of physical fonts for a given device. The role of the GDI font mapper is to select, by a process of elimination, the physical font that most closely matches the requested logical font.

Applications can create a logical font by using the **CreateFont** or **CreateFontIndirect** functions. These create a data structure, held internally by GDI, that will be used to select a physical font.

Applications direct GDI to choose a physical font by selecting the logical font for a display context using the **SelectObject** function. GDI invokes the font mapper which immediately examines all available physical fonts for the given device.

C.14 How the Font Mapper Works

GDI cannot guarantee that a physical font exists that exactly matches a requested logical font, so the font mapper attempts to pick a font that has the fewest differences from the requested logical font. Since fonts have many different attributes, GDI uses a penalty scale to determine which differences are more important than others.

To begin the mapping, GDI transforms the requested height and width of the logical font to device units. This transformation depends on the current mapping mode and window and viewport extents. GDI then asks the device to realize the physical font. A device can realize a font if it can create it or a font very close to it.

If the device can realize a physical font, GDI compares this font with its own set of fonts. If GDI has a font that more closely matches the logical font, GDI uses it. But if the device signals that it can take device-realized fonts only, GDI uses the realized font.

If the device cannot realize a font, GDI searches its own fonts for a match.

To determine how good a match a given physical font is to the requested logical font, the mapper takes the logical font and compares it one attribute at a time with each physical font in the system. The mapper checks the following attributes:

Attribute	Result
character set	The wrong character set is penalized highly. Fonts with the wrong character set are very rarely selected as the physical font. There is no default character set. This means a logical font must always specify the desired set.
pitch	The wrong pitch is penalized heavily. If the requested pitch is fixed, a wrong pitch is assessed a greater penalty since an application that handles fixed pitches may not be able to handle variable-pitch fonts.
facename	The wrong facename is penalized heavily. If the user asks for default facename, no penalty is assessed.
family	The wrong font family is penalized heavily. If the user asks for default family, no penalty is assessed.

height	The wrong height is penalized. GDI always chooses or synthesizes a shorter font if the exact height is not available. GDI can synthesize a font by expanding a font's character bitmaps by an integer multiple. GDI will expand a font a maximum of eight times. If a default height is requested, GDI arbitrarily searches for a twelve point font.
width	The wrong width is penalized. GDI always chooses or synthesizes a narrower font if the exact width is not available. If a default width is requested, GDI assesses a penalty for any difference between the aspect ratio of the device and the aspect ratio of the font. The mapper can give unexpected results if there are no fonts for the given aspect ratio.
bold	Although GDI can synthesize bold, an actual bold font is preferred. The mapper penalizes for synthesizing.
italics	Although GDI can synthesize italics, an actual italic font is preferred. The mapper penalizes for synthesizing.
underline	Although GDI can synthesize underlining, an actual underline font is preferred. The mapper penalizes for synthesizing.
strikeout	Although GDI can synthesize strikeouts, an actual strikeout font is preferred. The mapper penalizes for synthesizing.

If GDI synthesizes a font, the mapper assesses a penalty that depends on the number of times the font was replicated. Furthermore, a penalty is added if the font was synthesized in both directions and the synthesizing was uneven, that is, if the font was stretched more in one direction than in the other.

When the mapper has compared all the fonts in the system, it picks the one with the smallest penalty. The application should retrieve the metrics to find out the characteristics of the font it received.

Note

The mapper only selects fonts from those currently in the system. If an application wants the mapper to consider fonts that have not been loaded, it must load them using the **AddFontResource** function. An application can determine which fonts are in the system by using the **EnumFonts** function.

GDI assumes that the height and width of a logical font are in logical units. If an application changes the mapping mode, window extent, or viewport extent after selecting a font, GDI automatically attempts to remap the font using the new height and width transformations.

C.14.1 Scaling Device Drivers

Some devices perform graphics at one resolution and text at another. To give applications a uniform resolution in which to carry out text and graphics operations, GDI support scaling devices. The device driver of a scaling device scales all graphics operations (that is, compressed using **StretchBlt**) before mapping them to the device display surface.

Scaling is always carried out automatically and is always a reduction (to a lower resolution). The amount of scaling depends on the device's scaling factor. An application can retrieve the scaling factor by using the GETSCALINGFACTOR code with the **Escape** function. The scaling factor is a power of two. Thus, a factor of 2 means reduce by 4, a factor of 1 means reduce by 2.

You can find out if a device is a scaling device by using the **GetDeviceCaps** function and checking the RC_SCALING capability.

Programming Guide Index

ABORTDOC code, 196
Accelerators
  command short cuts, 60
  command values, 60
  definitions in menus, 60
  in the system menu, 188
  key names, 60
  loading, 61, 107
  resource definitions, 60
  table, 60
  translated value, 61
  translating, 61, 107
  with dialog boxes, 173
ALT-TAB key, 113
AnyPopup function, 136
Applications
  basic design, 32
  code segment, 26
  creating, 28
  data segment, 26
  instance, 16
  loading, 122
  module definition file, 99
  name, 26

Background mode
  TRANSPARENT, 40
  OPAQUE, 57
Banding, 194
BeginPaint function, 138
BM_ GETCHECK message, 152
BM_ GETSTATE message, 152
BM_ SETCHECK message, 152
BM_ SETSTATE message, 152
BN_ CLICKED code, 151
BN_ DISABLE code, 151
BN_ HILITE code, 151
BN_ PAINT code, 151
BN_ UNHILITE code, 151
BringWindowToTop function
  showing child windows, 134
  showing popup windows, 136
Brushes
  background, 19
Brushes *(continued)*
  creating, 3
  selected, 57
BUTTON class, 149, 152
Button controls, 149

C compiler, 4
C programming language, 4
Calling convention, 17
CallWindowProc function
  subclassing, 145
CANCEL key, 172
Caption bar, 10, 130
Caret, flashing for text, 83
CF_ BITMAP format, 208
CF_ DIF format, 208
CF_ DSPBITMAP format, 210
CF_ DSPMETAFILEPICT format, 210
CF_ DSPTEXT format, 210
CF_ METAFILEPICT format, 208, 212
CF_ OWNERDISPLAY format, 210
CF_ SYLK format, 208
CF_ TEXT format, 160, 208
ChangeMenu function, 181
Character width, 38
CheckDlgButton function, 152
Child window, 131
Class
  brush, 19
  cursor, 19
  icon, 19
  instance, 19
  menu, 19
  name, 19
  registration, 18
  style, 19
  window function, 19
Client area
  application output, 130
  background, 130
  child window, 131
  coordinate system, 55
  defined, 11
  described, 130

Client area *(continued)*
display context, 130
initial mapping mode, 140
origin, 56
painting, 130
screen location, 113
size, 114
Client coordinate system, 56
Clipboard
copying data from, 206
copying data into, 205
copying from edit controls, 160
copying to edit controls, 160
creating new formats, 208
cutting from edit controls, 160
data sharing, 121
described, 205
edit controls, 161
metafile picture contents, 213
rendering all formats, 207
rendering data, 207
setting display format for private formats, 210
sharing private formats, 208
standard data formats, 208
synchronizing tasks, 122
taking full control of CLIPBRD display, 210
using metafile pictures, 212
viewer application, 209
viewer chain members, 211
CLIPBRD
controlling display, 209
described, 209
displaying private formats, 209
taking full control, 210
viewer chain, 211
Clipping region, window, 140
Close command, 123, 124
Code segments, moveable, 121
Computer graphics, 3
CONTROL statement, 152, 162
Control window, 83. *See* Controls
Controls
automatic three-state check box, 151
button
described, 150
messages, 151, 152
check box, 150
default pushbutton, 150
described, 83, 149
Controls *(continued)*
edit, 149
additional messages, 160
automatic scrolling, 156
copying to the clipboard, 160
cutting to the clipboard, 160
described, 153
editing keys, 155
entering text, 155
fixed-pitch font, 155
formatting rectangle, 159
formatting text, 156
height, 154
messages, 158, 159
multiline, 153
multiple, 157
pasting from the clipboard, 160
printing text, 156
redrawing, 157
restricted messages, 160
scroll bars, 158
selection, 159
styles, 153
tab stops, 155
text buffer, 159, 160
text limit warning, 155
using, 153
word wrapping, 156, 157
group box, 151, 152
ID, 97
list boxes
changing, 166
control class, 149
creating, 166
described, 162
error return values, 166
messages, 163
sending messages, 166
space limitations, 166
styles, 163
predefined, 93
push buttons
defined, 150
designing, 153
radio button, 150
resource definitions, 97
retrieving values, 96
scroll bars, 149
alignment, 168
described, 167
notification messages, 167

scroll bars, 149 *(continued)*
setting thumb position, 168
setting thumb range, 168
styles, 167
setting values, 94
static, 149
as edit control labels, 154
creating icon controls, 162
described, 161
styles, 161
styles, 168
three-state check box, 151
user button, 151
Coordinate system, client area, 55
CreateDialog function, 170
CreateFont function, 242
CreateFontIndirect function, 242
CreateWindow function
creating a child window, 132
creating a tiled window, 131
creating edit controls, 154
CS_ CLASSDC style, 143
CS_ DBLCLKS style, 116
CS_ OWNDC style, 143
Cursors
class, 19
creating, 3
I-Beam for text, 83
using without a mouse, 188

Data areas
multiple, 120
segments, 120
Data exchange, 3
Data interchange, 161, 205
Data segments
locking, 177
multiple, 120
stack, 121
Data
borrowing from a previous instance, 77
sharing, 121
Dead key, 117
Default action, 16
DefWindowProc function, 185
DestroyWindow function
destroying modeless dialog boxes, 170
effect, 112
Device coordinate system. *See* physical coordinate system
Devices, scaling, 245
Dialog boxes
accelerators, 173
caption text, 171
controls, 94
creating, 93, 169
default responses, 172
described, 93, 169
exiting, 96
exporting the input function, 99
framed border, 172
initializing, 94
input function
instance address, 93
requirements, 169, 170
input messages, 171
keyboard input, 173
memory requirements, 170
menu input, 171
messages, 95
modal, 170
modeless, 170
resource definitions, 97
resource script file, 169
return values, 172
scrolling, 171
starting, 170
template, 97, 169
units of measure, 169, 171
using, 93, 170
DialogBox function
instance address, 122
starting a dialog box, 170
DialogEnd function, for modal dialog boxes, 170
Directory names, 165, 192
Disk files, usage rules, 190
Display context
attributes, 55
cache limit, 144
class, 55
class type, 143
client area, 130
default font, 241
default type, 143
described, 142
mapping mode, 55
modifying, 55
private, 55

Display context *(continued)*
private type, 143
releasing default type, 143
releasing private type, 144
resetting, 143
saving, 76
type, 142
updating
class type, 143
default type, 143
private type, 143
usage requirements, 142
viewport, 56
window, 56
Display screen, 4
Double click, 116
Drawing mode, overwriting, 57
Dynamic linking, 180

EDIT class, 149
Edit controls, 149, 153
Ellipse, drawing, 58
EM_ FMTLINES message, 156
EM_ GETHANDLE message, 160
EM_ GETLINE message, 156
EM_ GETLINECOUNT message, 156, 160
EM_ GETRECT message, 159
EM_ GETSEL message, 159
EM_ LIMITTEXT message, 155, 157
EM_ LINEINDEX message, 160
EM_ LINESCROLL message, 160
EM_ SETHANDLE message, 159
EM_ SETRECT message, 159
EM_ SETRECTNP message, 159
EM_ SETSEL message, 159
EN_ CHANGE code, 158
End Session command, 115, 124
ENDDOC code, 194
EndPaint function, 139
EN_ ERRSPACE code, 158
EN_ HSCROLL code, 158
EN_ KILLFOCUS code, 158
EN_ SETFOCUS code, 158
EN_ VSCROLL code, 158
ES_ AUTOHSCROLL style, 154
ES_ AUTOVSCROLL style, 154
Escape function
aborting, 196
printing, 194
Escape function *(continued)*
setting abort function, 195
ESCAPE key, 172
ES_ CENTER style, 153
ES_ LEFT style, 153
ES_ MULTILINE style, 153, 157
ES_ RIGHT style, 153
EXEC system call, 122
Execution stub, 26
Export, 9
EXPORT
definitions, 99
statement, 223

Far function, 17
File I/O, 4
File specification string, 165
Files
checking for existence, 193
creating, 191
directory names, 192
displaying errors while open, 193
international characters, 192
module definition, 9
name conversions, 192
name syntax, 192
opening an existing file, 191
prompting for files, 191
resource script, 9
source program, 9
TEMP variable, 193
temporary names, 192
usage rules, 190
FONT statement, 228
Font table, 227
Fonts
10 point Courier, 242
adding, 227
adding to libraries, 227
adding to resource-only files, 227
changing, 179
creating, 92, 227
default system, 41, 241
deleting, 227
described, 41, 227
files, 227
fixed-pitch, 41, 242
physical, 92
resource-only files, 228
scaled sizes, 227

Fonts *(continued)*
selected, 38, 41
selecting, 92
simulated attributes, 227
FONTTEST application, 81
Functions
call-back, 121
exported, 9, 27
far, 17, 121
imported, 9
instance address, 93, 121
local, 17, 37
main, 12, 16, 103
main loop, 107
message filter, 110
multiple instances, 121
name syntax, 180
Pascal, 121
printing abort, 195
window, 12, 16, 22, 103, 111
WinMain, 12

GDI
coordinate systems, 235
device point, 235
font mapper, 242
graphics functions, 139
implied clipping, 237
logical coordinate system, 235
logical font, 242
logical point, 235
logical screen size, 240
mapping modes, 237
mapping, described, 235
physical coordinate system, 235
physical font, 242
pixels, 235
transformation, 236, 237
transformation equations, 235
viewport extents, 235
viewport origin, 235
window extents, 235
window origin, 235
GetClassLong function, 45
GetDataInstance function, 120
GetDC function, 138, 143
GetKeyState function, 117
GetMessage function, 122
GetProcAddress function, 180
GETSCALINGFACTOR code, 245
GetSystemMenu function
changing system menu, 185
getting default system menu, 185
restoring default system menu, 186
GetSystemMetrics function, 154
GlobalLock function, 178
Graphics Display Interface (GDI). *See* GDI
Group box, 152

Handles, 201
HEAPSIZE statement, 223
HELLO application, 9
HIWORD macro, 112

Icon area, 4
Iconedit, 3, 28
Iconic, 5
Icons
class, 19
creating, 3
display, 201
Image mapping, 56
Implib program, 223
Import, 9
Input focus
activating, 31
from keyboard, 116
getting, 38
in dialog boxes, 170
messages without, 187
Input
control, 117
dead key, 117
double click, 116
key repeat count, 116
keyboard, 113, 115, 116
menu, 117
mouse, 115
mouse button, 115
mouse movement, 115
scroll bar, 117
system keys, 187
timer, 117
virtual keys, 116
Instance address, 121
Instances
borrowing data, 77
destroying, 120

Instances *(continued)*
first, 119
handle, 42, 104
module, 21
multiple, 30, 119
multiple data segments, 26
new, 16
previous, 104
prolog, 120
Interactive windows, 3

Keyboard input
ANSI characters, 106
foreign language characters, 106
making available to window, 38
OEM characters, 106
translation, 106
Keyboard
ANSI values, 116
dead key, 117
umlaut keys, 117
virtual keys, 116
with dialog boxes, 173

LB_ADDSTRING message, 164
LB_DELETESTRING message, 164
LB_DIR message, 165
LB_ERR return value, 166
LB_ERRSPACE return value, 166
LB_GETCOUNT message, 165
LB_GETCURSEL message, 165
LB_GETSEL message, 164
LB_GETTEXT message, 165
LB_GETTEXTLEN message, 165
LB_INSERTSTRING message, 164
LB_RESETCONTENT message, 164
LB_SELECTSTRING message, 165
LB_SETCURSEL message, 164
LB_SETSEL message, 164
LBS_MULTIPLESEL style, 163
LBS_NOREDRAW style, 163
LBS_NOTIFY style, 163
LBS_SORT style, 163
Libraries
calling to, 224
creating, 223
described, 223
dummy .LIB file, 223
initializing local heap, 223
Libraries *(continued)*
medium model for windows, 105
MLIBW.LIB, 105
register values, 224
sharing data, 121
SLIBW.LIB, 105
small model for windows, 105
LIBRARY statement, 223
Lines
drawing, 58
Link4 program, 4, 223
Linking, dynamic, 180
List box controls, 149, 162
ListBox class, 149
LISTBOX statement, 166
Local functions, 37
Local heap, 18, 27
LocalInit function, 223
Logical coordinates
function parameters, 57
system, 56, 235
LOWORD macro, 112

Main function
calling, 119
described, 12
Main program loop, 123
MakeProcInstance
address, 93
function, 121
Mapping mode
defined, 237
effect of changing window and viewport, 57
orientation of axes, 56
text, 140
units computed, 56
units of measure, 56
Medium model, 105
Memory management
tips, 177
Memory
changing discardable, 178
discardable, 178
discarded, 178
freeing, 20, 177
local allocation, 18
local heap, 18
locking, 177
locking data segment, 177

Menus
accelerator keys, 60
changing by command value, 181
changing by position, 181
checking items, 53
class default menu, 60
compiling, 59
described, 130
handle, 21
in applications, 59
item names, 59
item values, 59
messages, 53
name defined, 19
popup, 130
resource definitions, 59, 97
Message boxes
default responses, 174
described, 137
with open files, 193
MessageBox function, 137
Messages
activation, 113
ANSI character translation, 106
ANSI keys, 116
application defined, 118
application queue, 103, 105
bypassing the queue, 105
checking the queue, 108
closing, 112
command, 117
confirming termination, 124
correct usage, 109
creation, 38
default action, 16
default processing, 22
described, 105, 111
destroy message, 112
dispatching, 22, 105, 106
edit controls, 159
filter function, 110
filtering class messages, 111
generated by applications, 106
input, 115
input events, 105
input focus, 113
keyboard input, 106
list box controls, 163
local, 118
menu commands, 53
mouse button, 115
mouse movement, 115
moving, 113
no-focus, 187
OEM character translation, 106
painting, 114, 139, 140
parameters, 53, 111
peeking, 108
posting, 109
posting virtual key messages, 107
pulling, 106
pushing, 106
reading, 16, 22, 105, 106
reading without pulling, 108
registered, 119
removal limitations, 109
removing, 109
reserved, 118
restrictions, 109
scroll bar, 117
sending, 109
sending virtual key messages direct, 107
session ending, 115
sizing, 113, 140
special actions, 105
system commands, 123
termination, 112
translating, 22, 106
types, 118
virtual keys, 106, 116
waiting, 22
window creation, 112
window functions, 22, 105
window state, 112
Message timer, 117
METAFILEPICT structure, 212
MLIBW.LIB library, 105
MM_ANISOTROPIC mapping mode, 56, 73, 239
MM_HIENGLISH mode, 238
MM_HIMETRIC mode, 238
MM_ISOTROPIC mode, 239
MM_LOENGLISH mode, 238
MM_LOMETRIC mode, 238
MM_TEXT mode, 238
MM_TWIPS mode, 238
Module definition file, 4, 9, 25
Mouse
cursor, 188
input, 75

Mouse *(continued)*
input capture, 76
tracking, 75
MS-DOS EXEC system call, 122
MS-DOS Executive, 12, 115, 124, 131, 169
MS-DOS file attribute value, 165
MWINLIBC.LIB library, 223
MWLIBC.LIB library, 105

NEWFRAME code, 194
NEXTBAND code, 194
Notification codes, 158
Notification messages
list boxes, 163
scroll bar controls, 167

OEM interface, 3
OpenFile function, 190

Painting
asynchronous messages, 140
at any time, 138
background, 142
background erased, 139
by request, 138
clipping region, 140
colors, 142
described, 138
immediately, 139
initial mapping mode, 140
invalidating the entire window, 53
modifying the display context, 55
releasing a modified display context, 55
validating the painting area, 139
validating the window, 54
PAINTSTRUCT structure, 114
Parent window, 20
Pascal calling conventions, 180
PatBlt function, 75
Patterns, drawing, 75
PeekMessage function, 122
Pens, selected, 57
Physical coordinate system, 56, 235
Pixels, 235
Popup window, 134
Portability, 3
PostQuitMessage function, 120, 123
Prefix string, 165
Printers, set up, 200
Printing
Abort dialog box, 196
abort function
codes, 195
return values, 196
aborting, 196
described, 194
reporting errors, 199
spooler, 195
Program loop, 16

Queues
application, 22, 103, 105
checking, 108
hardware, 22

Rc, resource compiler, 4
RC_SCALING capability, 245
Rectangles, drawing, 57
RegisterWindow function, 129
ReleaseDC function, 143
Repeat count, 116
Resizing, 4
Resource script file, 9, 27
Resource script, dialog box template, 169
Resource-only files, creating, 228
Resources
definition, 59
sharing, 3
storage, 180
tips, 179
RETURN key, 172

SB_CTL argument, 168
SBS_BOTTOMALIGN style, 167
SBS_HORZ style, 167
SBS_LEFTALIGN style, 167
SBS_RIGHTALIGN style, 167
SBS_TOPALIGN style, 167
SBS_VERT style, 167
Scroll bar
controls, 149, 167
described, 130
thumb, 118

ScrollBar class, 149
Scrolling, 118
Selected font, 38
SelectObject function, 242
SendMessage function, 166
Session, ending, 115
SETABORTPROC code, 195
SetClassLong function, 111
SetMapMode function, 140
SetTimer function, 117
SetViewportExt function, 140
SetViewportOrg function, 140
SetWindowHook function, 110
SetWindowLong function
  for dialog box return values, 172
  subclassing, 145
SetWindowText function
  caption bar, 130
  setting dialog box caption, 171
ShowWindow function, 131
Size box, 10, 130
SLIBW.LIB library, 105
Small model, 105
Source program, 9
SS_BLACKFRAME style, 162
SS_BLACKRECT style, 162
SS_CENTER style, 161
SS_GRAYFRAME style, 162
SS_GRAYRECT style, 162
SS_ICON style, 162
SS_LEFT style, 161
SS_RIGHT style, 161
SS_USERITEM style, 162
SS_WHITEFRAME style, 162
SS_WHITERECT style, 162
Stack, 27, 121, 201
STARTDOC code, 194
STATIC class, 149
Static controls, 149, 161
Static variables, 37
Storage, temporary allocation, 18
STYLE statement, 172
Subclassing, windows, 144
SWINLIBC.LIB library, 223
SWLIBC.LIB library, 105
System colors, background, 142
System keys, 187
System menu
  accelerators, 188
  adding commands, 186
  box, 130
System menu *(continued)*
  changing, 185
  default, 185
  described, 10
  locking, 187
  messages, 185
  tips, 185

Tasks
  communicating, 122
  described, 119
  module loading, 122
  multitasking, 119
  priority, 119
  resource sharing, 119
  sharing program code, 119
  synchronizing, 122
  yielding control, 106, 119, 122
TEMP variable, 193
Termination
  application, 123
  Windows session, 123
Text
  external leading, 92
  height, 92
  metrics, 92
  writing, 21, 40, 92
Tiled window, 10, 130
Tiling, 4
Timer, 117
TranslateAccelerator function
  translating system keys, 188
  usage, 61, 107
TranslateMessage function, 106, 116

UpdateWindow function
  painting immediately, 139
  requesting painting, 138
User profile, changing, 179

ValidateRect function, canceling
    painting, 139
Viewport
  alternate definition, 57
  effect of changing extents, 57
  effect of window, 56
  extents, 74
  extents and units, 56

Viewport *(continued)*
orientation of axes, 56
origin, 56
physical coordinate system, 56
Virtual keys, 106, 116
Visual interface, 3

Wildcard characters, 165
Windows applications
application queue, 105
C language, 4
calling libraries, 224
data area, 119
data segments, 120
described, 4, 103
development, 3
dispatching messages, 106
end session prompt, 115
executive function, 103
filter function, 110
first instance, 119
instance handle, 104
medium model, 105
multiple instances, 119
previous instance handle, 104
pulling messages, 106
pushing messages, 106
reading messages, 106
small model, 105
stack, 110, 121
starting, 119
tasks, 119
terminating, 104, 120, 123
termination tasks, 124
window functions, 103
yielding control, 106, 108, 122
Windows
active, 113
alternate definition, 57
background, 114, 130, 142
background brush, 128
caption bar, 10, 127, 130
caption text, 20
child
close box, 133
described, 131
ID, 132
input, 132
messages, 133
moving, 133
child *(continued)*
overlapping, 134
painting, 134
parent relationship, 133
parent window, 132
redirected messages, 133
showing, 132
sizing, 133
class
attributes, 128
background brush, 128, 130
cursor, 128
described, 128
double click style, 116
function, 128
icon, 128
menu, 129
module, 128
name, 129
registering, 12, 18, 129
style, 128
usage, 129
client area, 11, 127
clipping region, 140
control icons, 127
creating, 12, 20
delayed paint messages, 140
described, 127
dialog box, 169
displaying, 21
effect of changing viewport extents, 57
effect of viewport, 56
effect of zooming on hidden windows, 141
erasing the background, 114
extents and units, 56
full-screen, 4
full-size, 4
functions, 12, 22, 103, 106, 111
hidden, 141
icon, 127
iconic, 5
include file, 17
input and output capabilities, 127
input focus, 30, 113
keyboard input, 116
logical coordinate system, 56
main, creating, 112
mapping mode, 140
menu, 127, 130

Windows *(continued)*
messages, 12, 103
multiple instances, 127
open, 127
orientation of axes, 56
origin, 56
painting, 16, 21, 114, 138
parent
described, 132
redirected messages, 133
popup
creating, 135
described, 134
message box, 137
messages, 135
overlapping, 136
painting under, 136
parent relationship, 136
parent window, 134
redirected messages, 135
showing, 135
window class, 135
preparing for termination, 115
queues, 12
resizing, 4
scroll bars, 127, 130
session end, 115, 123
size box, 10, 130
sizing, 140
sizing requirements, 141
styles, 20, 137
subclassing, 144
system menu, 10
system menu box, 130
tiled
creating, 131
showing, 131
updating, 21
WM_ERASEBKGND message, 114
WM_MOVE message, 113
WM_PAINT message, 114
WM_SIZE message, 114
zooming, 141
WINDOWS.H, 17
WIN.INI file, changing, 179
WinMain
defined, 12
exit statement, 104
function, 104
main loop, 107
WM_ACTIVATE message, 113
WM_ASKCBFORMATNAME message, 211
WM_CHAR message, 116
WM_CLOSE message, 112, 123, 124
WM_COMMAND message, 117, 151, 172
WM_COPY message, 160
WM_CREATE message, 112
WM_CUT message, 160
WM_DEADCHAR message, 116
WM_DESTROY message, 112, 123
WM_DESTROYCLIPBOARD message, 207
WM_DRAWCLIPBOARD message, 211
WM_ENDSESSION message, 115, 124
WM_ERASEBKGND message, 114, 142
WM_FONTCHANGE message, 179
WM_HSCROLL message, 117, 130, 167
WM_HSCROLLCLIPBOARD message, 211
WM_INITDIALOG message, 170, 171
WM_INITMENU message, 185, 186
WM_KEYDOWN message, 116
WM_KEYUP message, 116
WM_KILLFOCUS message, 113
WM_LBUTTONDBLCLK message, 115
WM_LBUTTONDOWN message, 115
WM_LBUTTONUP message, 115
WM_MOUSEMOVE message, 115
WM_MOVE message, 113
WM_PAINT message, 114, 130, 138, 140, 142
WM_PAINTCLIPBOARD message, 211
WM_PASTE message, 160
WM_QUERYENDSESSION message, 115, 124
WM_QUIT message, 112, 123
WM_RBUTTONDBLCLK message, 115
WM_RBUTTONDOWN message, 115
WM_RBUTTONUP message, 115
WM_RENDERALLFORMATS message, 207
WM_RENDERFORMAT message, 207
WM_SETFOCUS message, 113
WM_SETREDRAW message, 157, 163, 166

WM_SHOWINDOW message, 141
WM_SIZE message, 113, 140, 143
WM_SIZECLIPBOARD message, 211
WM_SYSCHAR message, 185
WM_SYSCOMMAND message, 123, 185
WM_SYSKEYDOWN message, 185
WM_SYSKEYUP message, 185
WM_TIMER message, 117
WM_VSCROLL message, 117, 130, 167
WM_VSCROLLCLIPBOARD message, 211
WM_WININCHANGE message, 179
Work area, 4
WS_BORDER style, 138, 154
WS_CAPTION style, 133, 138
WS_CHILD style, 132, 137
WS_CHILDWINDOW style, 138
WS_CLIPCHILDREN style, 137
WS_CLIPSIBLINGS style, 137
WS_DISABLED style, 137
WS_DLGFRAME style, 138, 172
WS_GROUP style, 168
WS_HSCROLL style, 138, 154
WS_ICONIC style, 137
WS_POPUP style, 135, 137
WS_POPUPWINDOW style, 138
WS_SIZEBOX style, 138
WS_SYSMENU style, 133, 138
WS_TABSTOP style, 168
WS_TILED style, 137
WS_TILEDWINDOW style, 131, 138
WS_VISIBLE style, 137, 163
WS_VSCOLL style, 138
WS_VSCROLL style, 154

Yield function, 122

Zooming, effect on hidden windows, 141